

Overview

Overview

Relevant source files

- [README.md](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/README.md> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/README.md>))
- [pom.xml](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml>))
- [src/main/java/com/techx/tradex/realtime/Application.java](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java>))

This document provides a high-level introduction to the tradex-realtime repository, a Spring Boot application designed for real-time processing of financial market data. The system operates as a Kafka consumer that processes various types of market updates and maintains fast-access caches and persistent storage for trading applications.

This overview covers the system's purpose, architecture, and key components. For detailed configuration management, see [Configuration System](#). For in-depth coverage of the real-time data processing pipeline, see [Real-time Data Processing](#). For deployment and operational details, see [Deployment and Operations](#).

Purpose and Scope

The tradex-realtime service serves as a critical component in the broader TradEx trading ecosystem, responsible for:

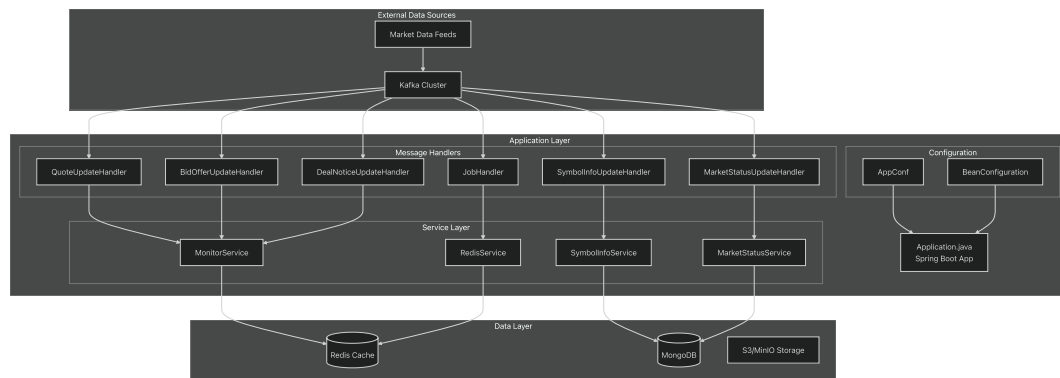
- **Real-time Market Data Processing:** Consuming and processing market data streams from Kafka topics including quotes, bid/offer updates, market status changes, deal notices, and symbol information
- **Multi-tiered Data Management:** Maintaining data across Redis for fast access, MongoDB for persistence, and S3/MinIO for archival storage
- **Performance Monitoring:** Tracking system performance and statistics for operational observability
- **Administrative Operations:** Handling job requests and market initialization tasks

The application is built on Spring Boot 3.1.1 with Java 17 and integrates with custom trading libraries to provide specialized financial market data processing capabilities.

System Architecture

The following diagram illustrates the overall system architecture and component relationships:

TradEx Realtime System Architecture



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L1-L34>

[src/main/java/com/techx/tradex/realtime/Application.java1-34](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L1-L34) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L1-L34>

[pom.xml1-191](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L1-L191) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L1-L191>

Technology Stack

The application leverages the following core technologies and dependencies:

Component	Technology	Version	Purpose
Application Framework	Spring Boot	3.1.1	Core application framework
Runtime	Java	17	Runtime environment
Message Streaming	Apache Kafka	2.3.1	Real-time data streaming
Caching	Redis	-	Fast data access layer
Database	MongoDB	-	Persistent data storage
Object Storage	AWS S3/MinIO	-	Data archival
JSON Processing	Jackson	-	Data serialization

Key Dependencies

The system integrates several custom libraries that provide domain-specific functionality:

- `market-common-v2` : Core market data models and utilities
- `kafka-common` : Kafka integration and message handling utilities
- `redis-dao` : Redis data access objects and operations
- `tradex-common-java` : Shared TradEx ecosystem components

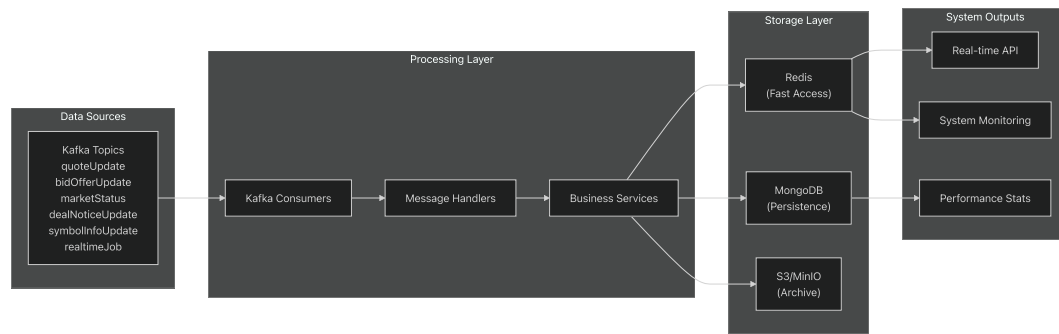
Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L27-L149>

[pom.xml27-149](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L27-L149) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L27-L149>

Data Processing Pipeline

The following diagram shows the high-level data flow through the system:

Data Processing Flow



Sources: Based on architectural diagrams provided

Core Components

Application Bootstrap

The main application class (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/%60Application.java%60>)

`Application.java` (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/%60Application.java%60>), serves as the Spring Boot entry point and configures:

- **Component Scanning:** Scans packages `com.difisoft.market` and `com.techx.tradex.realtime`
- **MongoDB Repositories:** Enables repositories with `SymbolInfoRepository`
- **Redis Integration:** Imports `RedisDao` and `MarketRedisDao` beans
- **Configuration Properties:** Enables `AppConf` configuration class
- **Scheduling:** Enables async processing and scheduled tasks

Configuration Management

The system uses externalized configuration through the `AppConf` class, which manages:

- Kafka topic configurations
- Feature toggles and system parameters
- Database connection settings
- Monitoring and notification configurations

For detailed configuration information, see [Application Configuration](#).

Message Processing

The application processes multiple types of market data through specialized handlers:

- Quote updates and recoveries
- Bid/offer spread data
- Market status changes
- Deal notices and advertisements
- Symbol information updates
- Administrative job requests

Each handler integrates with appropriate service layers for business logic processing. For comprehensive coverage of message processing, see [Real-time Data Processing](#).

Data Management

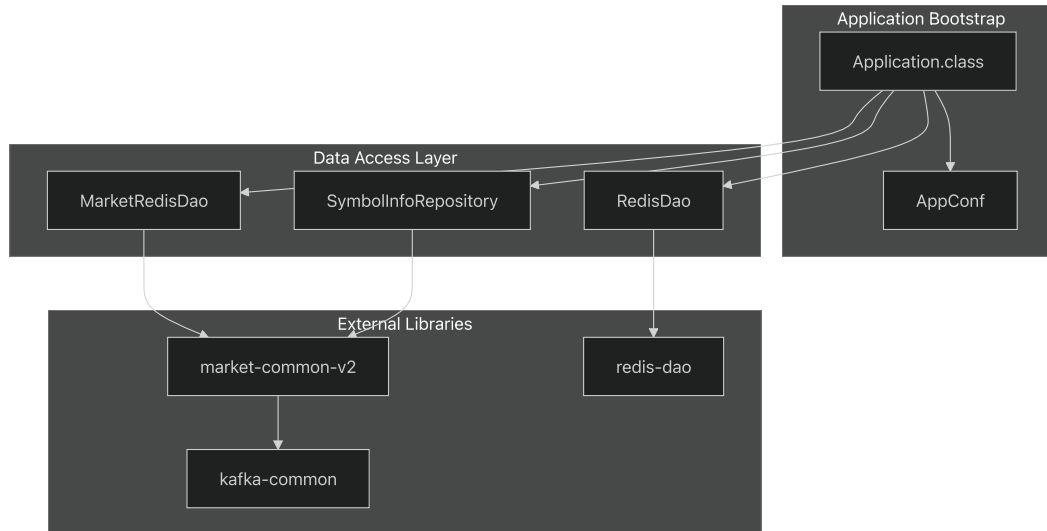
The system employs a three-tier data architecture:

1. **Redis:** High-performance caching for real-time access

2. **MongoDB**: Persistent document storage for complex data structures
3. **S3/MinIO**: Long-term archival and bulk data storage

Service Architecture

Core Service Dependencies



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L19-L25>

[src/main/java/com/techx/tradex/realtime/Application.java#L19-L25](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L19-L25) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L19-L25>

pom.xml#L61-L75 <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L61-L75> <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L61-L75>

System Integration

The `tradex-realtime` service operates as part of a larger financial technology ecosystem, integrating with:

- **Upstream Systems**: Market data providers and trading platforms that publish to Kafka
- **Downstream Systems**: Trading applications and analytical tools that consume processed data
- **Infrastructure**: Container orchestration and monitoring systems for deployment and operations

For deployment architecture and operational considerations, see [Docker Deployment](#) and [Service Orchestration](#).

The system maintains high availability and performance through containerized deployment, comprehensive monitoring, and robust error handling mechanisms built into each processing component.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/README.md#L1-L6>

[README.md#L1-L6](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/README.md#L1-L6) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/README.md#L1-L6>

pom.xml#L11-L12 <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L11-L12> <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L11-L12>

System Architecture

Relevant source files

- [pom.xml](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml> <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml>

- [

src/main/java/com/techx/tradex/realtime/Application.java](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java>))

Purpose and Scope

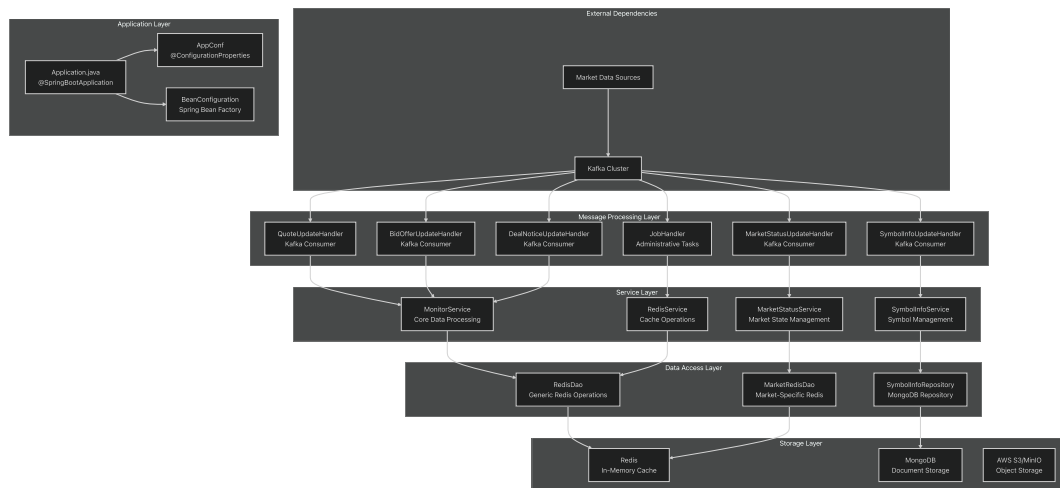
This document provides a comprehensive overview of the system architecture for the tradex-realtime application, a real-time market data processing system built on Spring Boot. It covers the high-level architectural components, technology stack, dependency relationships, and data flow patterns that form the foundation of the system.

For detailed information about configuration management, see [Configuration System](#). For specifics on real-time data processing components, see [Real-time Data Processing](#). For deployment and operational concerns, see [Deployment and Operations](#).

High-Level System Architecture

The tradex-realtime system follows a layered, event-driven architecture designed for high-throughput market data processing. The system integrates multiple external data sources through Kafka streams and maintains real-time state through a multi-tier storage strategy.

Core System Components



Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L1-L34>).

[src/main/java/com/techx/tradex/realtime/Application.java1-34](#) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L1-L34>)

pom.xml27-149](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L27-L149> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L27-L149>))

Technology Stack Integration

The system leverages a comprehensive technology stack organized into distinct functional layers:

Category	Technology	Version	Purpose
Framework	Spring Boot	3.1.1	Application framework and dependency injection
Runtime	Java	17	Runtime environment
Messaging	Apache Kafka	2.3.1	Event streaming and message processing
Caching	Redis	-	High-performance in-memory data store
Database	MongoDB	-	Document-based persistent storage
Object Storage	AWS S3/MinIO	8.3.7	Archive and backup storage
Build	Maven	-	Dependency management and build automation

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L20-L25>

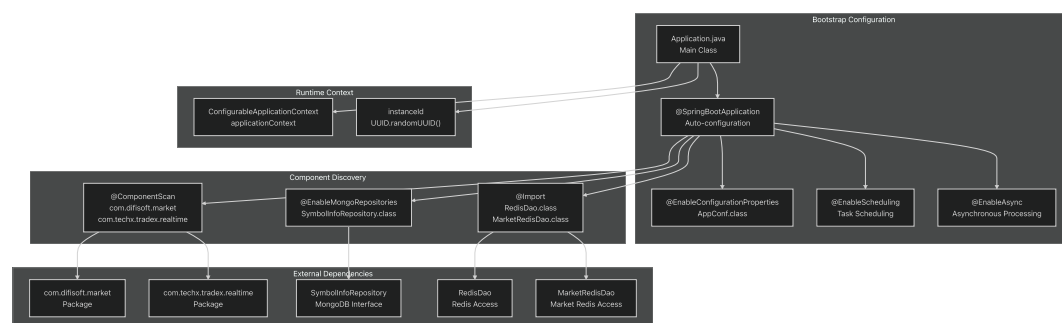
[pom.xml#L20-L25](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L20-L25) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L20-L25>)

[pom.xml#L123-L138](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L123-L138) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L123-L138>)

(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L123-L138>)

Application Bootstrap Architecture

The application initialization follows Spring Boot's auto-configuration pattern with custom component scanning and repository configuration.



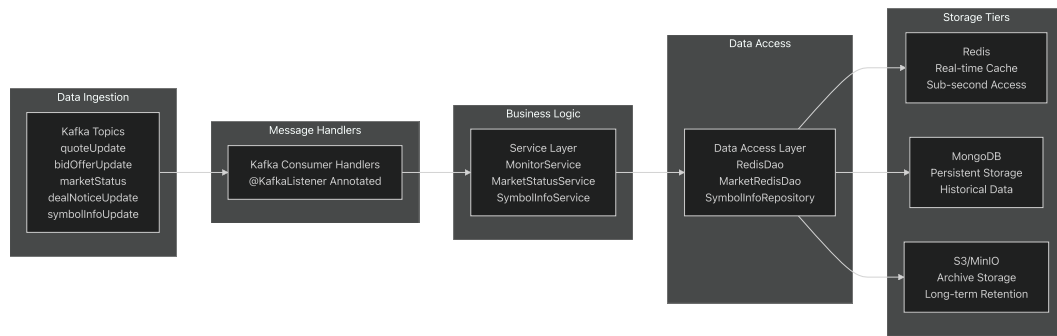
Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L19-L25>

[src/main/java/com/techx/tradex/realtime/Application.java#L19-L25](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L19-L25) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L19-L25>)

[src/main/java/com/techx/tradex/realtime/Application.java#L27-L33](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L27-L33) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L27-L33>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/Application.java#L27-L33>)

Data Flow Architecture

The system processes market data through a multi-stage pipeline that transforms raw Kafka messages into structured data stored across multiple persistence layers.



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L36-L41>

[pom.xml#L36-41](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L36-L41) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L36-L41> [

[pom.xml#L49-52](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L49-L52) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L49-L52> [

[%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L49-L52) [%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L49-L52)).

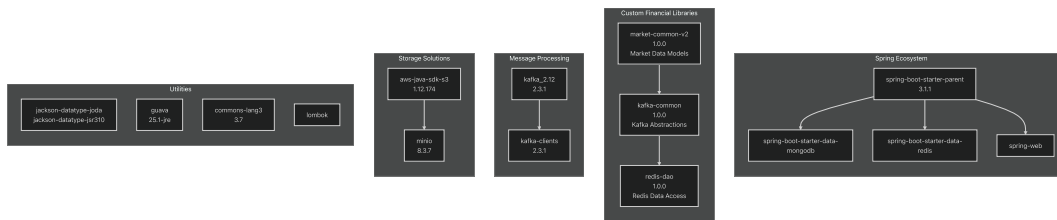
[pom.xml#L135-138](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L135-L138) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L135-L138>

<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L135-L138>)

Dependency Management Architecture

The application relies on both standard open-source libraries and custom financial domain libraries for specialized market data processing capabilities.

Core Dependencies



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L61-L75>

[pom.xml#L61-75](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L61-L75) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L61-L75> [

[pom.xml#L123-132](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L123-L132) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L123-L132> [

[%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L123-L132) [%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L123-L132)).

[pom.xml#L135-148](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L135-L148) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L135-L148>

<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L135-L148>)

The system architecture demonstrates a clear separation of concerns with well-defined boundaries between external data ingestion, internal processing, and data persistence layers. The heavy reliance on Spring Boot's auto-configuration and dependency injection provides a robust foundation for the complex real-time market data processing requirements.

Configuration System

Overview

Relevant source files

- [

src/main/java/com/techx/tradex/realtime/configurations/AppConf.java](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java>)

• [

src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java>)

The Configuration System manages all application settings, feature toggles, and external service configurations for the tradex-realtime application. This system provides centralized configuration management through Spring Boot's configuration properties mechanism and handles Spring Bean initialization for core services.

For information about how configurations are deployed and managed in different environments, see [Service Orchestration](#).

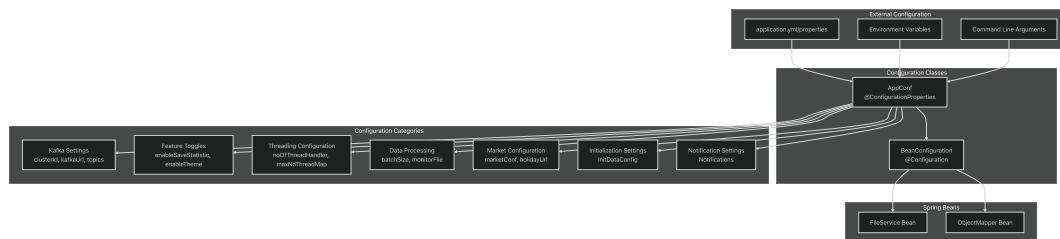
Overview

The configuration system consists of two primary components:

- **Application Configuration:** Centralized property management through the `AppConf` class
- **Bean Configuration:** Spring Bean initialization and customization through `BeanConfiguration`

Configuration Architecture

The configuration system follows a hierarchical structure that separates concerns into logical groupings:



Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L1-L107>)

<src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L1-L107> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L1-L107>)

src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L28](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L28>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L28>)

Application Configuration Structure

The `AppConf` class serves as the central configuration hub using Spring Boot's `@ConfigurationProperties` annotation:



Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L11-L144>)

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L11-L44](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L11-L44) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L11-L44>)

Configuration Categories

Kafka Configuration

The Kafka-related configuration manages message broker connectivity and topic definitions:

Configuration Property	Type	Purpose
<code>clusterId</code>	String	Identifies the Kafka cluster instance
<code>kafkaUrl</code>	String	Kafka broker URLs (semicolon-separated, converted to comma-separated)
<code>topics</code>	Topics	Contains all Kafka topic names used by the application

The `getKafkaBootstraps()` method converts the semicolon-separated `kafkaUrl` to comma-separated format for Kafka client compatibility. (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L93-L95>)

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L93-L95](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L93-L95) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L93-L95>)

The `Topics` inner class defines all Kafka topic names:

Topic Property	Purpose
<code>quoteUpdate</code>	Real-time quote updates
<code>quoteRecover</code>	Quote recovery messages
<code>quoteOddLotUpdate</code>	Odd lot quote updates
<code>bidOfferUpdate</code>	Bid/offer updates
<code>marketStatus</code>	Market status changes
<code>dealNoticeUpdate</code>	Deal notice updates
<code>realtimeJob</code>	Administrative job messages

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L70-L91>)

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L70-L91](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L70-L91) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L70-L91>)

Feature Toggle System

Feature toggles enable runtime control of application functionality:

Toggle	Default	Purpose
<code>enableSaveStatistic</code>	<code>true</code>	Controls statistic data persistence
<code>enableTheme</code>	<code>true</code>	Enables theme-related processing
<code>enableSaveBidOffer</code>	<code>true</code>	Controls bid/offer data persistence
<code>enableCheckOrderQuote</code>	<code>false</code>	Enables quote order validation
<code>enableSaveQuote</code>	<code>false</code>	Controls quote data persistence
<code>enableInitData</code>	<code>false</code>	Enables initialization data loading
<code>enableQuotePartition</code>	<code>false</code>	Enables quote partitioning

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L28-L38> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L28-L38>)

Threading Configuration

The system provides configurable threading for Kafka consumers:

- `noOfThreadHandler` : Default number of handler threads (default: 10)
- `defaultConsumerThread` : Default consumer threads per topic (default: 1)
- `maxNoThreadMap` : Topic-specific thread limits

The `getMaxThread(String topic)` method returns topic-specific or default thread counts (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L46-L49>).

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java46-49](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L46-L49) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L46-L49>)

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L17-L25> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L17-L25>)

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java46-49](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L46-L49) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L46-L49>)

Initialization Configuration

The `InitDataConfig` inner class manages system initialization timing:

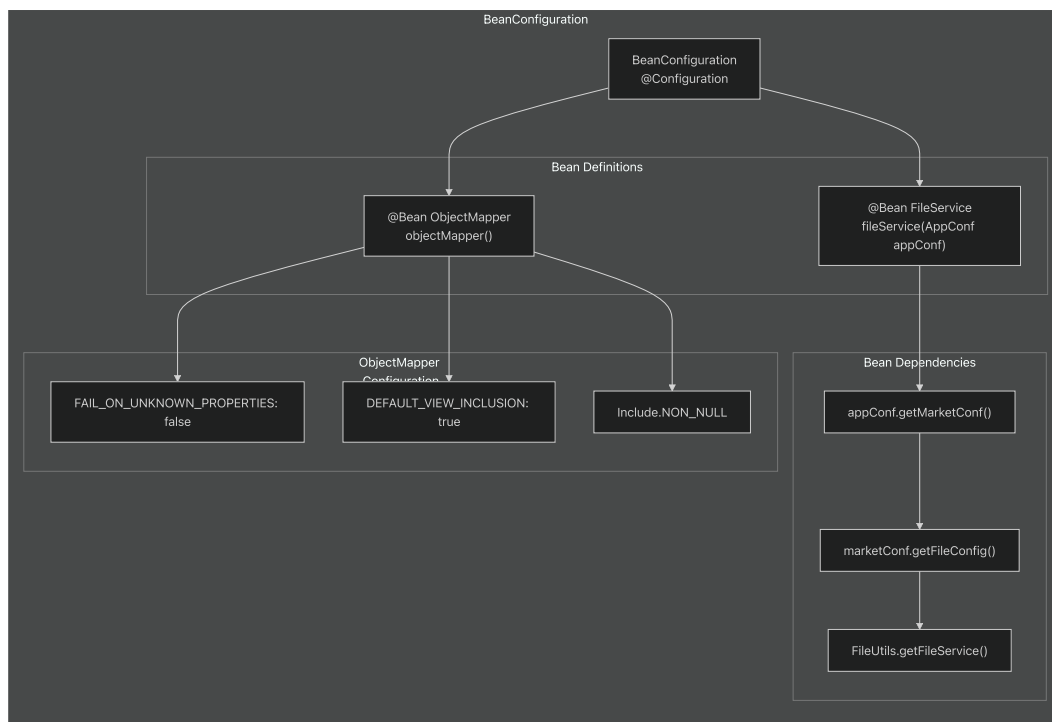
Property	Default	Purpose
version	1	Configuration version (supports 1, 2)
startDay	"17:00:00"	Daily cycle start time
startInit	"00:00:00"	Initialization start time
endInitTime	"01:44:55"	Initialization end time
endTradingHour	"08:05:00"	Trading session end time
startInitTheme	"01:04:00"	Theme initialization start time

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L98-L105>

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java98-105](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L98-L105) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L98-L105>)

Spring Bean Configuration

The `BeanConfiguration` class manages Spring Bean creation and customization:



FileService Bean

The `FileService` bean is configured based on the market configuration's file settings. This service handles file operations throughout the application. (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L14-L17>)

[src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java14-17](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L14-L17) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L14-L17>)

ObjectMapper Bean

The `ObjectMapper` bean is configured with specific Jackson settings for JSON processing:

- **Deserialization:** Ignores unknown properties to handle API evolution
- **View Inclusion:** Includes default views for flexible JSON rendering
- **Serialization:** Excludes null values from JSON output

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L19-L26>)
[src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java19-26](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L19-L26) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L19-L26>)

Configuration Usage Patterns

The configuration system is accessed throughout the application via dependency injection. Key usage patterns include:

1. **Direct Property Access:** Components inject `AppConf` to access configuration values
2. **Conditional Logic:** Feature toggles control application behavior at runtime
3. **Threading Control:** Consumer configurations use thread settings for optimal performance
4. **Topic Resolution:** Kafka consumers use topic names from the `Topics` configuration

The centralized configuration approach ensures consistency across the application and enables environment-specific deployments without code changes.

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L1-L107>)
[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java1-107](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L1-L107) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L1-L107>)
[src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java1-28](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L28) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L28>)
[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L28](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L28) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L28>)

Application Configuration

Relevant source files

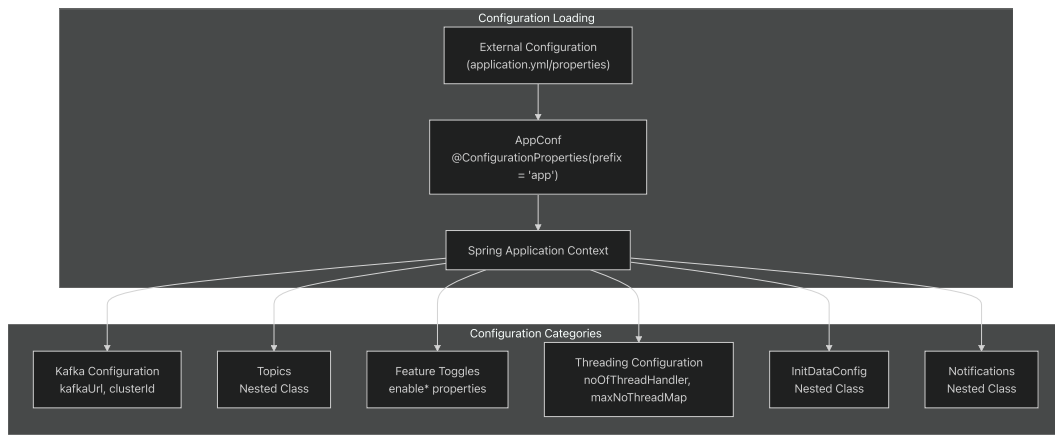
- [[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java)] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java>)
[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java>)

This page documents the central configuration system of the tradex-realtime application, specifically the `AppConf` class and its nested configuration structures. This covers application-wide settings, feature toggles, Kafka topic definitions, and runtime parameters that control system behavior.

For information about Spring bean configuration, see [Bean Configuration](#). For deployment-specific environment configuration, see [Service Orchestration](#).

Overview

The application configuration is centralized in the `AppConf` class, which uses Spring Boot's `@ConfigurationProperties` mechanism to bind external configuration properties to Java objects. This provides type-safe access to configuration values throughout the application.



Sources: [https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L11-L13)

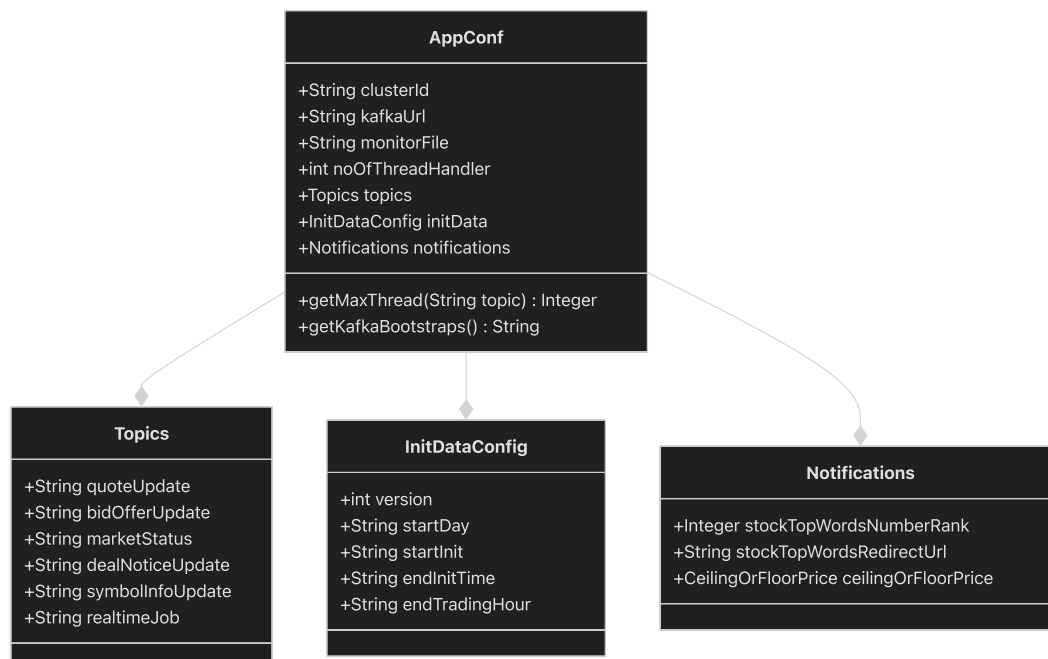
[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L11-L13](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L11-L13)

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java11-13](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L11-L13) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L11-L13>

Configuration Structure

The `AppConf` class serves as the root configuration container with several nested static classes that organize related configuration properties:

Configuration Class	Purpose	Key Properties
<code>AppConf</code>	Root configuration	<code>clusterId</code> , <code>kafkaUrl</code> , <code>noOfThreadHandler</code>
<code>Topics</code>	Kafka topic names	<code>quoteUpdate</code> , <code>bidOfferUpdate</code> , <code>marketStatus</code>
<code>InitDataConfig</code>	Initialization parameters	<code>version</code> , <code>startDay</code> , <code>endInitTime</code>
<code>Notifications</code>	Notification settings	<code>stockTopWordsNumberRank</code> , <code>ceilingOrFloorPrice</code>
<code>PublishConf</code>	Publishing configuration	<code>topic</code> , <code>channel</code>



Sources: <https://github.com/nhsv-tradex/realtime->

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L13-L106](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L13-L106).

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java13-106](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L13-L106) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L13-L106>).

Kafka Configuration

The configuration manages Kafka connectivity and topic definitions essential for the real-time data processing pipeline:

Connection Settings

- `kafkaUrl` : Comma or semicolon-separated list of Kafka bootstrap servers
- `clusterId` : Identifier for the Kafka cluster
- `getKafkaBootstraps()` : Method that normalizes the URL format by replacing semicolons with commas

Topic Configuration

The nested `Topics` class defines all Kafka topic names used by the application's consumers:

Topic Category	Properties	Purpose
Quote Processing	<code>quoteUpdate</code> , <code>quoteRecover</code> , <code>quoteOddLotUpdate</code>	Quote data streams
Bid/Offer Processing	<code>bidOfferUpdate</code> , <code>bidOfferOddLotUpdate</code>	Bid and offer data
Market Operations	<code>marketStatus</code> , <code>marketInit</code>	Market state management
Administrative	<code>realtimeJob</code> , <code>requestListener</code>	Job and request handling
Notifications	<code>virtualNotification</code> , <code>userUtility</code>	User-facing notifications

Sources: <https://github.com/nhsv-tradex/realtime->

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L14-L15](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L14-L15).

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java14-15](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L14-L15) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L14-L15>)

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java70-91](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L70-L91) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L70-L91>)

[%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L70-L91)).

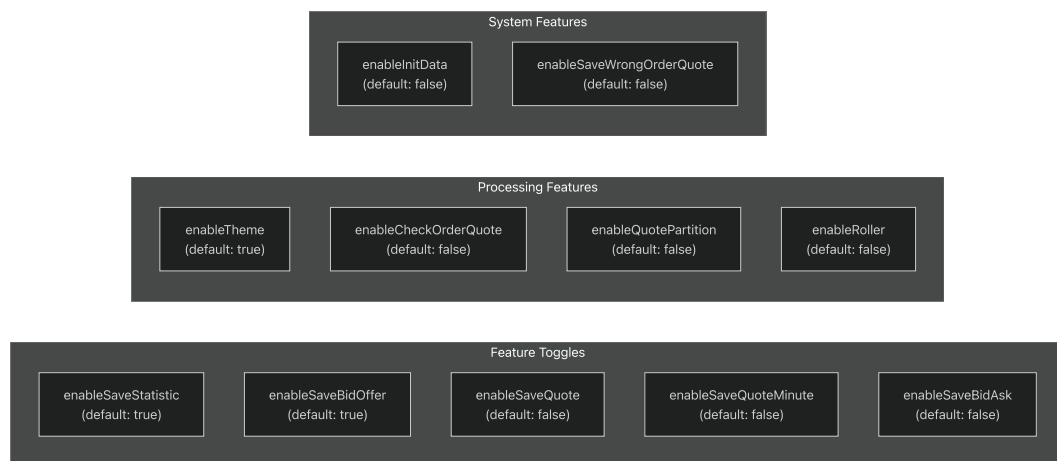
[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java93-95](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L93-L95) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L93-L95>)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L93-L95](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L93-L95))).

Feature Toggle System

The configuration includes extensive feature toggles that control various system behaviors:

Data Persistence Features



These boolean flags allow runtime control of system features without code changes, supporting different deployment configurations and gradual feature rollouts.

Sources: [https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L28-L38)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L28-L38](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L28-L38)

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java28-38](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L28-L38) ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L28-L38)

Threading Configuration

The system provides flexible threading configuration for Kafka consumers and processing tasks:

Thread Management Properties

- `noOfThreadHandler` : Default number of handler threads (default: 10)
- `defaultConsumerThread` : Default threads per consumer (default: 1)
- `maxNoThreadMap` : Topic-specific thread limits as a `Map<String, Integer>`

Thread Allocation Logic

The `getMaxThread(String topic)` method implements fallback logic:

- Check if topic has specific thread limit in `maxNoThreadMap`
- Fall back to `defaultConsumerThread` if no specific limit exists

This allows fine-tuned performance optimization for high-volume topics while maintaining simple defaults.

Sources: [https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L17-L17)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L17-L17](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L17-L17)

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java17](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L17-L17) ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L17-L17)

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java24-25](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L24-L25)] ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L24-L25)

[src/main/java/com/techx/tradex/realtime/configurations/AppConf.java46-49](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L46-L49)] ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L46-L49)

Initialization Configuration

The `InitDataConfig` nested class controls the system's initialization and trading day cycle:

Time-based Configuration

Property	Default	Purpose
<code>version</code>	1	Configuration schema version (supports 1, 2)
<code>startDay</code>	"17:00:00"	Trading day start time
<code>startInit</code>	"00:00:00"	Initialization start time
<code>endInitTime</code>	"01:44:55"	Initialization end time
<code>endTradingHour</code>	"08:05:00"	Trading day end time
<code>startInitTheme</code>	"01:04:00"	Theme initialization time

These timestamps coordinate the application's lifecycle with market operating hours and ensure proper initialization sequencing.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L22-L22> [src/main/java/com/techx/tradex/realtime/configurations/AppConf.java22](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L22-L22) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L22-L22> [src/main/java/com/techx/tradex/realtime/configurations/AppConf.java98-105](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L98-L105) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L98-L105> <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L98-L105>)

Processing Parameters

Additional configuration parameters control data processing behavior:

Batch and Partition Settings

- `batchSize` : Processing batch size (default: 1000)
- `quotePartitionIntervalSecond` : Quote partitioning interval (default: 30 seconds)
- `quotePartitionMinimumSize` : Minimum partition size (default: 100)
- `cycleToRecoverMinute` : Recovery cycle interval (default: 15 minutes)

File and URL Configuration

- `monitorFile` : Monitor data file path (default: "/data/monitor.data")
- `holidayUrl` : External holiday data URL
- `holidayLocalFile` : Local holiday data file path

These parameters allow tuning of system performance and data processing characteristics based on deployment requirements.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L16-L16> [src/main/java/com/techx/tradex/realtime/configurations/AppConf.java16](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L16-L16) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L16-L16> <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L16-L16>)

src/main/java/com/techx/tradex/realtime/configurations/AppConf.java19-21](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L19-L21>)([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L19-L21)).

src/main/java/com/techx/tradex/realtime/configurations/AppConf.java40-42](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L40-L42>)([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/AppConf.java#L40-L42))).

Bean Configuration

Relevant source files

- [

src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java>)([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java))).

This document explains how Spring beans are configured and initialized in the tradex-realtime application through the `BeanConfiguration` class. This covers the creation and configuration of shared service beans that provide common functionality across the application.

For information about application-wide settings and feature toggles, see [Application Configuration](#). For details about deployment configuration, see [Docker Deployment](#).

Overview

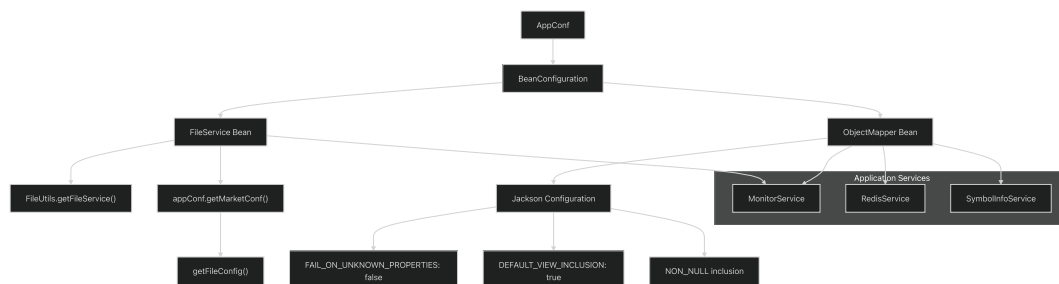
The bean configuration system in tradex-realtime uses Spring's Java-based configuration approach to define and configure essential service beans. The `BeanConfiguration` class serves as the central location for configuring beans that require custom initialization or depend on application configuration.

The configuration creates two primary beans:

- `FileService` for file operations and storage access
- `ObjectMapper` for JSON serialization and deserialization

Bean Configuration Architecture

The following diagram shows how the `BeanConfiguration` class integrates with the application's configuration system:



Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L27>).

src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java1-27](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L27>).

Bean Definitions

Configuration Class Structure

The `BeanConfiguration` class uses Spring's `@Configuration` annotation to indicate it contains bean definitions:

Annotation	Purpose
<code>@Configuration</code>	Marks the class as a source of bean definitions
<code>@Bean</code>	Indicates methods that produce beans managed by Spring container

Sources: [https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L12-L13)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L12-L13](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L12-L13))

[src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java12-13](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L12-L13) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L12-L13>).

FileService Bean

The `FileService` bean provides file operations and storage access capabilities:



The bean configuration:

- Takes `AppConf` as a dependency injection parameter
- Extracts file configuration from the market configuration
- Uses `FileUtils.getFileService()` to create the service instance

Sources: [https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L14-L17)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L14-L17](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L14-L17))

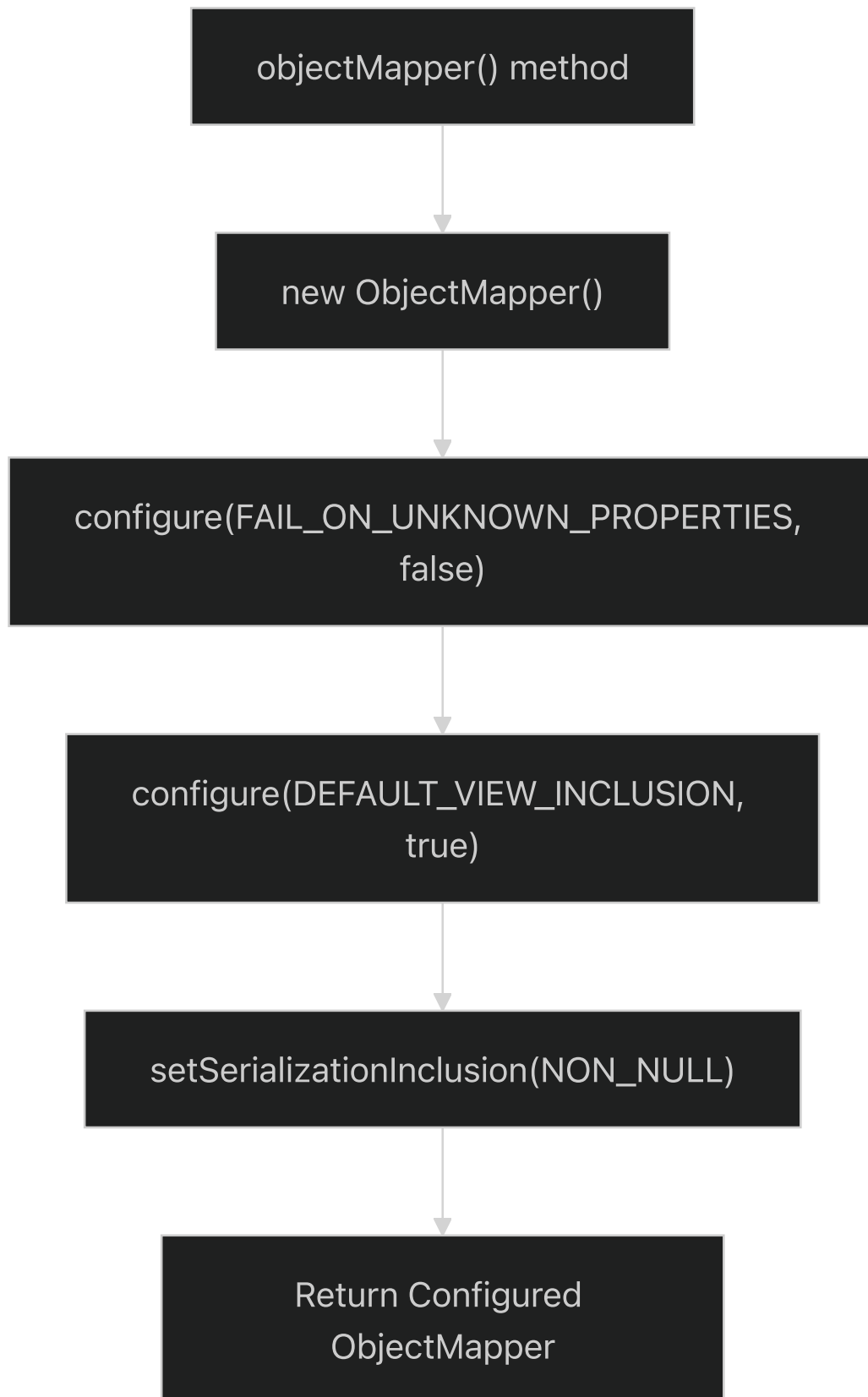
[src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java14-17](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L14-L17) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L14-L17>).

ObjectMapper Bean

The `ObjectMapper` bean configures Jackson for JSON processing with specific settings optimized for the trading system:

Configuration Setting	Value	Purpose
<code>FAIL_ON_UNKNOWN_PROPERTIES</code>	<code>false</code>	Allows processing of JSON with additional fields
<code>DEFAULT_VIEW_INCLUSION</code>	<code>true</code>	Includes all properties in JSON views by default
<code>SerializationInclusion</code>	<code>NON_NULL</code>	Excludes null values from JSON output

The configuration process:



These settings ensure:

- Backward compatibility when consuming external JSON data
- Consistent JSON output format
- Efficient serialization by omitting null fields

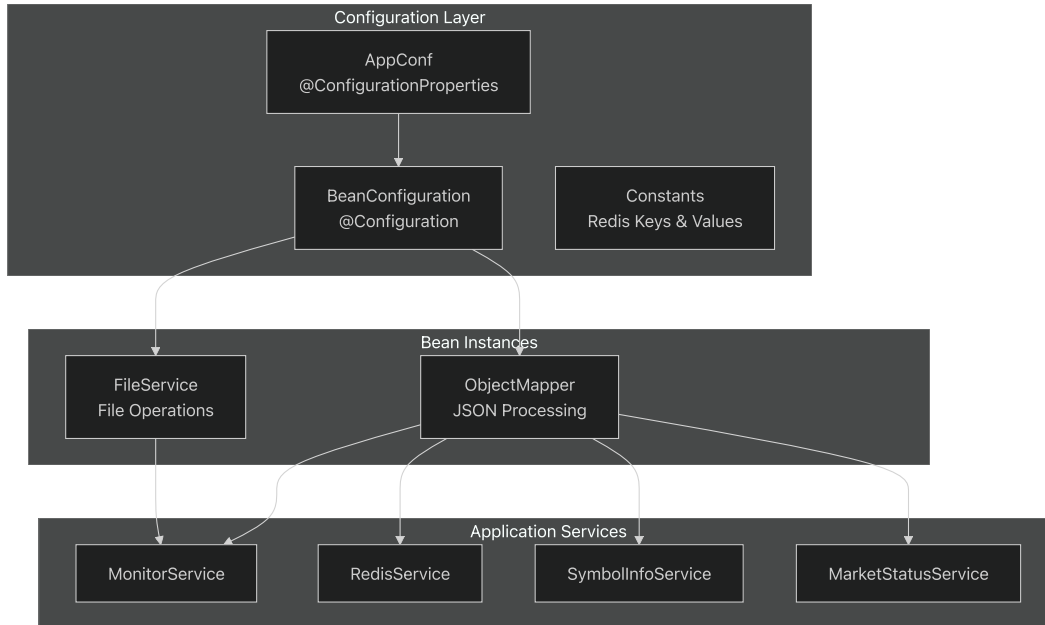
Sources: <https://github.com/nhsv-tradex/realtime->

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L19-L26](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L19-L26)

[src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java19-26](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L19-L26) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L19-L26>).

Integration with Application Configuration

The bean configuration system integrates with the broader configuration architecture:



The `BeanConfiguration` class bridges the gap between configuration properties and service implementations by:

- Injecting configuration dependencies into bean creation methods
- Applying configuration-specific settings to service instances
- Providing configured beans to the Spring application context

Sources: <https://github.com/nhsv-tradex/realtime->

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L27](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L27)

[src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java1-27](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L27) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/configurations/BeanConfiguration.java#L1-L27>).

Real-time Data Processing

Overview

Relevant source files

- [[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java)] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java>))
- [

src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java>)

- [

src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java>)

Purpose and Scope

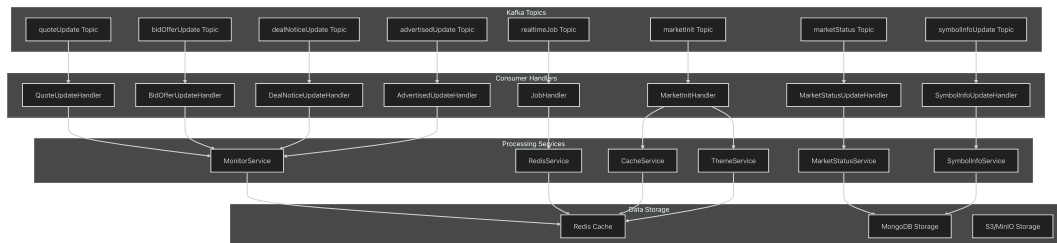
This document provides an overview of the real-time data processing pipeline in the tradex-realtime system. It covers the core components responsible for consuming market data from Kafka topics, processing different types of financial data updates, and managing the flow of information through the system's various storage layers.

The real-time processing system handles multiple data types including quote updates, bid/offer data, market status changes, symbol information, deal notices, and administrative jobs. For detailed information about specific data consumers and their processing logic, see sections [3.1](#) through [3.7](#). For system configuration details, see [Configuration System](#).

Architecture Overview

The real-time data processing pipeline follows an event-driven architecture where Kafka consumers handle different types of market data messages. Each consumer type is specialized for a specific data format and delegates processing to appropriate services.

Core Processing Pipeline



Sources:[](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/System>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/System>) architecture diagrams)

[System architecture diagrams](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/System>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/System>) architecture diagrams)[

src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java1-43](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L1-L43>)] ([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L1-L43))

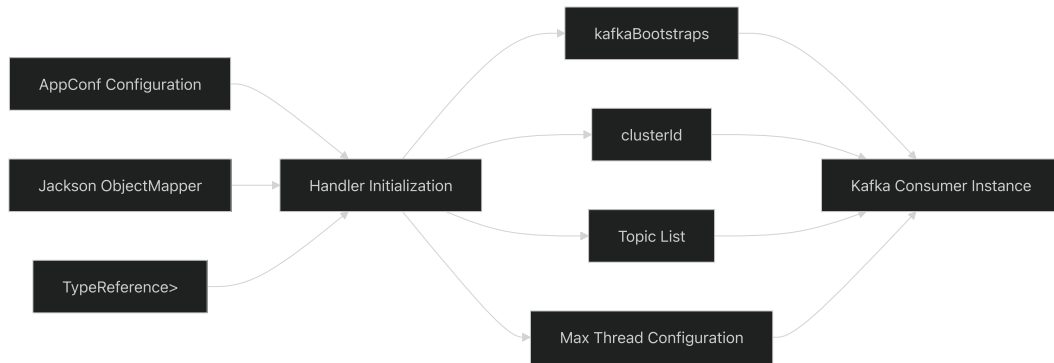
src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java1-51] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L1-L51>) ([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L1-L51))

Consumer Handler Pattern

All real-time data consumers in the system extend the `KafkaConsumerHandler<T>` base class and follow a consistent pattern for message processing. Each handler is responsible for:

1. **Message Deserialization:** Converting Kafka messages into typed objects using Jackson `TypeReference`
2. **Data Validation:** Checking message and data validity before processing
3. **Service Delegation:** Forwarding validated data to appropriate business logic services
4. **Error Handling:** Logging errors and handling exceptions gracefully

Handler Initialization Pattern



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L21-L32>.

[src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java21-32](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L21-L32) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L21-L32>

[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java22-33](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L22-L33) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L22-L33> <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L22-L33>

Message Processing Flow

Each consumer handler implements a `handle()` method that processes incoming Kafka messages. The processing follows these steps:

Step	Description	Example Implementation
Validation	Check if message and data are not null	<code>` if (message == null</code>
Data Extraction	Extract typed data from message wrapper	<code>SymbolQuote symbolQuote = message.getData()</code>
Service Delegation	Pass data to appropriate service for processing	<code>monitorService.rcv(symbolQuote)</code>
Performance Tracking	Log processing time and statistics	<code>log.info("accept marketStatus take: {}", (t2 - t1))</code>
Error Handling	Catch and log exceptions with message ID	<code>log.error("error while handle marketStatus: {}", message.getMessageId(), e)</code>

Sources: <https://github.com/nhsv-tradex/realtime->

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L34-L41](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L34-L41).

[src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java34-41](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L34-L41) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L34-L41>)

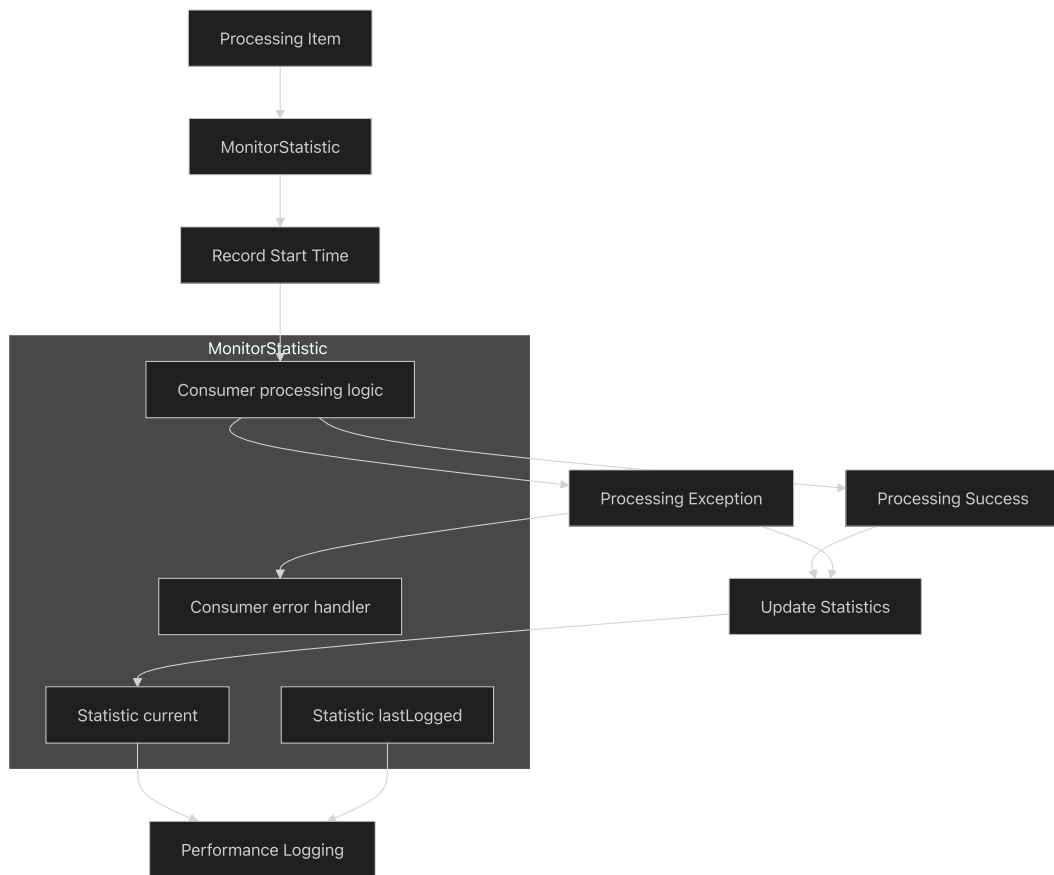
[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java35-49](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L35-L49)

(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L35-L49>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L35-L49>)

Performance Monitoring

The system includes comprehensive performance monitoring through the `MonitorStatistic<T>` class, which tracks processing metrics for each data type.

Monitoring Architecture



Sources: <https://github.com/nhsv-tradex/realtime->

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L1-L73](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L1-L73).

[src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java1-73](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L1-L73) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L1-L73>)

Statistical Tracking

The `MonitorStatistic` class tracks the following metrics:

- **Total Processed Items:** Cumulative count of all processed items
- **Total Processing Time:** Aggregate time spent processing items in milliseconds
- **Incremental Metrics:** Difference between current and last logged statistics

- **Error Handling:** Separate error consumer for exception management

Key methods in `MonitorStatistic`:

Method	Purpose	Implementation
<code>process(T item)</code>	Process item with timing	Records start time, executes consumer, updates statistics
<code>log(Logger logger)</code>	Log performance metrics	Compares current vs last logged statistics
<code>compare(Statistic last)</code>	Calculate incremental metrics	Returns difference between current and previous statistics

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L21-L42>.

[src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java21-42](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L21-L42) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L21-L42>).

Data Type Processing

The real-time processing system handles multiple specialized data types, each with dedicated consumer handlers:

Data Type	Handler Class	Processing Service	Primary Storage
Quote Updates	<code>QuoteUpdateHandler</code>	<code>MonitorService</code>	Redis Cache
Bid/Offer Updates	<code>BidOfferUpdateHandler</code>	<code>MonitorService</code>	Redis Cache
Market Status	<code>MarketStatusUpdateHandler</code>	<code>MarketStatusService</code>	MongoDB
Symbol Information	<code>SymbolInfoUpdateHandler</code>	<code>SymbolInfoService</code>	MongoDB
Deal Notices	<code>DealNoticeUpdateHandler</code>	<code>MonitorService</code>	Redis Cache
Advertisements	<code>AdvertisedUpdateHandler</code>	<code>MonitorService</code>	Redis Cache
Administrative Jobs	<code>JobHandler</code>	<code>RedisService</code>	Redis Cache
Market Initialization	<code>MarketInitHandler</code>	<code>CacheService</code> , <code>ThemeService</code>	Redis Cache

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/System> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/System>), architecture diagrams)

[System architecture diagrams] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/System>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/System>), architecture diagrams)

[src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java19-40](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L19-L40) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L19-L40>) ([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L19-L40))

src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java19-43]
[\(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L19-L43](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L19-L43)
[\(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L19-L43\)\)](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L19-L43)

Threading and Scalability

Each consumer handler is configured with thread pool settings managed through the `AppConf` configuration. The threading model allows for:

- **Per-Topic Thread Configuration:** Individual thread pool sizing via `appConf.getMaxThread(topicName)`
- **Concurrent Message Processing:** Multiple messages processed simultaneously within each handler
- **Resource Isolation:** Separate thread pools prevent one data type from blocking others

The thread configuration is topic-specific, allowing fine-tuned performance optimization for different data volumes and processing requirements.

Sources: [\(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L31-L31\)](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L31-L31)

<src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java31>
[\(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L31-L31\)](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L31-L31)

src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java32]
[\(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L32-L32](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L32-L32)
[\(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L32-L32\)\)](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L32-L32)

Market Data Consumers

Relevant source files

- [

 src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java>

[\(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java\)](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java)
- [

 src/main/java/com/techx/tradex/realtime/consumers/ExtraQuoteUpdateHandler.java]

[\(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/ExtraQuoteUpdateHandler.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/ExtraQuoteUpdateHandler.java)

[\(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/ExtraQuoteUpdateHandler.java\)\)](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/ExtraQuoteUpdateHandler.java)
- [

 src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java]

[\(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java)

[\(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java\)\)](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java)
- [

src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java>)

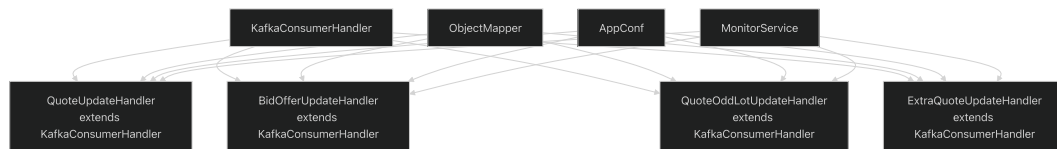
This document covers the Kafka consumer handlers that process real-time market data streams in the tradex-realtime system. These consumers receive various types of market data messages from Kafka topics and forward them to appropriate processing services.

For information about the services that process the consumed data, see [Quote Processing](#) and [Bid/Offer Processing](#). For details on market status and symbol information processing, see [Market Status Management](#) and [Symbol Information Processing](#).

Consumer Handler Pattern

All market data consumers in the system extend the `KafkaConsumerHandler` base class from the kafka-common library. They follow a consistent architectural pattern for processing typed messages from specific Kafka topics.

Common Architecture



Each consumer handler requires three core dependencies:

- `ObjectMapper` for JSON deserialization
- `AppConf` for configuration including Kafka bootstraps and topic names
- `MonitorService` for processing the received data

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L21-L32>).

<src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L21-L32> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L21-L32>)

src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L23-L34]

(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L23-L34>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L23-L34>)

Quote Data Consumers

QuoteUpdateHandler

The `QuoteUpdateHandler` processes standard symbol quote updates from the `quoteUpdate` topic. It handles `SymbolQuote` objects wrapped in Kafka `Message` containers.

Property	Value
Data Type	<code>SymbolQuote</code>
Topic	<code>appConf.getTopics().getQuoteUpdate()</code>
Max Threads	<code>appConf.getMaxThread(appConf.getTopics().getQuoteUpdate())</code>

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L17-L43> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L17-L43>)

QuoteOddLotUpdateHandler

The `QuoteOddLotUpdateHandler` processes odd lot quote updates from the `quoteOddLotUpdate` topic. It handles `SymbolQuoteOddLot` objects for non-standard trading lot sizes.

Property	Value
Data Type	<code>SymbolQuoteOddLot</code>
Topic	<code>appConf.getTopics().getQuoteOddLotUpdate()</code>
Max Threads	<code>appConf.getMaxThread(appConf.getTopics().getQuoteOddLotUpdate())</code>

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java#L16-L43> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java#L16-L43>)

ExtraQuoteUpdateHandler

The `ExtraQuoteUpdateHandler` processes additional quote information from the `extraUpdate` topic. It handles `ExtraQuote` objects containing supplementary quote data.

Property	Value
Data Type	<code>ExtraQuote</code>
Topic	<code>appConf.getTopics().getExtraUpdate()</code>
Max Threads	<code>appConf.getMaxThread(appConf.getTopics().getExtraUpdate())</code>

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/ExtraQuoteUpdateHandler.java#L16-L42> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/ExtraQuoteUpdateHandler.java#L16-L42>)

Bid/Offer Data Consumer

BidOfferUpdateHandler

The `BidOfferUpdateHandler` processes bid and offer updates from the `bidOfferUpdate` topic. It handles `BidOffer` objects containing current bid and offer prices.

Property	Value
Data Type	<code>BidOffer</code>
Topic	<code>appConf.getTopics().getBidOfferUpdate()</code>
Max Threads	<code>appConf.getMaxThread(appConf.getTopics().getBidOfferUpdate())</code>

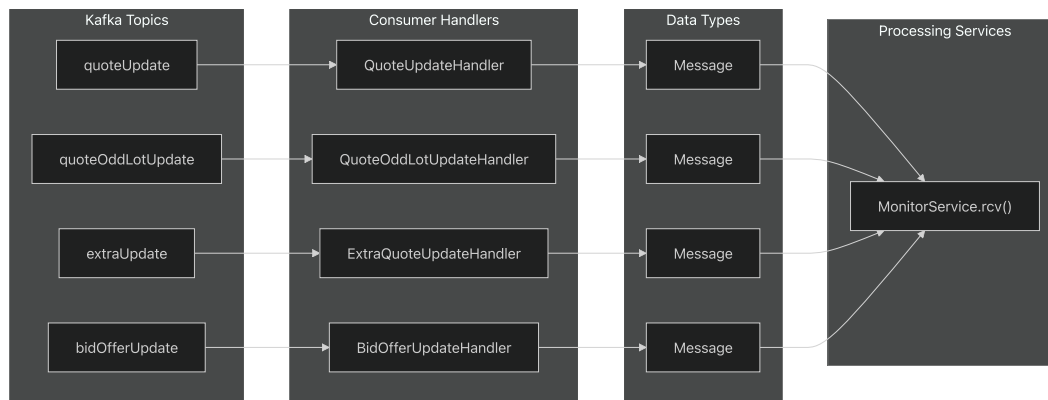
Sources: ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L18-L44)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L18-L44](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L18-L44))

[src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java18-44](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L18-L44) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L18-L44>)

Message Processing Flow

The following diagram shows how market data flows from Kafka topics through the consumer handlers to the processing services:



Sources: ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L34-L41)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L34-L41](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L34-L41))

[src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java34-41](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L34-L41) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java#L34-L41>)

[src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java36-43](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L36-L43)

([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L36-L43)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L36-L43](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L36-L43))

([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L36-L43)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L36-L43](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L36-L43))

Consumer Initialization

All consumer handlers follow the same initialization pattern in their constructors:

- TypeReference Creation** - Define the generic type for message deserialization
- Super Initialization** - Call parent `init()` method with:
 - `TypeReference<Message<T>>` for type safety
 - `ObjectMapper` for JSON processing
 - Kafka bootstrap servers from `appConf.getKafkaBootstraps()`
 - Cluster ID from `appConf.getClusterId()`

- Single topic list from topic configuration
- Empty `Properties` object
- Max thread count for the specific topic

Error Handling

Each `handle()` method implements consistent validation:

```

if (message == null
    | | message.getData() == null) { |
|--- | --- |
    | return; |
    | } |
    | `` |

```

This ensures that only valid messages with non-null data are processed and passed to the `M

****Sources:**** [(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/t

[src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java27-32] (https://g:

src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java34-38] (https://git

Quote Processing

Relevant source files

- [
 - src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java] (https:
- [
 - src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java] (https://git
- [
 - src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java] (https://git

This document details the quote processing subsystem within the tradex-realtime application

For information about bid/offer processing, see [Bid/Offer Processing] (#bidoffer-processing)

Overview

The quote processing subsystem consists of three specialized Kafka consumer handlers that p

****Quote Processing Architecture****

![Chart 1] (realtime-v2-images/realtime-v2-section7-1.svg)

Sources: [(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/tech:

[src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java1-43] (https://git

src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java1-44] (https:

src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java1-43] (https://git

Quote Handler Components

Standard Quote Processing

The `QuoteUpdateHandler` processes standard market quotes through the `quoteUpdate` Kafka t

Component	Description
Handler Class	`QuoteUpdateHandler`
Data Model	`SymbolQuote`
Kafka Topic	`appConf.getTopics().getQuoteUpdate()`

```
| Processing Method | `monitorService.rcv(symbolQuote)` |

**Key Implementation Details:**

- Extends `KafkaConsumerHandler<SymbolQuote>` [

    src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java17] (https://g

- Uses `TypeReference<Message<SymbolQuote>>` for JSON deserialization [

    src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java27-28] (https:

- Thread pool size configured via `appConf.getMaxThread()` [

    src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java31] (https://g

##### Odd Lot Quote Processing

The `QuoteOddLotUpdateHandler` specifically handles odd lot quotes, which represent trades

| Component | Description |
|---|---|
| Handler Class | `QuoteOddLotUpdateHandler` |
| Data Model | `SymbolQuoteOddLot` |
| Kafka Topic | `appConf.getTopics().getQuoteOddLotUpdate()` |
| Processing Method | `monitorService.rcv(symbolQuote)` |

**Implementation Notes:**

- Specialized for `SymbolQuoteOddLot` data model [

    src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java16] (http

- Contains commented legacy code for direct topic publishing [

    src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java40] (http

##### Quote Recovery Processing

The `QuoteRecoverHandler` manages quote recovery operations, handling requests to restore c

| Component | Description |
|---|---|
| Handler Class | `QuoteRecoverHandler` |
| Data Model | `QuoteRecover` |
| Kafka Topic | `appConf.getTopics().getQuoteRecover()` |
| Processing Method | `monitorService.rcv(symbolQuote)` |

**Recovery Processing Flow:**

- Processes `QuoteRecover` requests but converts to `SymbolQuote` for downstream processi

    src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java39-40] (https:

- Maintains same error handling pattern as other quote handlers [

    src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java34-37] (https:

Sources: [] (https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/tech

[src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java17-31] (https://g

src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java16-30] (http

src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java18-31] (https://g

#### Data Processing Flow
```

****Quote Processing Pipeline****

![[Chart 2](realtime-v2-images/realtime-v2-section7-2.svg)]

All quote handlers implement identical error handling and processing patterns:

1. ****Message Validation****: Check for null message or null data[
src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java34-37] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java34-37>)
2. ****Data Extraction****: Extract the quote data from the Kafka message[
src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java39] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java39>)
3. ****Service Delegation****: Forward to `MonitorService.rcv()` for processing[
src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java40] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java40>)

Sources: [] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java34-41>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java33-41>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java34-41>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java34-41>)

[src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java34-41] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java34-41>)

src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java33-41] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java33-41>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java34-41>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java34-41>)

src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java34-41] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java34-41>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java34-41>)

Configuration and Initialization

Each quote handler follows a consistent initialization pattern using dependency injection &

****Handler Configuration Pattern****

Configuration Element	Source	Purpose
ObjectMapper	Constructor injection	JSON serialization/deserialization
AppConf	Constructor injection	Application configuration
MonitorService	Constructor injection	Quote processing service
KafkaBootstraps	appConf.getKafkaBootstraps()	Kafka cluster connection
ClusterId	appConf.getClusterId()	Kafka cluster identification
Thread Pool Size	appConf.getMaxThread(topicName)	Consumer thread configuration

****Initialization Flow****

![[Chart 3](realtime-v2-images/realtime-v2-section7-3.svg)]

All handlers use `Collections.singletonList()` to subscribe to exactly one Kafka topic and

Sources: [] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java21-31>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java21-31>)

[src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java21-31] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java21-31>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java21-31>)

src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java20-31] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java20-31>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java20-31>)

src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java21-31] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java21-31>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java21-31>)

Error Handling and Logging

Quote handlers implement uniform error handling with SLF4J logging:

- ****Null Safety****: All handlers check for null messages and null data before processing[
src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java34-37] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java34-37>)
- ****Error Logging****: Invalid data scenarios are logged with `log.error("Invalid data")`[
src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java34-37] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java34-37>)


```

src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java35] (https://g:
- **Graceful Degradation**: Handlers return early on invalid data without throwing except

src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java36] (https://g:

Each handler class is annotated with `@Slf4j` to enable logging capabilities through Lombok

Sources: [] (https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/tech:

[src/main/java/com/techx/tradex/realtime/consumers/QuoteUpdateHandler.java16-37] (https://g:

src/main/java/com/techx/tradex/realtime/consumers/QuoteOddLotUpdateHandler.java15-37] (http:

src/main/java/com/techx/tradex/realtime/consumers/QuoteRecoverHandler.java17-37] (https://g:

---

### Bid/Offer Processing

Relevant source files

- [

src/main/java/com/techx/tradex/realtime/constants/BidAskType.java] (https://github.com/r

- [

src/main/java/com/techx/tradex/realtime/consumers/BidOfferOddLotUpdateHandler.java] (ht

- [

src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java] (https://g:

#### Purpose and Scope

This document covers the processing of bid and offer market data within the tradex-realtime

For quote processing information, see [Quote Processing](#quote-processing). For general ma

#### Overview

The bid/offer processing system handles two distinct types of market depth data:

- **Standard Bid/Offer Updates**: Regular market depth information for standard lot sizes
- **Odd Lot Bid/Offer Updates**: Market depth information for non-standard lot transacti

Both processing streams follow the same architectural pattern, consuming messages from ded:

#### Bid/Ask Type Classification

![[Chart 1] (realtime-v2-images/realtime-v2-section8-1.svg)]

**Sources:** [] (https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/t

[src/main/java/com/techx/tradex/realtime/constants/BidAskType.java1-6] (https://github.com/r

#### Standard Bid/Offer Processing

The `BidOfferUpdateHandler` processes standard bid/offer market data from the `bidOfferUpd:

#### Handler Architecture

![[Chart 2] (realtime-v2-images/realtime-v2-section8-2.svg)]

#### Implementation Details

```

The `BidOfferUpdateHandler` class extends `KafkaConsumerHandler<BidOffer>` and implements the following interface:

Component	Details
Message Type	<code>Message<BidOffer></code>
Kafka Topic	Configured via <code>appConf.getTopics().getBidOfferUpdate()</code>
Threading	Multi-threaded based on <code>appConf.getMaxThread()</code> configuration
Error Handling	Logs errors for null messages or invalid data

The handler's `handle` method validates incoming messages and forwards valid `BidOffer` objects to the `monitorService`.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L44>

Odd Lot Bid/Offer Processing

The `BidOfferOddLotUpdateHandler` processes odd lot bid/offer data from the `bidOfferOddLotTopic`.

Handler Architecture



Implementation Comparison

Aspect	Standard Handler	Odd Lot Handler
Data Type	<code>BidOffer</code>	<code>BidOfferOddLot</code>
Kafka Topic	<code>bidOfferUpdate</code>	<code>bidOfferOddLotUpdate</code>
Message Type	<code>Message<BidOffer></code>	<code>Message<BidOfferOddLot></code>
Processing Method	<code>monitorService.rcv(bidOffer)</code>	<code>monitorService.rcv(bidOfferOddLot)</code>

Both handlers share identical error handling logic and architectural patterns, differing only in the data type and Kafka topic they process.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferOddLotUpdateHandler.java#L42>

Processing Flow

The complete bid/offer processing flow integrates both standard and odd lot processing streams.



Configuration and Dependencies

Both bid/offer handlers require the following dependencies for initialization:

Dependency	Purpose
<code>ObjectMapper</code>	JSON deserialization of Kafka messages
<code>AppConf</code>	Application configuration including Kafka settings
<code>MonitorService</code>	Processing service for market data

Kafka Configuration

The handlers are configured with the following Kafka settings from `AppConf`:

- Bootstrap Servers:** `appConf.getKafkaBoostraps()`
- Cluster ID:** `appConf.getClusterId()`
- Topic Names:** `appConf.getTopics().getBidOfferUpdate()` and `appConf.getTopics().getBidOfferOddLotUpdate()`
- Thread Pool Size:** `appConf.getMaxThread()` for respective topics

```
**Sources:**[] (https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L23-33) (https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferOddLotUpdateHandler.java#L21-31) (https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/BidOfferUpdateHandler.java#L23-33)
```

— — —

Market Status Management

Relevant source files

— [

```
src/main/java/com/techx/tradex/realtime/consumers/MarketInitHandler.java] (https://github.com/techx/tradex)
```

— [

src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java] (<https://github.com/techxtrading/tradex/blob/master/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java>)

This document explains how market status updates are processed and managed within the real-

For information about other market data processing, see [Market Data Consumers] (#market-data-consumers).

Overview

The market status management system is responsible for consuming market status update messa

```
##### Market Status Processing Architecture
```

![[Chart 1](realtime-v2-images/realtime-v2-section9-1.svg)]

****Sources:**** [] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/t>

```
[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java:51] (http://www.javacodegists.com/snippets/using-logback-1.2.3-with-log4j-2.0.0-beta4-1)
```

```
#### Market Status Update Handler
```

The `MarketStatusUpdateHandler` serves as the primary consumer for market status updates, e

Handler Configuration

The handler is configured during initialization with the following key components:

```
| Component | Purpose | Configuration Source |
|---|---|---|
| `ObjectMapper` | JSON deserialization | Injected dependency |
| `AppConf` | Application configuration | Configuration properties |
| `MarketStatusService` | Business logic processing | Service layer |
| Topic subscription | Kafka topic binding | `appConf.getTopics().getMarketStatus()` |
| Threading | Consumer thread pool | `appConf.getMaxThread()` |
```

The handler initialization occurs in the constructor at [\[1\]](https://github.com/nhsv-tradex/re) (<https://github.com/nhsv-tradex/re>)

[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java22-33] (ht

****Sources:**** [] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/t>

```
[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java22-33] (ht
```

```
##### Message Processing Flow
```

! [Chart 2] (realtime-v2-images/realtime-v2-section9-2.svg)

```

**Sources:**[] (https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L35-L49) (https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L35-L49) (https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L35-L49)

#### Message Processing Implementation

##### Data Validation and Error Handling

The handler implements robust data validation before processing market status updates:

```

```

public void handle(Message message) {
    if (message == null

```

	message.getData() == null) {
return;	
}	
// Processing continues...	
}	
...	

This validation ensures that only well-formed messages with valid market status data proceed to business logic processing.

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L36-L39>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L36-L39>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L36-L39>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L36-L39>)

Performance Monitoring

The handler includes built-in performance monitoring that measures processing time for each market status update:

- Start time capture before service call:[

```

src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java41]
(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L41-L41) (https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L41-L41)

```

- End time capture after processing:[

```

src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java44]
(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L44-L44) (https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L44-L44)

```

- Performance logging:[

```

src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java45]
(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L45-L45)

```

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L45-L45](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L45-L45) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L45-L45>))

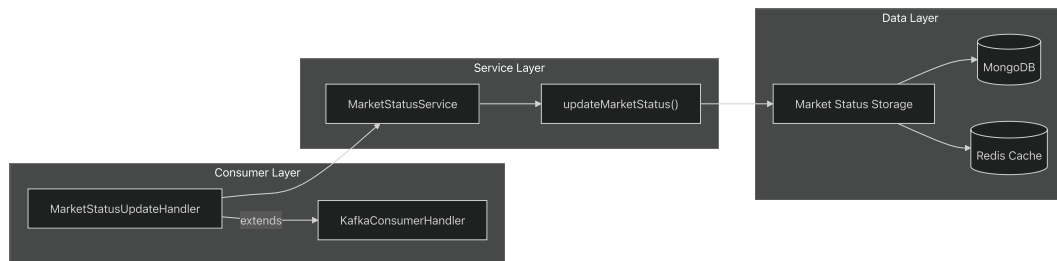
This timing information helps monitor system performance and identify potential bottlenecks in market status processing.

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L41-L45>)
[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L41-L45](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L41-L45) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L41-L45>)

Integration Architecture

Service Layer Integration

The `MarketStatusUpdateHandler` delegates business logic processing to the `MarketStatusService`, maintaining separation of concerns between message consumption and business logic:



Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L16-L19>)
[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L16-19](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L16-L19) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L16-L19>)

[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#43](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L43-L43) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L43-L43>)

Configuration Dependencies

The handler relies on several configuration components from the `AppConf` system:

Configuration Property	Usage	Method Call
Kafka bootstrap servers	Consumer connection	<code>appConf.getKafkaBootstraps()</code>
Cluster ID	Consumer group identification	<code>appConf.getClusterId()</code>
Market status topic	Topic subscription	<code>appConf.getTopics().getMarketStatus()</code>
Thread pool size	Consumer concurrency	<code>appConf.getMaxThread()</code>

These configurations enable flexible deployment across different environments while maintaining consistent processing behavior.

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L30-L32>)

[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java30-32](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L30-L32) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L30-L32>)

Error Handling and Logging

The market status handler implements comprehensive error handling with structured logging:

- **Invalid data logging:** Captures cases where messages or data are null
- **Processing error logging:** Records exceptions during service calls with message ID context
- **Performance logging:** Tracks processing time for successful operations
- **Exception context:** Includes message ID for debugging failed processing attempts

All logging uses the SLF4J framework with appropriate log levels for different scenarios, enabling effective monitoring and troubleshooting in production environments.

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L17-L17>)

[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java17](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L17-L17) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L17-L17>)

[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java38](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L38-L38) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L38-L38>)

[src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java47](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L47-L47) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/MarketStatusUpdateHandler.java#L47-L47>)

Symbol Information Processing

Relevant source files

- [[src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java>)
- [[src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java>)

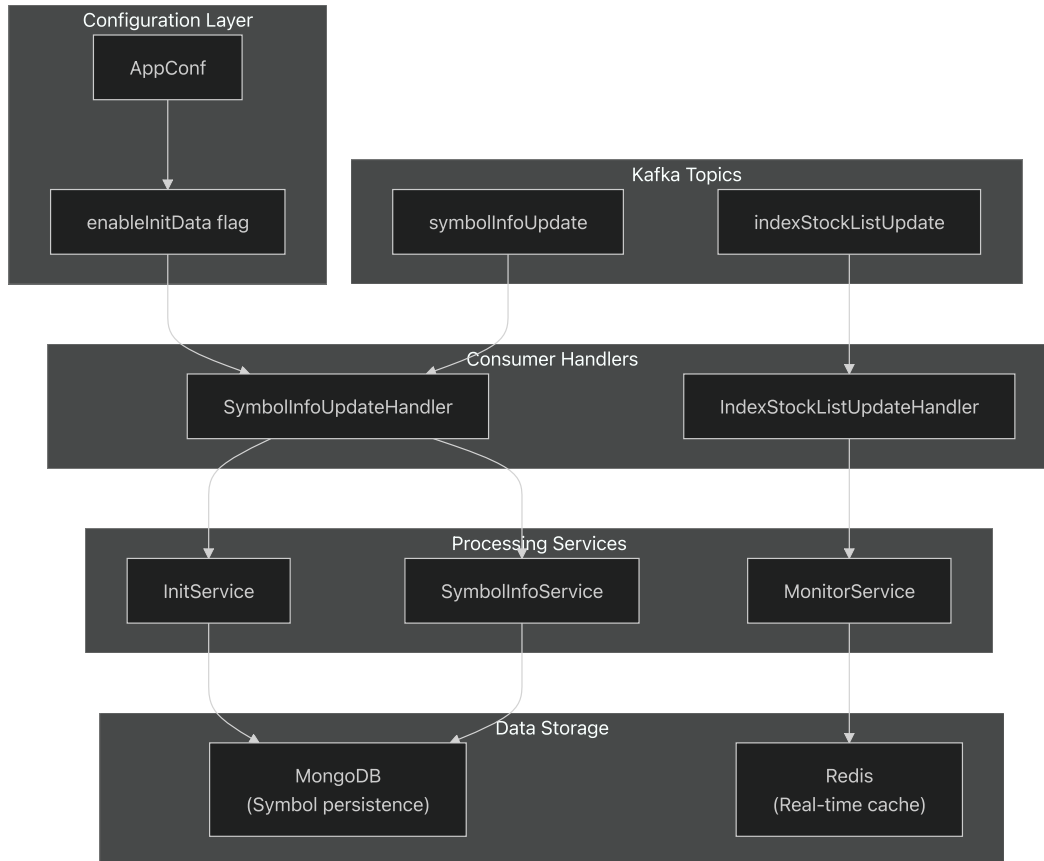
Purpose and Scope

This document covers the symbol information processing subsystem within the TradEx realtime data pipeline. This subsystem handles real-time updates to symbol metadata, including individual symbol information changes and bulk index stock list updates. The processing involves consuming Kafka messages containing symbol updates and routing them to appropriate services based on system configuration.

For information about market data quote processing, see [Quote Processing](#). For market status updates, see [Market Status Management](#).

System Architecture

The symbol information processing system operates as part of the broader real-time data processing pipeline, consuming symbol-related messages from Kafka topics and processing them through specialized services.



Sources: ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L17-L58)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L17-L58](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L17-L58)).

[src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L17-58](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L17-L58) ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L17-L58)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L17-L58](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L17-L58))

[src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L17-58](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L17-L58) ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L17-L58)

Kafka Consumer Handlers

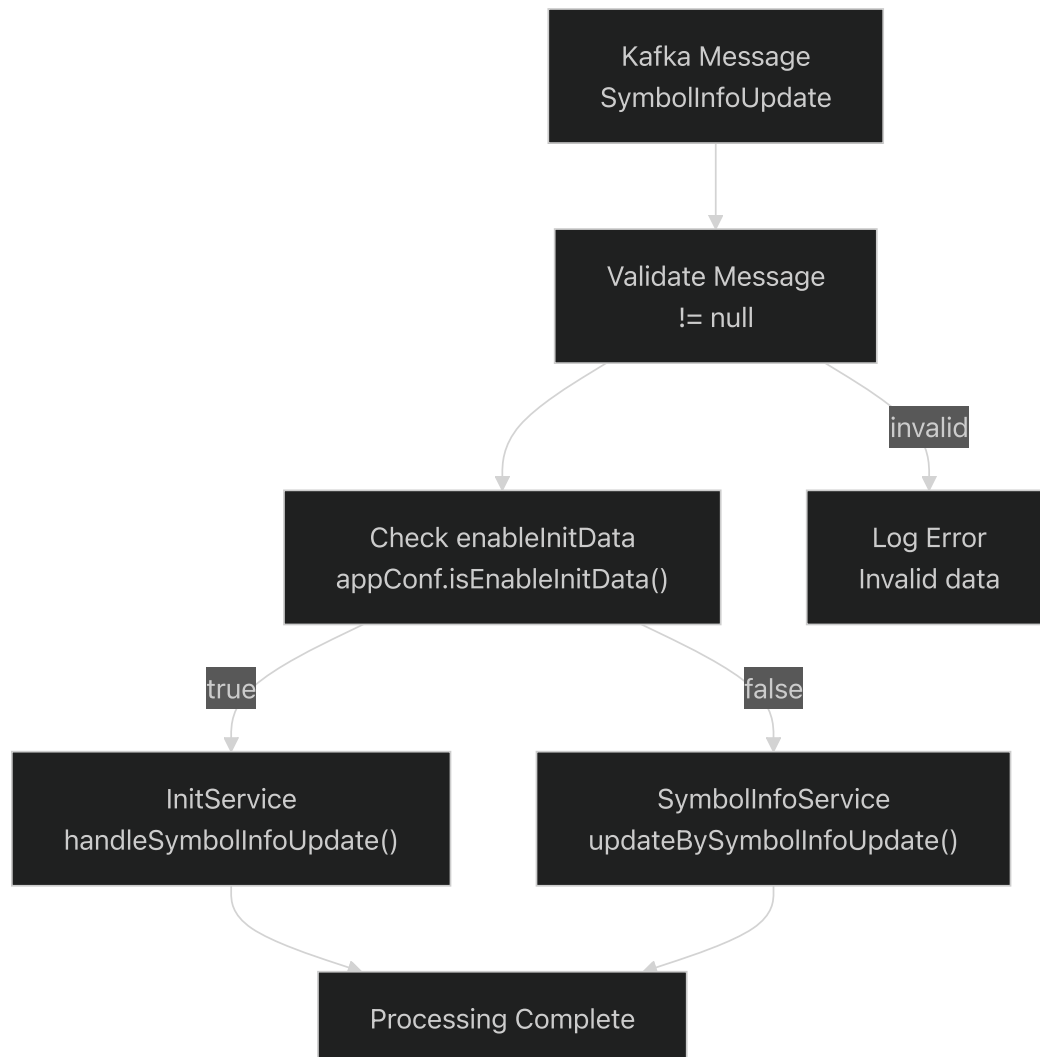
The symbol information processing system includes two primary Kafka consumer handlers that serve as entry points for different types of symbol-related data.

SymbolInfoUpdateHandler

The `SymbolInfoUpdateHandler` processes individual symbol information updates received from the `symbolInfoUpdate` Kafka topic.

Component	Description
Type Reference	<code>Message<SymbolInfoUpdate></code>
Topic	Configured via <code>appConf.getTopics().getSymbolInfoUpdate()</code>
Thread Pool	Configured via <code>appConf.getMaxThread()</code> for the topic
Dependencies	<code>InitService</code> , <code>SymbolInfoService</code> , <code>AppConf</code>

The handler implements conditional processing logic based on the `enableInitData` configuration flag:



Sources: ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L41-L56)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L41-L56](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L41-L56))

[src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java41-56](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L41-L56) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L41-L56>)

IndexStockListUpdateHandler

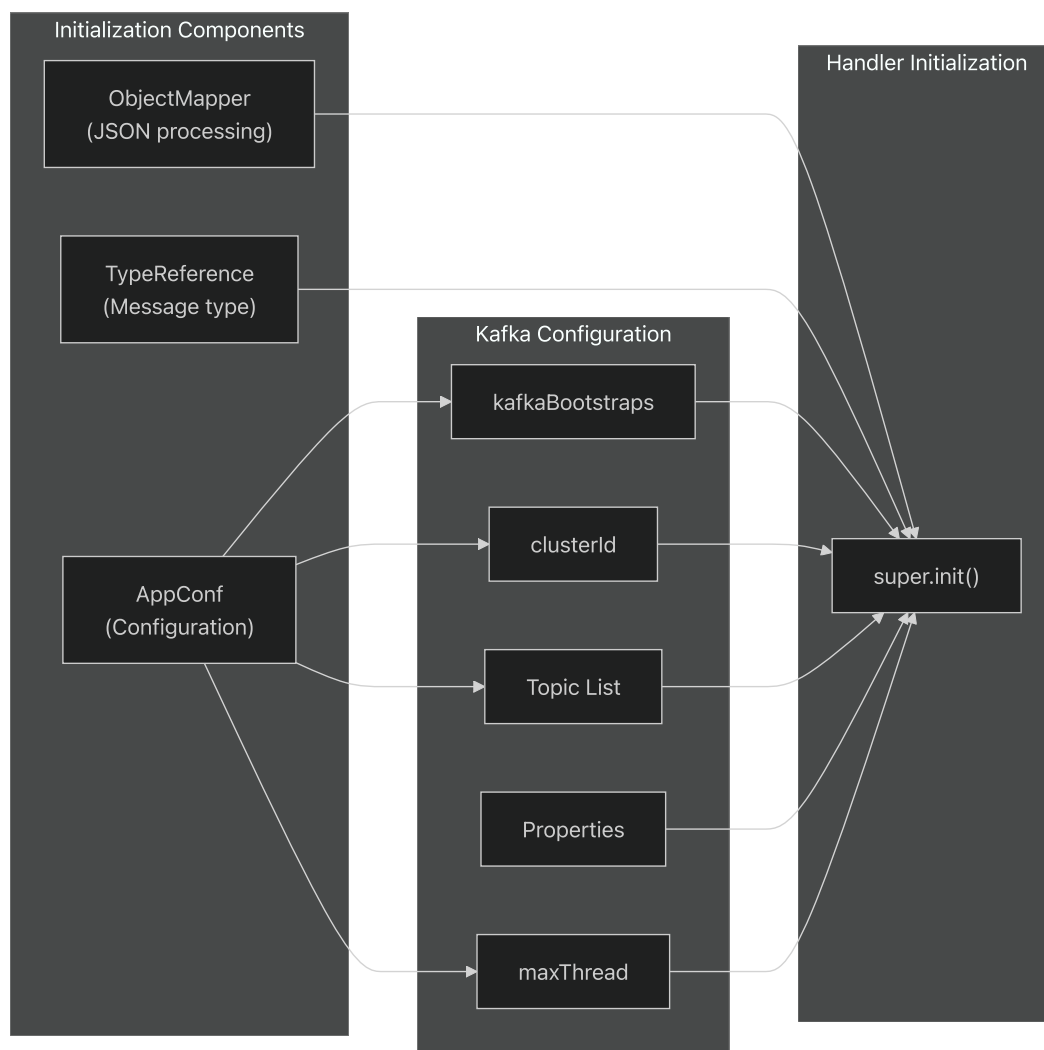
The `IndexStockListUpdateHandler` processes bulk index stock list updates from the `indexStockListUpdate` Kafka topic.

Component	Description
Type Reference	<code>Message<IndexStockList></code>
Topic	Configured via <code>appConf.getTopics().getIndexStockListUpdate()</code>
Dependencies	<code>MonitorService</code>
Processing	Direct delegation to <code>MonitorService.rcv()</code>

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java#L35-L42>
[src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java35-42](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java#L35-L42)
<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java#L35-L42>

Consumer Handler Configuration

Both handlers extend the `KafkaConsumerHandler` base class and are configured with similar initialization patterns:



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L27-L38>

[src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java27-38](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L27-L38) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L27-L38>)

src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java23-32] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java#L23-L32>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java#L23-L32>)

Error Handling and Logging

The symbol information processing system implements consistent error handling patterns across both consumer handlers:

Handler	Error Handling Approach
SymbolInfoUpdateHandler	Try-catch block around processing logic with message ID logging
IndexStockListUpdateHandler	Null validation with error logging for invalid data

Both handlers use SLF4J loggers for consistent logging throughout the processing pipeline:

- Invalid message validation errors
- Processing exceptions with message IDs for debugging
- Error context preservation for troubleshooting

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L53-L55>)

[src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java53-55](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L53-L55) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/SymbolInfoUpdateHandler.java#L53-L55>)

src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java36-39] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java#L36-L39>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/IndexStockListUpdateHandler.java#L36-L39>)

Deal and Advertisement Processing

Relevant source files

- [
src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java]
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java>)
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java>)
- [
src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java]
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java>)
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java>)

This document describes the system components responsible for processing real-time deal notices and advertisements in the tradex-realtime application. These components consume market data from Kafka topics and forward the processed data to the monitoring system for further handling.

For information about quote and bid/offer processing, see [Quote Processing](#) and [Bid/Offer Processing](#). For broader market data consumer architecture, see [Market Data Consumers](#).

Purpose and Scope

The deal and advertisement processing subsystem handles two specific types of market data:

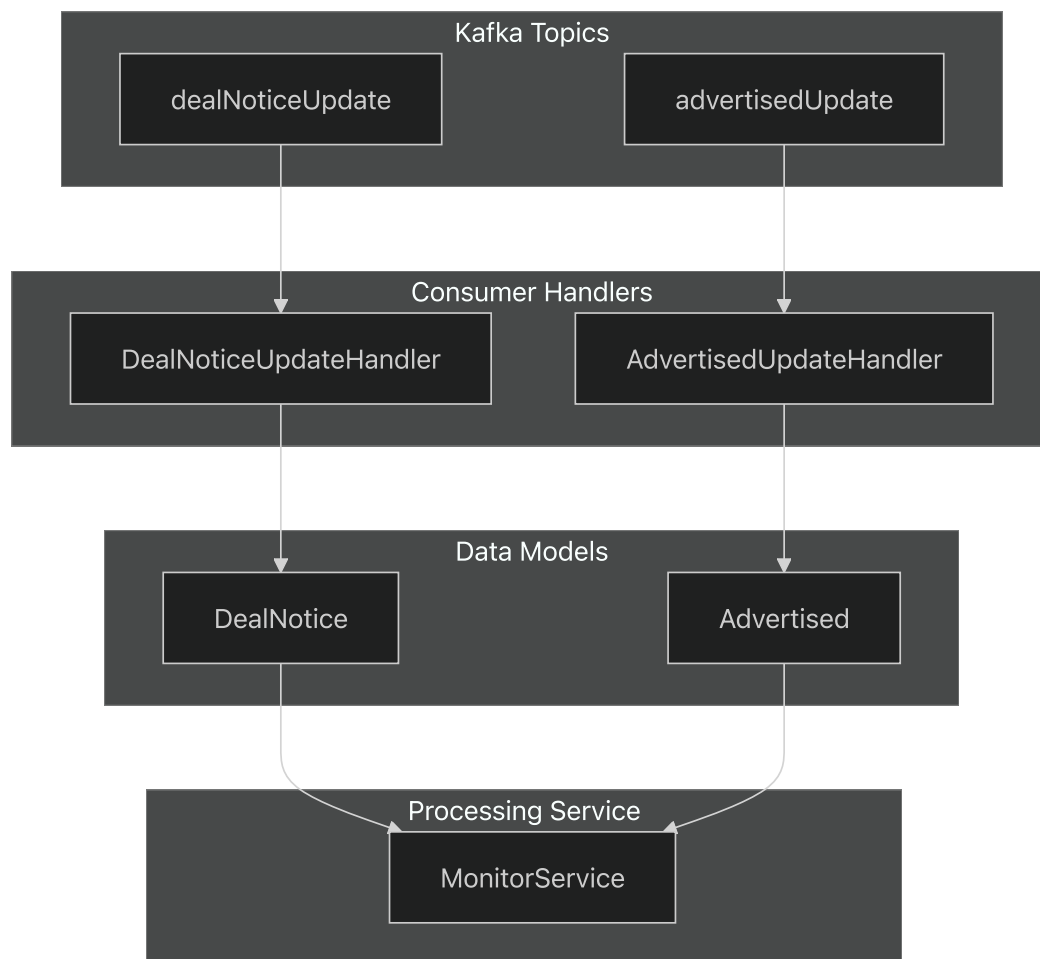
- **Deal Notices:** Transaction records representing completed trades
- **Advertisements:** Market advertisement data from trading participants

Both data types are consumed from dedicated Kafka topics and processed through the `MonitorService` for real-time data aggregation and caching.

Architecture Overview

The processing architecture follows the standard Kafka consumer pattern used throughout the system, with dedicated handlers for each data type that delegate to the centralized monitoring service.

Deal and Advertisement Consumer Architecture



Sources: <https://github.com/nhsv-tradex/realtime->

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L19-L61](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L19-L61).

[src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java19-61](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L19-L61) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L19-L61>)

src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java16-41]
<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L16-L41>
<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L16-L41>))

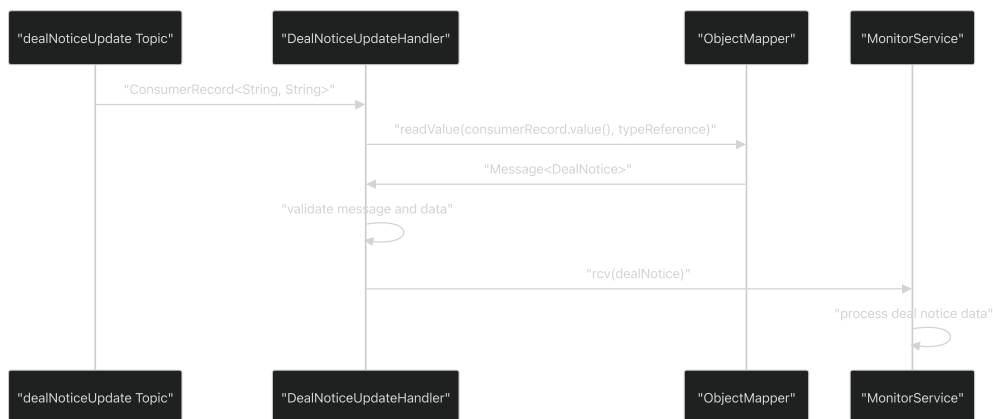
Deal Notice Processing

The `DealNoticeUpdateHandler` processes transaction completion notifications from the `dealNoticeUpdate` Kafka topic. This handler uses a manual consumer implementation approach.

DealNoticeUpdateHandler Implementation

Component	Details
Base Class	<code>ConsumerHandler<String, String></code>
Topic	<code>appConf.getTopics().getDealNoticeUpdate()</code>
Data Type	<code>DealNotice</code>
Processing Method	<code>MonitorService.rcv(dealNotice)</code>
Consumer Type	<code>ThreadedKafkaConsumer</code>

The handler follows this processing flow:



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L40-L59>.

[src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java40-59](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L40-L59) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L40-L59>)

Deal Notice Message Flow

The `DealNoticeUpdateHandler` implements manual message deserialization and validation:

1. **Message Reception:** Receives `ConsumerRecord<String, String>` from Kafka[

src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java41]
<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L41-L41> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L41-L41>)

2. **Deserialization:** Uses `ObjectMapper` with `TypeReference<Message<DealNotice>>` to deserialize JSON[

src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java44]
<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L44-L44> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L44-L44>))

3. **Validation:** Checks for null message or null data[

src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java53-56]
<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L53-L56> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L53-L56>))

4. **Processing:** Delegates to `MonitorService.rcv(dealNotice)` [

src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java58]
<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L58-L58> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L58-L58>))

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L19-L61>).

[src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java19-61](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L19-L61) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L19-L61>)

Advertisement Processing

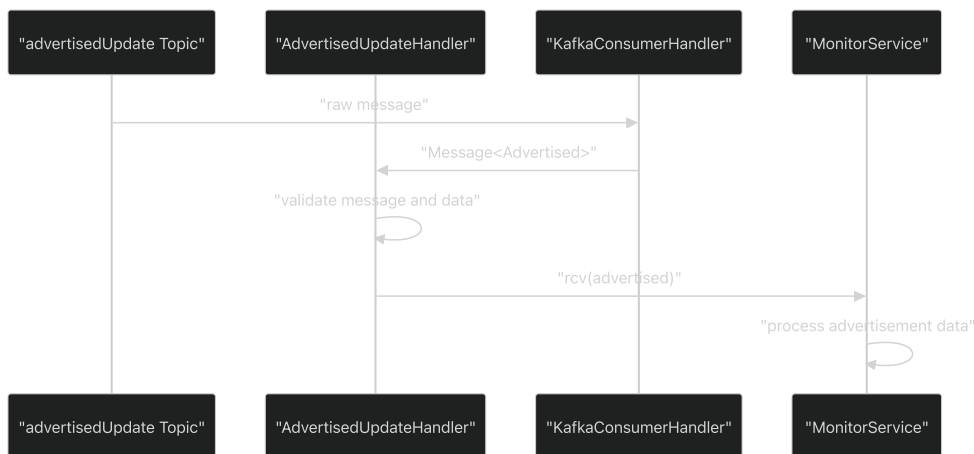
The `AdvertisedUpdateHandler` processes market advertisement data from the `advertisedUpdate` Kafka topic using an abstracted consumer base class.

AdvertisedUpdateHandler Implementation

Component	Details
Base Class	<code>KafkaConsumerHandler<Advertised></code>
Topic	<code>appConf.getTopics().getAdvertisedUpdate()</code>
Data Type	<code>Advertised</code>
Processing Method	<code>MonitorService.rcv(advertised)</code>
Consumer Type	Abstracted through base class

Advertisement Message Processing

The `AdvertisedUpdateHandler` uses a simpler, more abstracted approach:



Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L32-L39>).

<src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L32-L39> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L32-L39>).

The processing flow is streamlined:

1. **Message Reception:** Receives pre-processed `Message<Advised>` from base class[

`src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L32`
<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L32-L39> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L32-L39>)

2. **Validation:** Performs null checks on message and data[

`src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L33-36`
<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L33-L36> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L33-L36>)

3. **Processing:** Delegates to `MonitorService.rcv(advised)` [

`src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L38`
<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L38-L41> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L38-L41>)

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L16-L41>).

<src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L16-L41> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvisedUpdateHandler.java#L16-L41>).

Implementation Differences

The two handlers demonstrate different approaches to Kafka consumption within the same application:

Consumer Pattern Comparison

Aspect	DealNoticeUpdateHandler	AdvertisedUpdateHandler
Base Class	<code>ConsumerHandler<String, String></code>	<code>KafkaConsumerHandler<Advertised></code>
Consumer Setup	Manual <code>ThreadedKafkaConsumer</code>	Inherited from base class
Message Handling	Manual deserialization	Automatic deserialization
Type Safety	Runtime type checking	Compile-time type safety
Code Complexity	Higher (manual handling)	Lower (abstracted handling)

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L19-L19> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L19-L19>) [<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L19-L19>]

<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L16-L16> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L16-L16>)

Configuration and Threading

Both handlers are configured through the `AppConf` configuration system:

Thread Configuration

- **Deal Notice Handler:** Uses `appConf.getMaxThread(appConf.getTopics().getDealNoticeUpdate())` [<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L35-L35>] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L35-L35>)
- **Advertisement Handler:** Uses `appConf.getMaxThread(appConf.getTopics().getAdvertisedUpdate())` [<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L29-L29>] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L29-L29>)

Kafka Configuration

Both handlers share common configuration patterns:

- **Bootstrap Servers:** `appConf.getKafkaBoostraps()`
- **Cluster ID:** `appConf.getClusterId()`
- **Consumer Properties:** `new Properties()` (default configuration)
- **Topic Lists:** Single-topic consumption using `Collections.singletonList()`

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L33-L35>

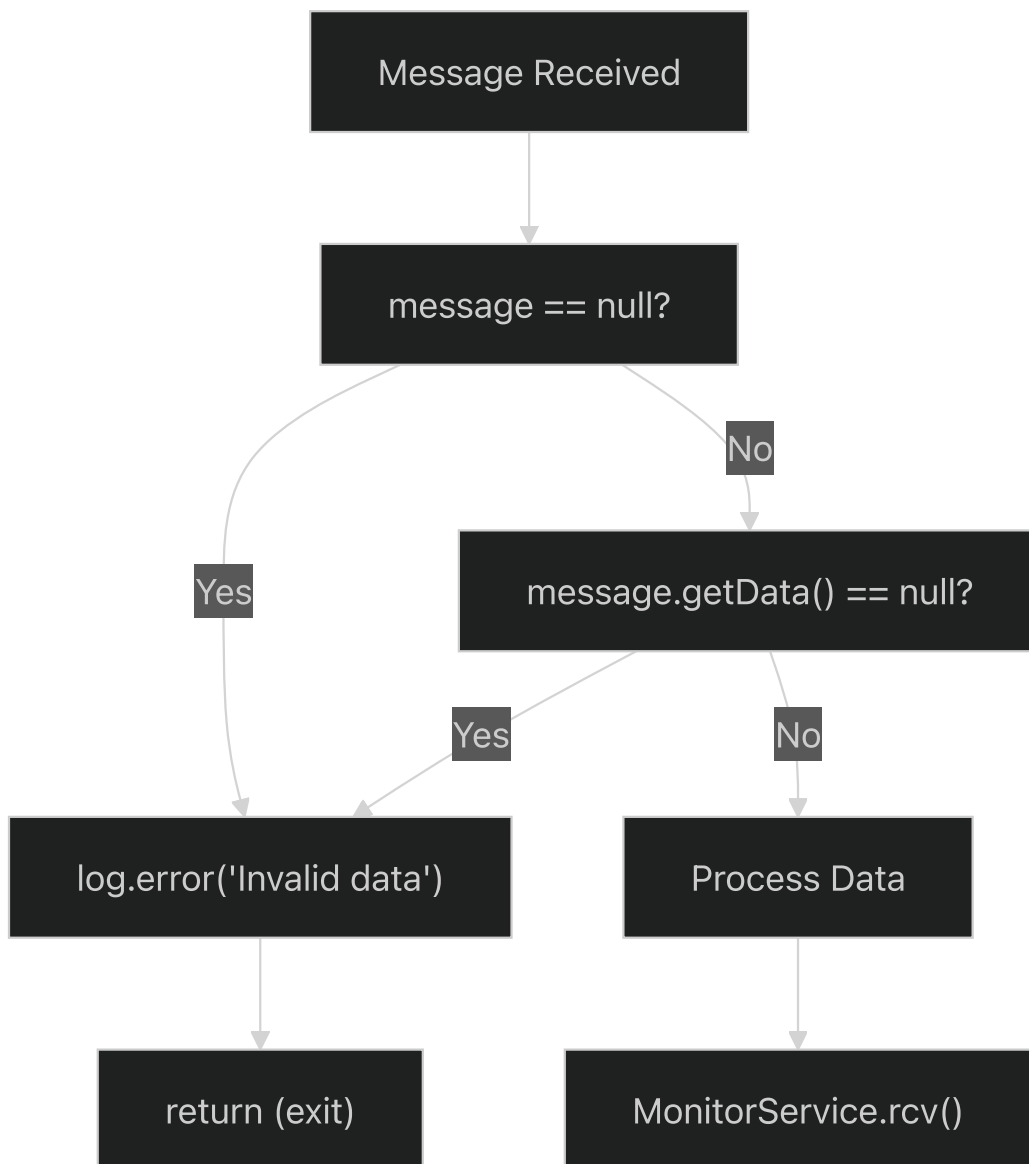
[src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java33-35](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L33-L35) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L33-L35>)

[src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java26-29](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L26-L29) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L26-L29>)
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L26-L29>)

Error Handling

Both handlers implement similar error handling strategies:

Validation Patterns



Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L53-L56>).

[src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java53-56](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L53-L56) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L53-L56>)

[src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java33-36](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L33-36) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L33-36>)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L33-L36](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L33-L36)
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L33-L36>))

Additional Error Handling

The `DealNoticeUpdateHandler` includes JSON deserialization error handling:

- Catches `IOException` during message parsing[

`src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java`45-48]

(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L45-L48>

[L48](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L45-L48) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L45-L48>))

- Logs the problematic message content for debugging
- Gracefully exits processing without crashing the consumer thread

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L42-L50>).

[src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L42-L50)42-50 (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L42-L50>)

Integration with Monitoring System

Both handlers delegate processing to the `MonitorService` through the `rcv()` method, maintaining consistency with other market data processors in the system. This centralized approach ensures uniform handling of different data types within the monitoring and caching infrastructure.

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L58-L58>).

[src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L58-L58)58 (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L58-L58>)

[src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L58-L58)58 (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/DealNoticeUpdateHandler.java#L58-L58>)

`src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java`38](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L38-L38>

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L38-L38](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L38-L38)

(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L38-L38>))

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L38-L38](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/AdvertisedUpdateHandler.java#L38-L38)))

Job and Request Handling

Relevant source files

- [

`src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java`](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java>

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java>))

- [

`src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java`](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java>

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java>

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java)))

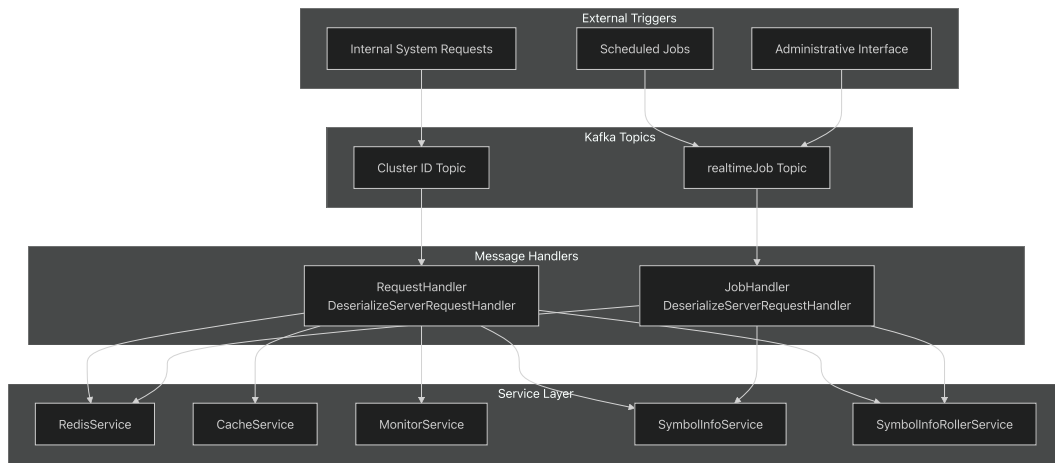
This document covers the administrative and operational task handling mechanisms in the tradex-realtime system. The system provides two distinct pathways for executing background jobs and handling internal requests through Kafka-based message consumers. For information about market data processing consumers, see [3.1](#).

Purpose and Architecture

The job and request handling system enables remote execution of administrative tasks and operational commands through Kafka messaging. Two specialized handlers manage different types of operations:

`JobHandler` for scheduled and administrative jobs, and `RequestHandler` for internal system requests and recovery operations.

System Overview



Sources: <https://github.com/nhsv-tradex/realtime->

<v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L1-L46>

<src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L1-L46> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L1-L46>)

<src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L1-L47> (<https://github.com/nhsv-tradex/realtime->

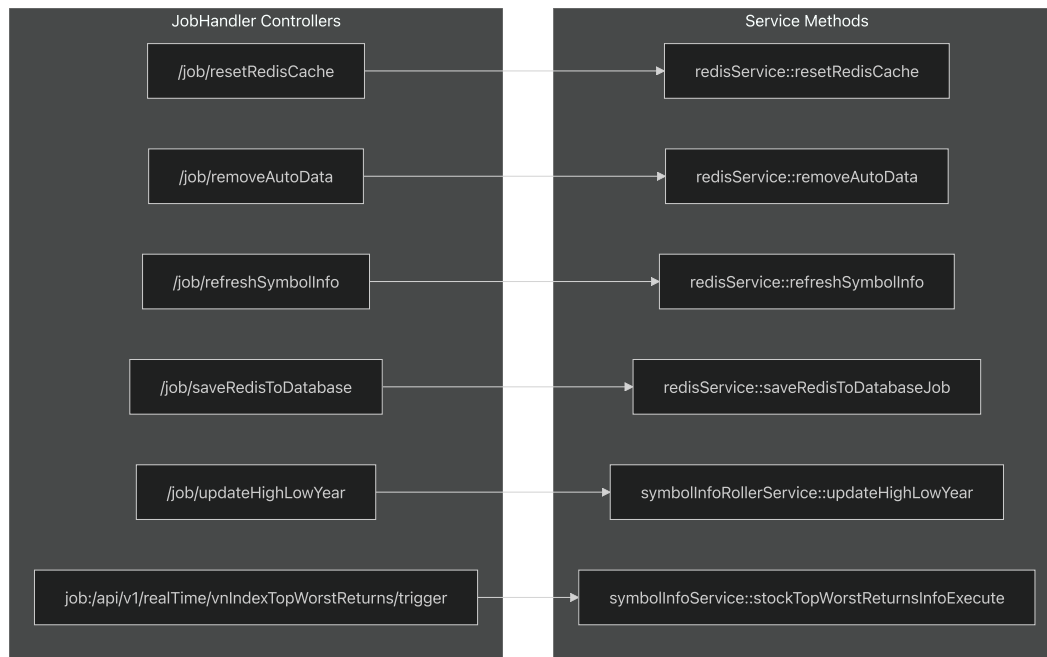
<v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L1-L47>

(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L1-L47>))

JobHandler Implementation

The `JobHandler` class extends `DeserializeServerRequestHandler` and manages administrative jobs through the `realtimeJob` Kafka topic. It implements a controller mapping pattern where each job endpoint is mapped to specific service operations.

Job Controller Mappings



Job Endpoint	Service Method	Pu
/job/resetRedisCache	redisService::resetRedisCache	Cle cac
/job/removeAutoData	redisService::removeAutoData	Rem aut ger dat
/job/refreshSymbolInfo	redisService::refreshSymbolInfo	Ref syn inf cac
/job/saveRedisToDatabase	redisService::saveRedisToDatabaseJob	Per dat dat
/job/updateHighLowYear	symbolInfoRollerService::updateHighLowYear	Up yea hig dat
job:/api/v1/realTime/vnIndexTopWorstReturns/trigger	symbolInfoService::stockTopWorstReturnsInfoExecute	Cal sto per me

Sources: [https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L32-L39)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L32-L39](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L32-L39)

[src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java32-39](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L32-L39) ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L32-L39)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L32-L39](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L32-L39))

RequestHandler Implementation

The `RequestHandler` class manages internal system requests and recovery operations through the cluster ID topic. It provides endpoints for data recovery, cache management, and system maintenance operations.

Request Controller Mappings



Request Endpoint	Service Method	Input Type	Purpose
/reloadSymbolInfo	redisService::reloadSymbolInfo	Object	Reload symbol information
/mergeWrongOrder	monitorService::doMergeWrongOrderQuote	String	Fix quote ordering issues
/recoverQuoteMinute	monitorService::doRecoverQuoteMinute	String	Recover minute-level quote data
/recoverAllQuoteMinute	monitorService::recoverAllMinute	Integer	Recover all minute data
/reloadSymbolDaily	redisService::reloadSymbolDaily	Object	Reload daily symbol data
/resetCache	cacheService::reset	Object	Reset application cache
/calculateRoller	symbolInfoRollerService::rollerData	Object	Calculate rolling statistics
/uploadMarketStaticFile	symbolInfoService::uploadMarketStaticFile	Object	Upload market static files

Sources: <https://github.com/nhsv-tradex/realtime->

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L32-L40](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L32-L40)

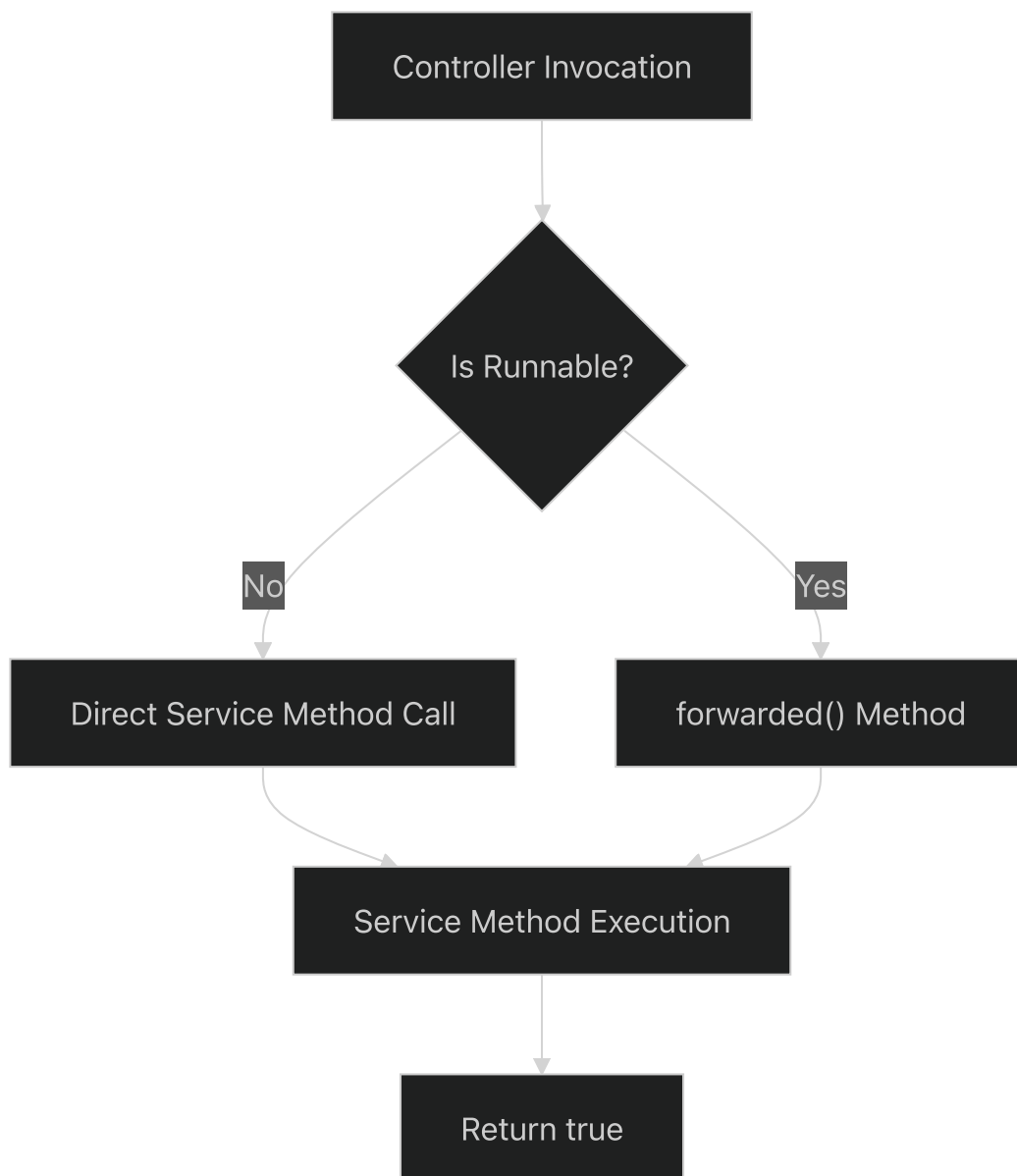
[src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java32-40](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L32-L40) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L32-L40>).

Controller Pattern Implementation

Both handlers use a common pattern where controller mappings are defined during construction and operations are delegated to appropriate service methods.

Forwarding Mechanism

The `forwarded` method in both classes provides a standardized way to execute `Runnable` operations:



The `forwarded` method (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L42-L45>).

[src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java42-45](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L42-L45) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L42-L45>) executes the provided `Runnable` and returns `true`, providing a consistent return pattern for void operations.

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L42-L45>).

[src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java42-45](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L42-L45) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L42-L45>)

[src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java43-46](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L43-L46) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L43-L46>)

Service Integration

Both handlers integrate with multiple service layers to perform their operations:

Service Dependencies

Handler	Service Dependencies
JobHandler	RedisService , SymbolInfoRollerService , SymbolInfoService
RequestHandler	RedisService , CacheService , SymbolInfoRollerService , MonitorService , SymbolInfoService

Configuration Integration

Both handlers are configured through `AppConf` for:

- Kafka bootstrap servers via `appConf.getKafkaBootstraps()`
- Topic configuration via `appConf.getTopics().getRealtimeJob()` and `appConf.getClusterId()`
- Consumer group sizing (both use 5 consumers)

Sources: ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L24-L31)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L24-L31](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L24-L31))

[src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java24-31](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L24-L31) ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/JobHandler.java#L24-L31)

[src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java21-30](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L21-L30)) ([https://github.com/nhsv-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L21-L30)

[tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L21-L30)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L21-L30](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L21-L30) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/consumers/RequestHandler.java#L21-L30>))

Data Models and Constants

Overview

Relevant source files

- [[src/main/java/com/techx/tradex/realtime/constants/Constants.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java)] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java>)
- [[src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java)] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java>)

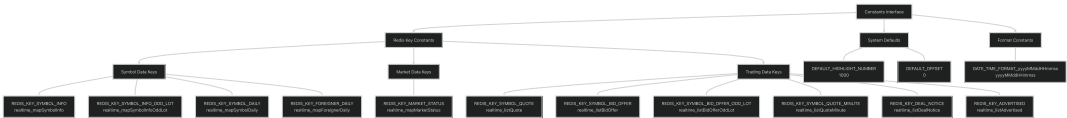
This document provides a comprehensive overview of the data models, DTOs, constants, and enumerations used throughout the tradex-realtime system. These foundational elements define the structure and behavior of market data processing, Redis cache keys, and system-wide configuration values.

The material covers the core data structures that flow through the real-time processing pipeline and the constants that govern system behavior. For information about how these models are processed by specific handlers, see [Real-time Data Processing](#). For details on how constants are configured and managed, see [Configuration System](#).

System Constants Architecture

The system relies on a centralized constants interface that defines Redis cache keys, default values, and system-wide configuration parameters. These constants provide a single source of truth for cache key management and system behavior.

Redis Key Constants Structure



The constants are organized into logical groupings that correspond to different data types and system functions. Each Redis key follows a consistent naming convention with the `realtime_` prefix.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L1-L23> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L1-L23>).

Constants Usage Patterns

Constant Type	Pattern	Usage
Symbol Data	<code>realtime_mapSymbol*</code>	Hash-based storage for symbol information
List Data	<code>realtime_list*</code>	List-based storage for time-series data
Market Data	<code>realtime_mapMarket*</code>	Hash-based storage for market status
Configuration	<code>DEFAULT_*</code>	System default values
Formats	<code>*_FORMAT_*</code>	Date/time format specifications

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L4-L22> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L4-L22>).

Data Model Overview

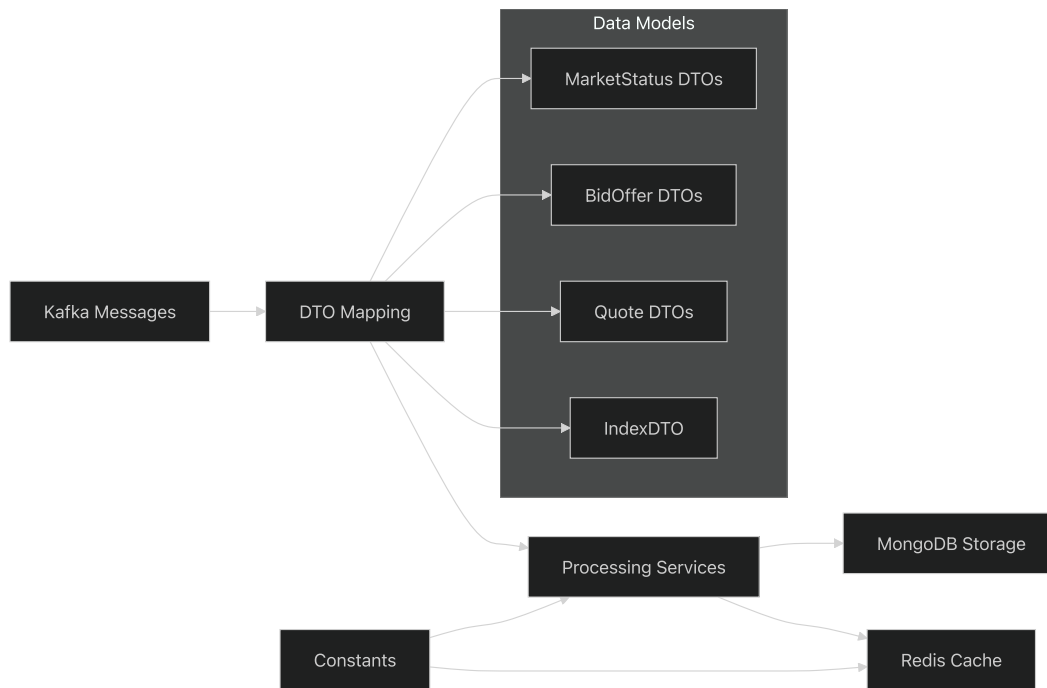
The system employs a variety of data models that represent different aspects of market data and system entities. These models serve as the foundation for data transfer between system components.

DTO Structure

The `IndexDTO` class represents the basic structure for index-related data transfers, containing the essential `stockCode` field that serves as a primary identifier for market instruments.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L1-L9> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L1-L9>).

Data Flow Integration



This diagram illustrates how constants and data models integrate with the broader system architecture, providing the structural foundation for data processing and storage operations.

Sources: [https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L1-L23)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L1-L23](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L1-L23)

[src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L1-L9](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L1-L9) ([https://github.com/nhsv-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L1-L9)

[tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L1-L9](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L1-L9))

[src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L1-L9](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L1-L9) ([https://github.com/nhsv-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L1-L9)

Constant Categories and Applications

Redis Key Management

The system uses a comprehensive set of Redis keys to organize different types of market data:

Redis Key	Data Type	Purpose
REDIS_KEY_SYMBOL_INFO	Hash Map	Standard symbol information
REDIS_KEY_SYMBOL_INFO_ODD_LOT	Hash Map	Odd lot symbol information
REDIS_KEY_SYMBOL_DAILY	Hash Map	Daily symbol statistics
REDIS_KEY_FOREIGNER_DAILY	Hash Map	Foreign investor daily data
REDIS_KEY_SYMBOL_QUOTE	List	Real-time quote updates
REDIS_KEY_SYMBOL_BID_OFFER	List	Bid/offer data
REDIS_KEY_SYMBOL_BID_OFFER_ODD_LOT	List	Odd lot bid/offer data
REDIS_KEY_SYMBOL_QUOTE_MINUTE	List	Minute-level quote data
REDIS_KEY_DEAL_NOTICE	List	Deal notice updates
REDIS_KEY_ADVERTISED	List	Advertisement data
REDIS_KEY_MARKET_STATUS	Hash Map	Market status information

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L4-L14>)

[src/main/java/com/techx/tradex/realtime/constants/Constants.java4-14](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L4-L14) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L4-L14>)

System Configuration Constants

The constants interface defines several system-wide configuration values:

- `DEFAULT_HIGHLIGHT_NUMBER` : Set to 1000, controls highlighting behavior in the UI
- `DEFAULT_OFFSET` : Set to 0, provides default pagination offset
- `DATE_TIME_FORMAT_yyyyMMddHHmmss` : Standardizes date/time formatting across the system
- `KAFKA_REQUEST_ASYNC` : Identifier for asynchronous Kafka request processing
- `INDEX_NOT_FOUND` and `VN_INDEX` : Constants related to Vietnamese market index handling

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L15-L21>)

[src/main/java/com/techx/tradex/realtime/constants/Constants.java15-21](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L15-L21) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L15-L21>)

Notification and Template Constants

The system includes constants for notification management:

- `PAAVE_STOCK_TOP_WORST_RETURNS_NOTIFICATION_TEMPLATE_NAME` : Template name for stock performance notifications

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L18-L18>)

[src/main/java/com/techx/tradex/realtime/constants/Constants.java18](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L18-L18) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L18-L18>)

Model Design Patterns

Lombok Integration

The data models utilize Lombok annotations for boilerplate code generation, as evidenced by the `@Data` annotation in `IndexDTO`. This pattern reduces code verbosity while maintaining full getter/setter functionality.

Inheritance Hierarchy

The DTO classes follow a hierarchical structure where specific DTOs like `IndexDTO` inherit from base classes, promoting code reuse and consistent field management across different data types.

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L3-L8>)

[src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java3-8](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L3-L8) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/dto/IndexDTO.java#L3-L8>)

System Constants

Relevant source files

- [

`src/main/java/com/techx/tradex/realtime/constants/Constants.java` (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java>)

This document describes the system constants defined in the tradex-realtime application, primarily focusing on Redis key patterns, default values, and system identifiers used throughout the real-time market data processing pipeline. These constants provide a centralized definition of key naming conventions and default values that ensure consistency across all components of the system.

For information about configuration properties and application settings, see [Application Configuration](#). For details about enumerations used for type-safe constants, see [Enumerations](#).

Redis Key Constants

The majority of system constants define Redis key patterns used for storing and retrieving real-time market data. These keys follow a consistent naming convention with the prefix `realtime_` followed by a descriptive suffix indicating the data type and structure.

Data Structure Mapping



Symbol Information Keys

Constant	Redis Key	Data Structure	Purpose
REDIS_KEY_SYMBOL_INFO	realtime_mapSymbolInfo	Hash Map	Primary symbol information storage
REDIS_KEY_SYMBOL_INFO_ODD_LOT	realtime_mapSymbolInfoOddLot	Hash Map	Symbol information for odd lot trading
REDIS_KEY_SYMBOL_DAILY	realtime_mapSymbolDaily	Hash Map	Daily aggregated symbol data
REDIS_KEY_FOREIGNER_DAILY	realtime_mapForeignerDaily	Hash Map	Daily foreign investor activity

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L4-L7>),
<src/main/java/com/techx/tradex/realtime/constants/Constants.java4-7> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L4-L7>).

Market Data Keys

Constant	Redis Key	Data Structure	Purpose
REDIS_KEY_SYMBOL_QUOTE	realtime_listQuote	List	Real-time quote updates
REDIS_KEY_SYMBOL_BID_OFFER	realtime_listBidOffer	List	Bid and offer data
REDIS_KEY_SYMBOL_BID_OFFER_ODD_LOT	realtime_listBidOfferOddLot	List	Odd lot bid/offer data
REDIS_KEY_SYMBOL_QUOTE_MINUTE	realtime_listQuoteMinute	List	Minute-level quote aggregation

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L8-L11>),
<src/main/java/com/techx/tradex/realtime/constants/Constants.java8-11> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L8-L11>).

Transaction and Status Keys

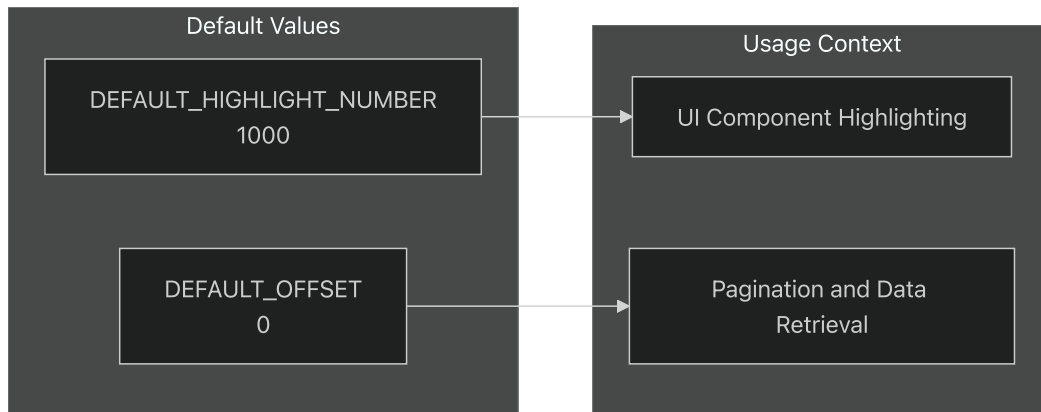
Constant	Redis Key	Data Structure	Purpose
REDIS_KEY_DEAL_NOTICE	realtime_listDealNotice	List	Transaction notifications
REDIS_KEY_ADVERTISED	realtime_listAdvertised	List	Advertised transactions
REDIS_KEY_MARKET_STATUS	realtime_mapMarketStatus	Hash Map	Market status information

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L12-L14>
[src/main/java/com/techx/tradex/realtime/constants/Constants.java12-14](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L12-L14) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L12-L14>)

Default Values and Configuration Constants

The system defines several default values used across various components for consistent behavior and fallback scenarios.

Numeric Defaults



Constant	Value	Purpose
DEFAULT_HIGHLIGHT_NUMBER	1000	Default number for UI highlighting functionality
DEFAULT_OFFSET	0	Default offset for pagination and data queries

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L15-L19>
[src/main/java/com/techx/tradex/realtime/constants/Constants.java15-19](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L15-L19) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L15-L19>)

System Identifiers and Formats

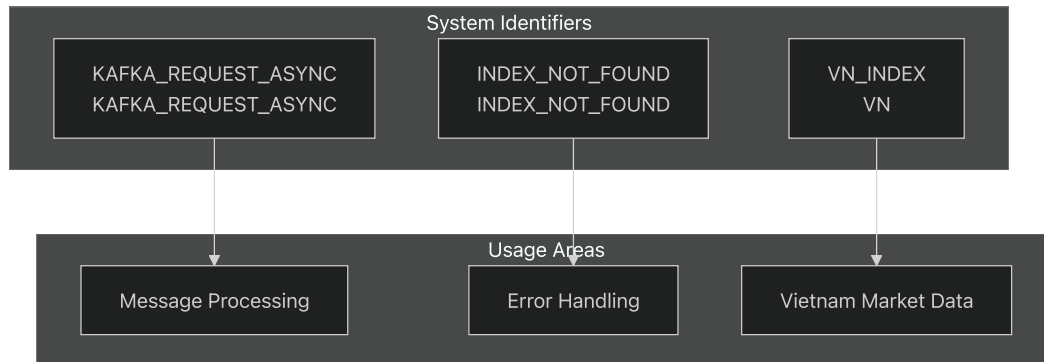
Date and Time Formatting

The system uses a standardized date-time format for consistent data representation:

Constant	Value	Format Description
DATE_TIME_FORMAT_yyyyMMddHHmmss	"yyyyMMddHHmmss"	Compact timestamp format without separators

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L16-L16>
[src/main/java/com/techx/tradex/realtime/constants/Constants.java16](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L16-L16) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L16-L16>

System and Market Identifiers



Constant	Value	Purpose
KAFKA_REQUEST_ASYNC	"KAFKA_REQUEST_ASYNC"	Identifier for asynchronous Kafka request processing
INDEX_NOT_FOUND	"INDEX_NOT_FOUND"	Error indicator for missing index data
VN_INDEX	"VN"	Vietnam market index identifier

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L17-L21>
[src/main/java/com/techx/tradex/realtime/constants/Constants.java17-21](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L17-L21) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L17-L21>

Notification Template Constants

The system includes specialized constants for notification templates used in market analysis and alerts.

Constant	Value	Purpose
PAAVE_STOCK_TOP_WORST_RETURNS_NOTIFICATION_TEMPLATE_NAME	"paave_stock_top_worst_vn_index_returns"	Template name for Vietnam index performance notification

This constant supports the notification system for tracking and alerting on significant market movements, particularly for Vietnamese market indices.

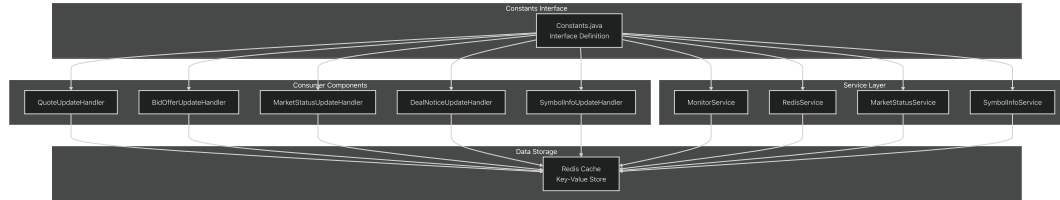
Sources: [https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L18-L18)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L18-L18](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L18-L18))

[src/main/java/com/techx/tradex/realtime/constants/Constants.java18](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L18-L18) ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L18-L18)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L18-L18](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L18-L18))

Constant Usage Throughout the System



The constants defined in this interface are imported and used throughout the application to ensure consistent Redis key naming, error handling, and system identification. This centralized approach prevents key naming conflicts and provides a single source of truth for system-wide constants.

Sources: [https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L1-L24)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L1-L24](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L1-L24))

[src/main/java/com/techx/tradex/realtime/constants/Constants.java1-24](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L1-L24) ([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L1-L24)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L1-L24](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/Constants.java#L1-L24))

Enumerations

Relevant source files

- [[src/main/java/com/techx/tradex/realtime/constants/BidAskType.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/BidAskType.java)] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/BidAskType.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/BidAskType.java>)
- [[src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java)] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java>)
- [[src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeEnum.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeEnum.java)] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeEnum.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeEnum.java>)
- [[src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java)] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java>)

src/main/java/com/techx/tradex/realtime/constants/enums/KafkaDomainEnum.java]
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/KafkaDomainEnum.java>
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/KafkaDomainEnum.java>))

This document details the various enumerations used throughout the tradex-realtime system for type-safe constants and standardized value sets. These enumerations provide compile-time safety and consistency across market data processing, index management, and system configuration components.

For information about system constants and configuration values, see [System Constants](#). For performance monitoring data structures, see [Monitoring and Statistics](#).

Enumeration Categories

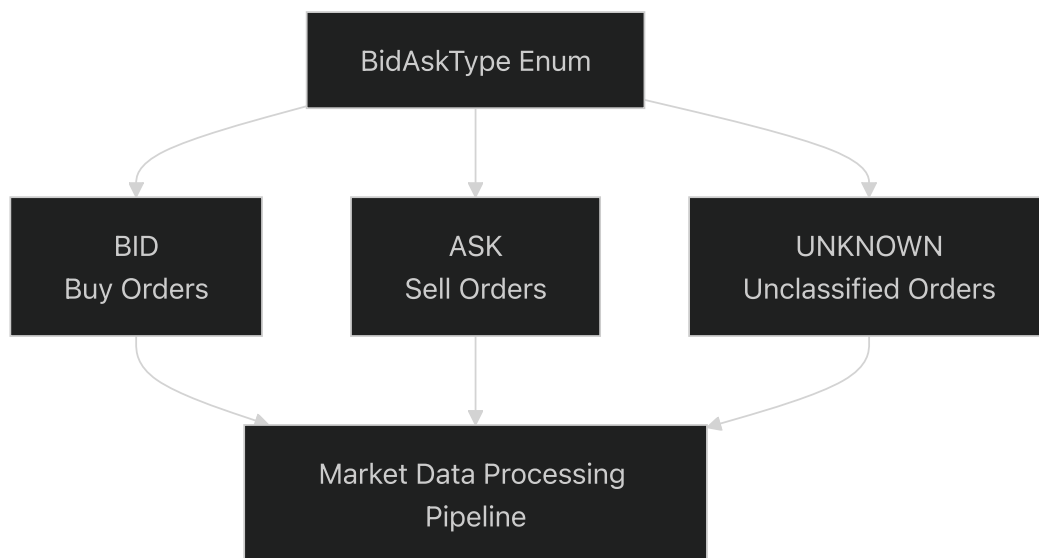
The system enumerations are organized into three primary functional categories:

Category	Purpose	Enumerations
Market Data	Define market data types and bid/ask classifications	<code>BidAskType</code>
Index Management	Handle Vietnamese stock exchange indices and sorting criteria	<code>IndexTypeEnum</code> , <code>IndexRankType</code> , <code>IndexTypeSortEnum</code>
System Configuration	Configure Kafka domains and messaging parameters	<code>KafkaDomainEnum</code>

Market Data Enumerations

BidAskType

The `BidAskType` enumeration defines the fundamental market data classification for order book entries.



Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/BidAskType.java#L1-L6>).
<src/main/java/com/techx/tradex/realtime/constants/BidAskType.java#L1-L6> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/BidAskType.java#L1-L6>).

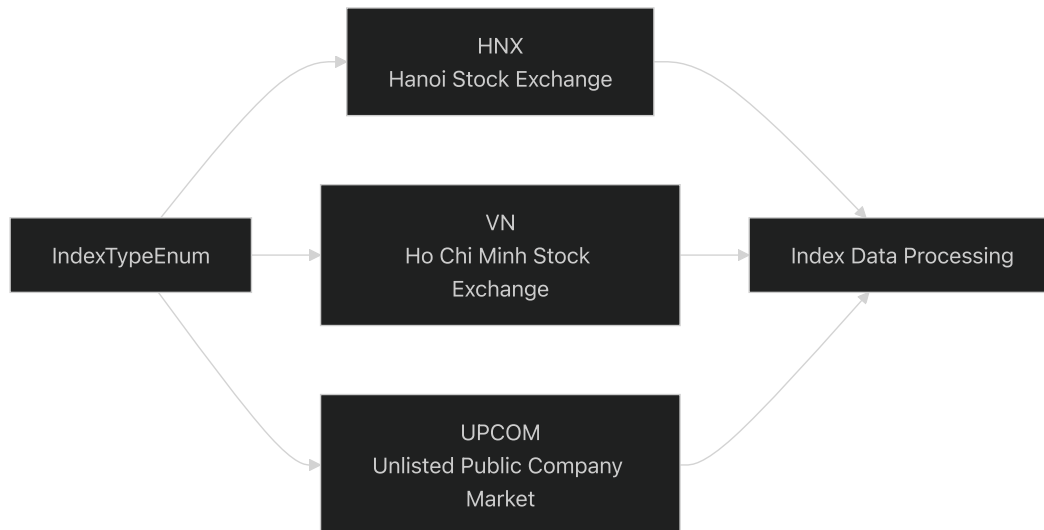
The enumeration contains three values:

- `BID` : Represents buy orders in the order book
- `ASK` : Represents sell orders in the order book
- `UNKNOWN` : Handles unclassified or malformed order data

Index Management Enumerations

Exchange Type Classification

The `IndexTypeEnum` defines the Vietnamese stock exchange markets supported by the system.



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeEnum.java#L1-L6>
[src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeEnum.java1-6](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeEnum.java#L1-L6) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeEnum.java#L1-L6>).

Ranking and Sorting Mechanisms

Two similar enumerations handle index ranking and sorting criteria:

`IndexRankType`

Defines ranking criteria for index performance analysis:

- `TRADING_VOLUME` : Rank by trading volume
- `RATE_DESC` : Rank by rate in descending order
- `RATE_ASC` : Rank by rate in ascending order

`IndexTypeSortEnum`

Provides identical sorting options with the same values:

- `TRADING_VOLUME` : Sort by trading volume
- `RATE_DESC` : Sort by rate descending
- `RATE_ASC` : Sort by rate ascending

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java#L1-L6>
[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java#L1-L6](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java#L1-L6)

[src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java1-6](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java#L1-L6) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java#L1-L6>)

src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java1-6]

([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java#L1-L6)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java#L1-L6](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java#L1-L6)

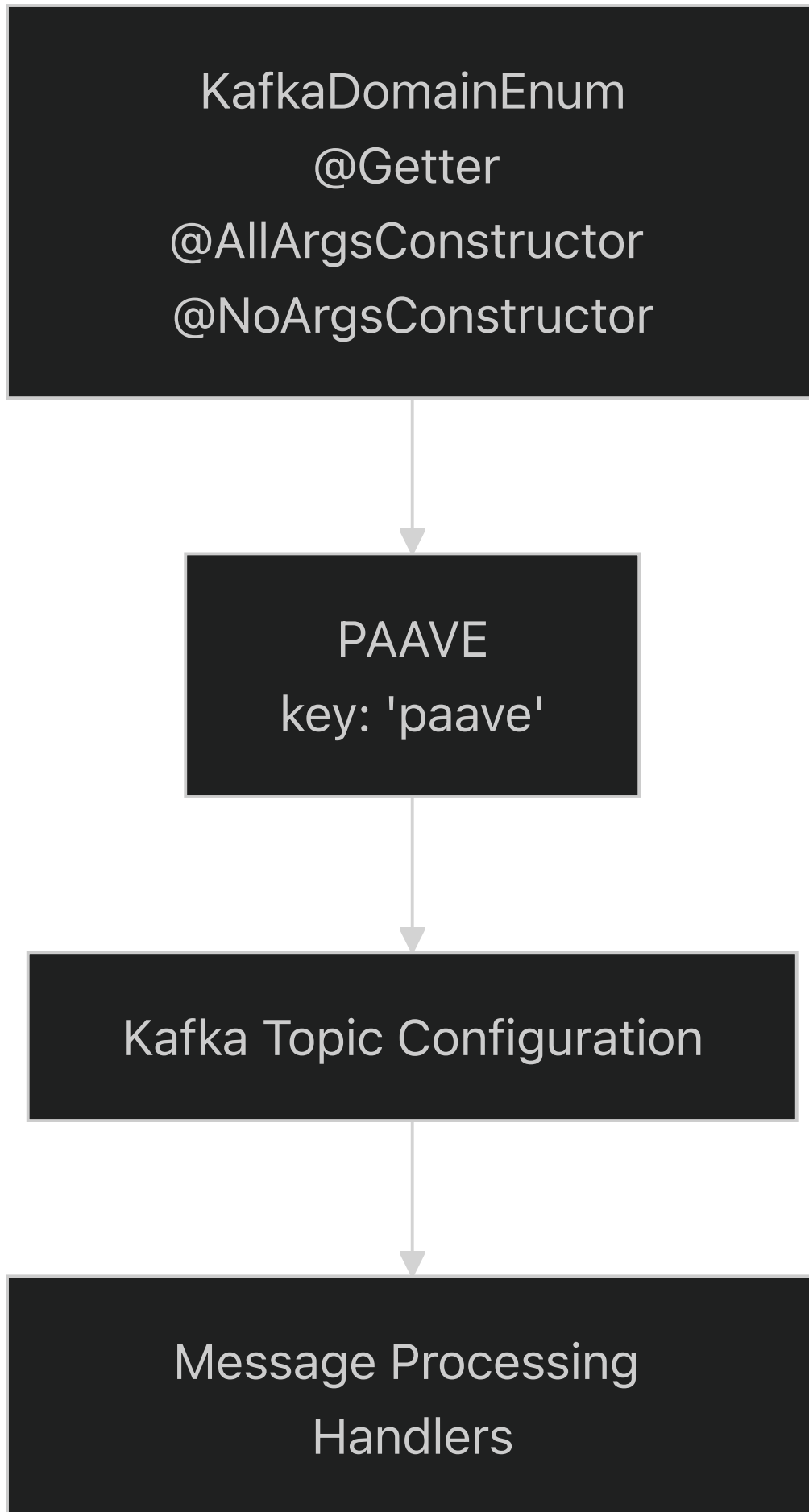
([https://github.com/nhsv-tradex/realtime-](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java#L1-L6)

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java#L1-L6](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java#L1-L6)))

System Configuration Enumerations

KafkaDomainEnum

The `KafkaDomainEnum` configures Kafka messaging domains with Lombok annotations for enhanced functionality.



Sources: <https://github.com/nhsv-tradex/realtime->

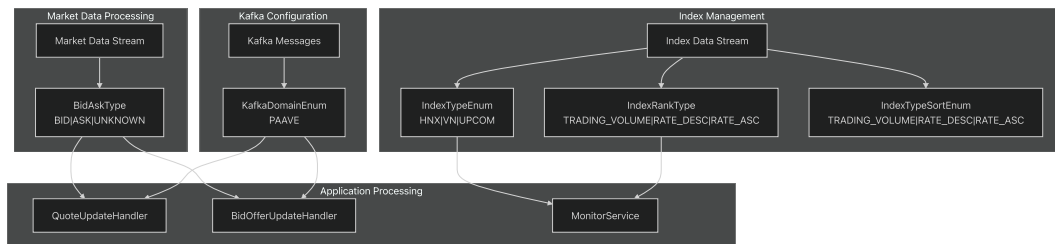
<v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/KafkaDomainEnum.java#L1-L15>

<src/main/java/com/techx/tradex/realtime/constants/enums/KafkaDomainEnum.java#L1-L15> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/KafkaDomainEnum.java#L1-L15>)

The enumeration includes:

- PAAVE ("paave") : Defines the primary Kafka domain with key value "paave"
- Lombok annotations:
 - `@Getter` : Provides getter methods for the key field
 - `@AllArgsConstructor` : Generates constructor with all arguments
 - `@NoArgsConstructor` : Generates default constructor

Enumeration Integration Architecture



Sources: <https://github.com/nhsv-tradex/realtime->

<v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/BidAskType.java#L1-L6>

<src/main/java/com/techx/tradex/realtime/constants/BidAskType.java#L1-L6> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/BidAskType.java#L1-L6>)

<src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeEnum.java#L1-L6> (<https://github.com/nhsv-tradex/realtime->

<v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeEnum.java#L1-L6>)

(<https://github.com/nhsv-tradex/realtime->

[%5B">v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeEnum.java#L1-L6">%5B](v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeEnum.java#L1-L6)

<src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java#L1-L6> (<https://github.com/nhsv-tradex/realtime->

<v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java#L1-L6>)

(<https://github.com/nhsv-tradex/realtime->

[%5B">v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java#L1-L6">%5B](v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexRankType.java#L1-L6)

<src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java#L1-L6>

(<https://github.com/nhsv-tradex/realtime->

<v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java#L1-L6>)

(<https://github.com/nhsv-tradex/realtime->

[%5B">v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java#L1-L6">%5B](v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/IndexTypeSortEnum.java#L1-L6)

<src/main/java/com/techx/tradex/realtime/constants/enums/KafkaDomainEnum.java#L1-L15>

(<https://github.com/nhsv-tradex/realtime->

<v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/KafkaDomainEnum.java#L1-L15>

(<https://github.com/nhsv-tradex/realtime->

[%5B">v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/KafkaDomainEnum.java#L1-L15">%5B](v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/constants/enums/KafkaDomainEnum.java#L1-L15)

Usage Patterns

The enumerations follow consistent patterns throughout the codebase:

Pattern	Description	Examples
Simple Value Enums	Basic enumeration without additional properties	<code>BidAskType</code> , <code>IndexTypeEnum</code> , <code>IndexRankType</code> , <code>IndexTypeSortEnum</code>
Keyed Enums	Enumerations with associated string values	<code>KafkaDomainEnum</code> with key field
Lombok Enhanced	Enumerations using Lombok annotations for boilerplate reduction	<code>KafkaDomainEnum</code> with <code>@Getter</code> , <code>@AllArgsConstructor</code> , <code>@NoArgsConstructor</code>

These enumerations provide type safety and consistency across the real-time market data processing pipeline, ensuring reliable classification and processing of financial data streams from Vietnamese stock exchanges.

Monitoring and Statistics

Relevant source files

- [
[src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java)](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java>)
<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java>)

This document covers the performance monitoring and statistical tracking capabilities of the tradex-realtime system. The monitoring system provides runtime performance metrics for data processing operations throughout the application, enabling observability into system throughput and processing times.

For information about system configuration and feature toggles related to monitoring, see [Application Configuration](#).

Overview

The monitoring system is built around a lightweight wrapper pattern that instruments data processing operations with timing metrics and error tracking. It provides both cumulative and incremental statistics that are periodically logged for operational monitoring.

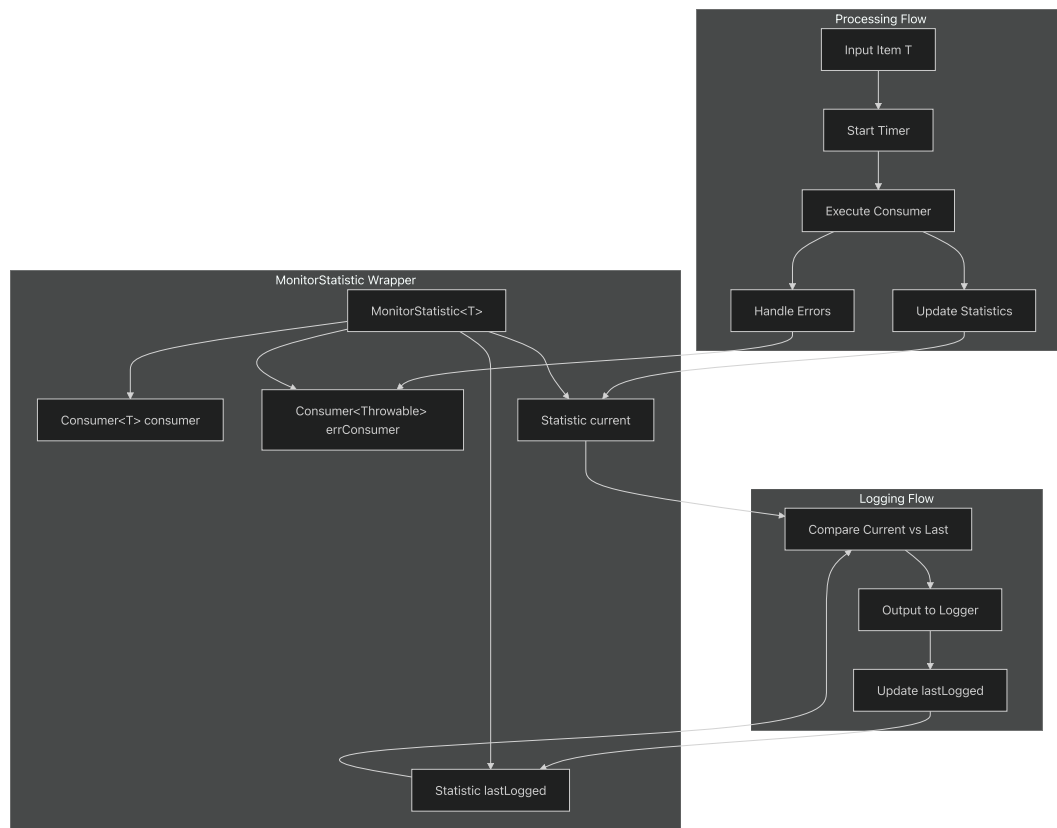
MonitorStatistic Architecture

The core monitoring functionality is implemented through the `MonitorStatistic<T>` class, which acts as a generic wrapper around processing operations.

Core Components

Component	Purpose	Key Methods
<code>MonitorStatistic<T></code>	Main monitoring wrapper	<code>process()</code> , <code>log()</code>
<code>Statistic</code>	Inner class for metric storage	<code>compare()</code> , <code>copy()</code>

The monitoring system follows this architectural pattern:



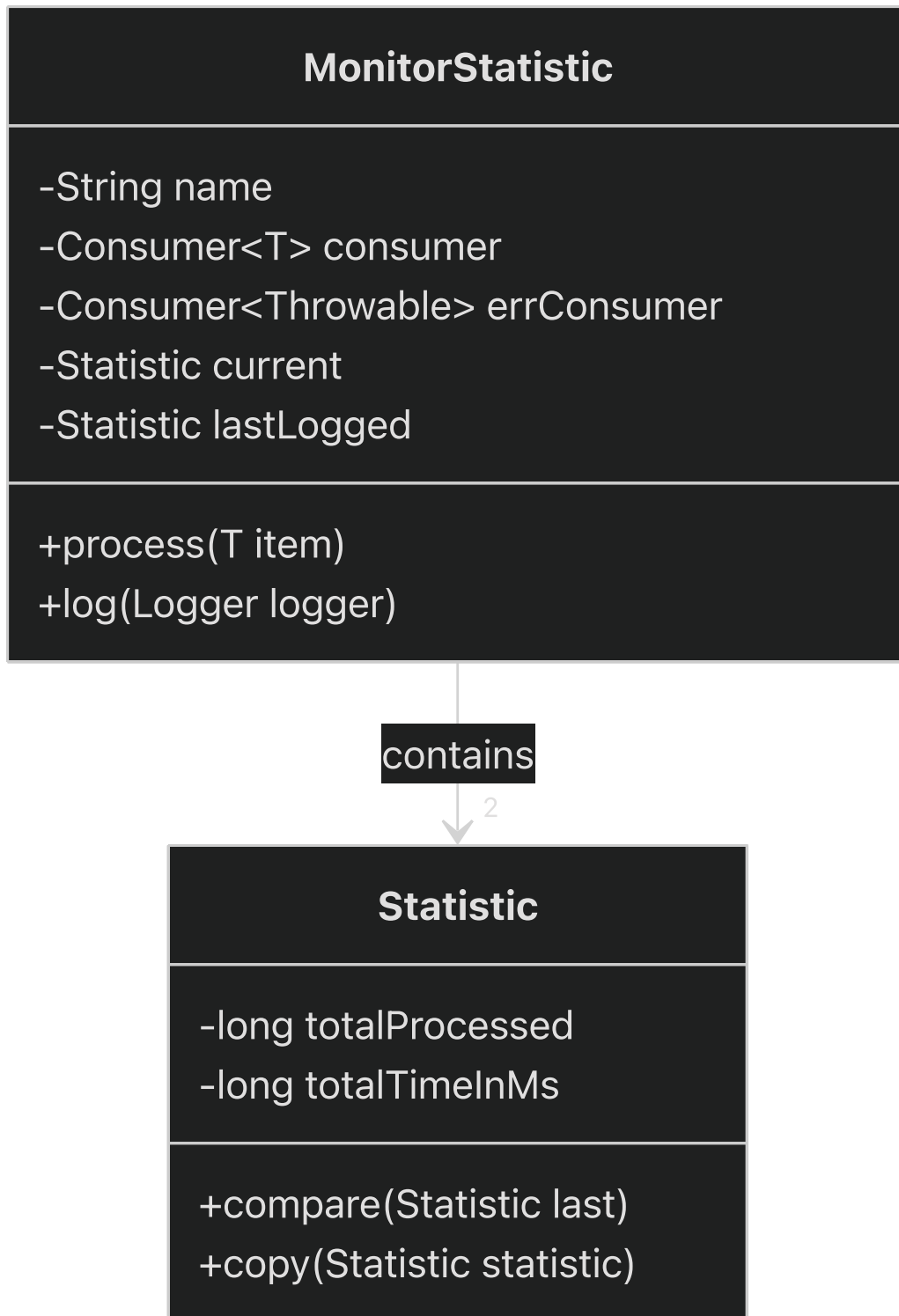
Sources: <https://github.com/nhsv-tradex/realtime->

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L8-L73](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L8-L73)

[src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java8-73](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L8-L73) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L8-L73>)

Statistic Data Model

The `Statistic` inner class maintains two key metrics:



Sources: <https://github.com/nhsv-tradex/realtime->

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L44-L72](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L44-L72)

[src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java44-72](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L44-L72) (<https://github.com/nhsv-tradex/realtime->

[v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L44-L72](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L44-L72))

Processing Instrumentation

The `process()` method wraps the actual business logic with performance measurement:

Execution Flow

1. **Timing Start:** Records current timestamp[

```
src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java22](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L22-L22)
(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L22-L22))
```

2. **Processing:** Executes the wrapped consumer function[

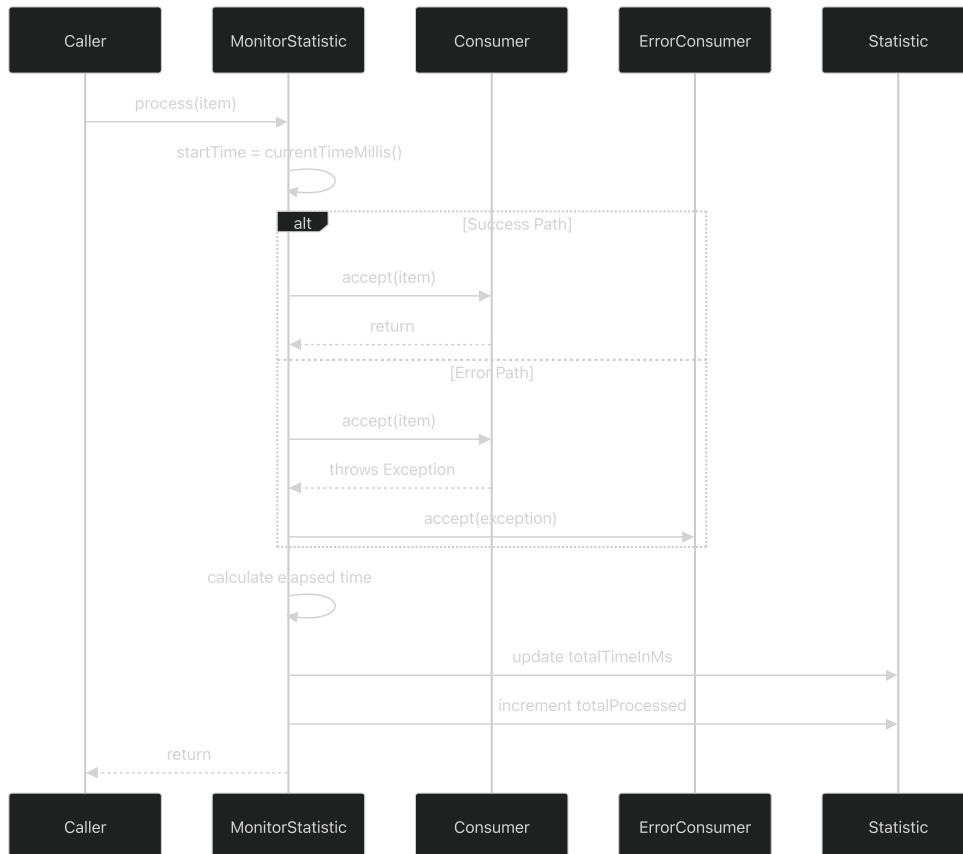
```
src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java24](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L24-L24)
(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L24-L24))
```

3. **Error Handling:** Catches and delegates exceptions to error consumer[

```
src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java25-27](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L25-L27)
(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L25-L27))
```

4. **Metric Update:** Updates total processing time and count[

```
src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java28-31](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L28-L31)
(https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L28-L31))
```



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L21-L31> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L21-L31>)

[src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java21-31](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L21-L31) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L21-L31>)

Statistics Logging

The logging system provides both cumulative and incremental metrics through the `log()` method:

Logging Mechanism

The system tracks two sets of metrics:

- **Cumulative:** Total counts and times since startup
- **Incremental:** Counts and times since last log output

The comparison logic calculates incremental values by subtracting the last logged statistics from current statistics (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L34-L34>)

[src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java34](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L34-L34) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L34-L34>)

Log Output Format

The log statement provides comprehensive performance information:

```
statistic {name} total processed {totalCount} in {totalTime} ms. processed {incrementalCou
```

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L33-L42> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L33-L42>)

[src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java33-42](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L33-L42) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L33-L42>)

Integration with System Components

Based on the overall system architecture, the `MonitorStatistic` class is likely used throughout the real-time data processing pipeline to instrument:

- Kafka message handlers
- Service layer operations
- Data processing workflows
- Cache operations

Usage Pattern

The typical usage pattern involves:

1. Creating a `MonitorStatistic` instance with a descriptive name
2. Wrapping processing logic in the `process()` method
3. Periodically calling `log()` to output performance metrics
4. Configuring error handling through the error consumer

This monitoring approach provides non-intrusive performance tracking across the distributed real-time processing system.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L1-L73> (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L1-L73>)

[src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L1-L73](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L1-L73) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/src/main/java/com/techx/tradex/realtime/model/MonitorStatistic.java#L1-L73>).

Deployment and Operations

Overview

Relevant source files

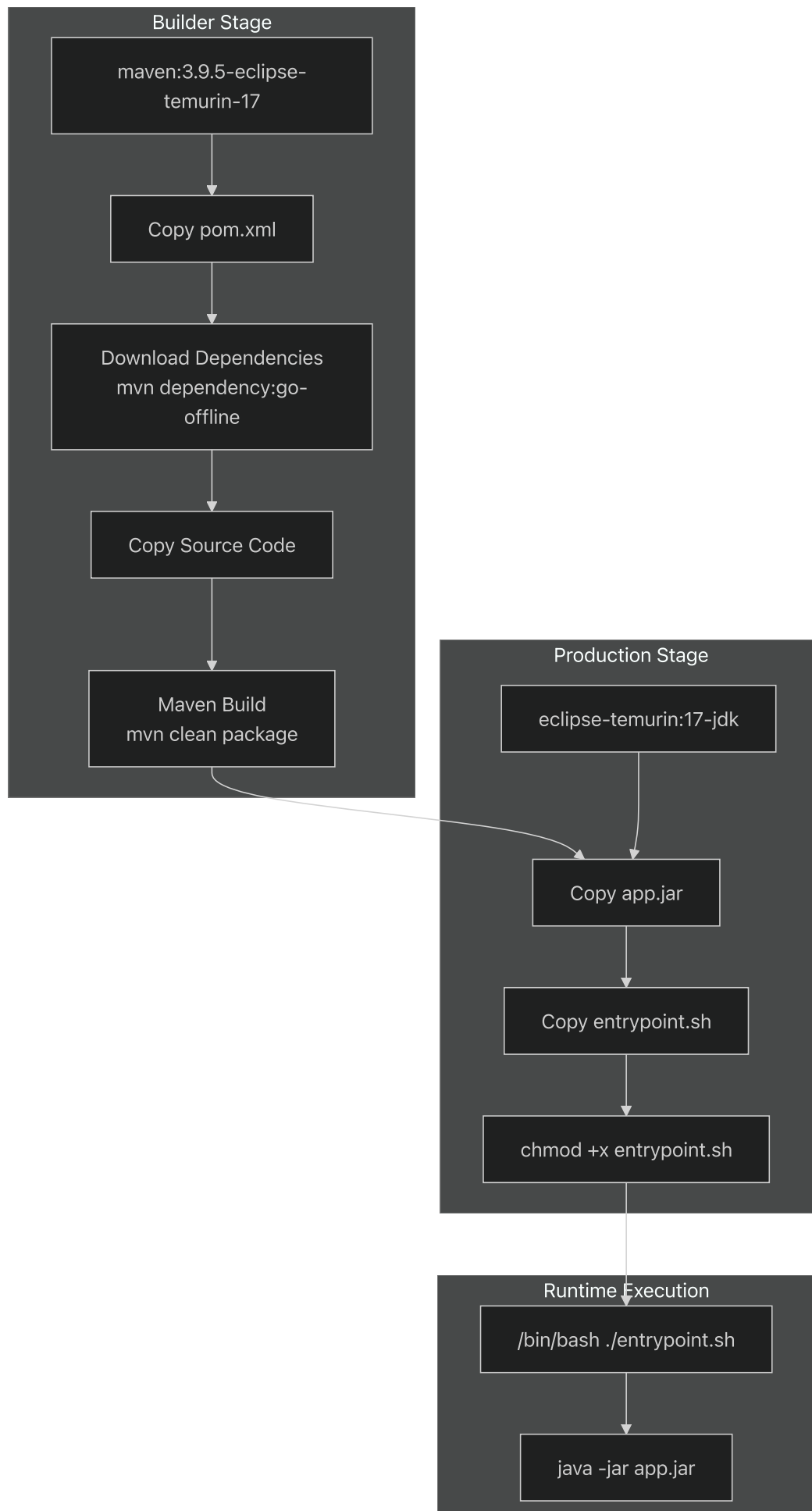
- [[.gitignore](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore)] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore>)
- [[dockerfile](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile)] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile>)
- [[entrypoint.sh](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh)] (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh>)

This document covers the deployment architecture, containerization strategy, and operational aspects of the tradex-realtime application. It explains how the application is packaged using Docker, how service orchestration is handled through the entrypoint script, and how environment configuration is managed across different deployment targets.

For information about application configuration and bean setup, see [Configuration System](#). For details about the development environment and code structure, see [Development Environment](#).

Docker Deployment Architecture

The application uses a multi-stage Docker build process designed for optimal layer caching and production deployment. The build separates dependency resolution from source compilation to improve build efficiency.



Build Process

The Docker build follows a two-stage approach:

Builder Stage: Uses the Maven image to compile the application

- Copies `pom.xml` first for dependency caching optimization
- Downloads dependencies using `mvn dependency:go-offline`
- Copies source code and builds with `mvn clean package -DskipTests`

Production Stage: Creates a minimal runtime environment

- Based on Eclipse Temurin JDK 17
- Installs essential packages (`bash` , `curl`)
- Copies the built JAR and entrypoint script
- Sets executable permissions on the entrypoint script

Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L1-L43>)

[dockerfile1-43](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L1-L43) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L1-L43>).

Container Entrypoint

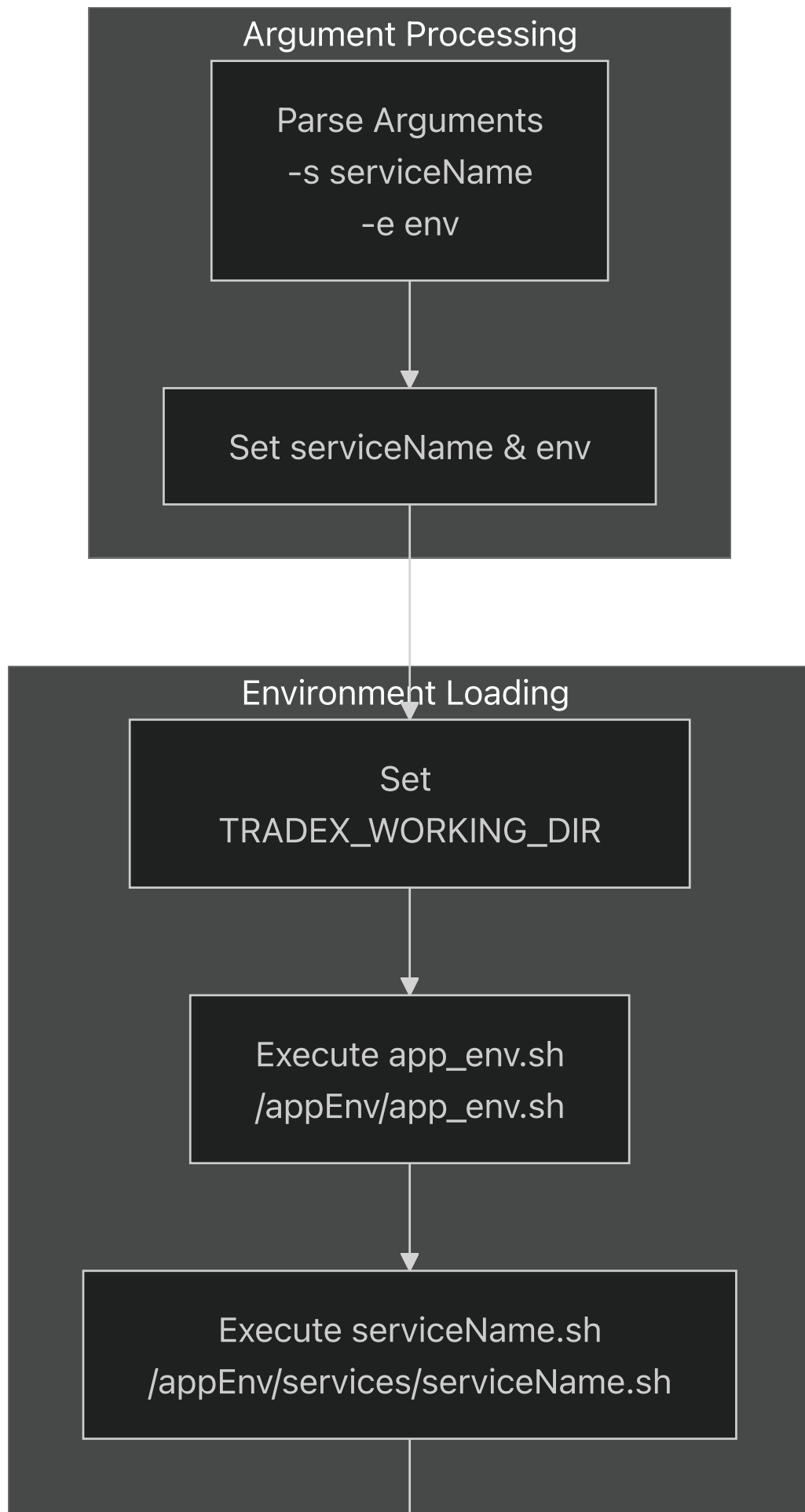
The container uses a bash-based entrypoint that delegates to the `entrypoint.sh` script rather than directly executing the Java application. This allows for complex service orchestration and environment setup before application startup.

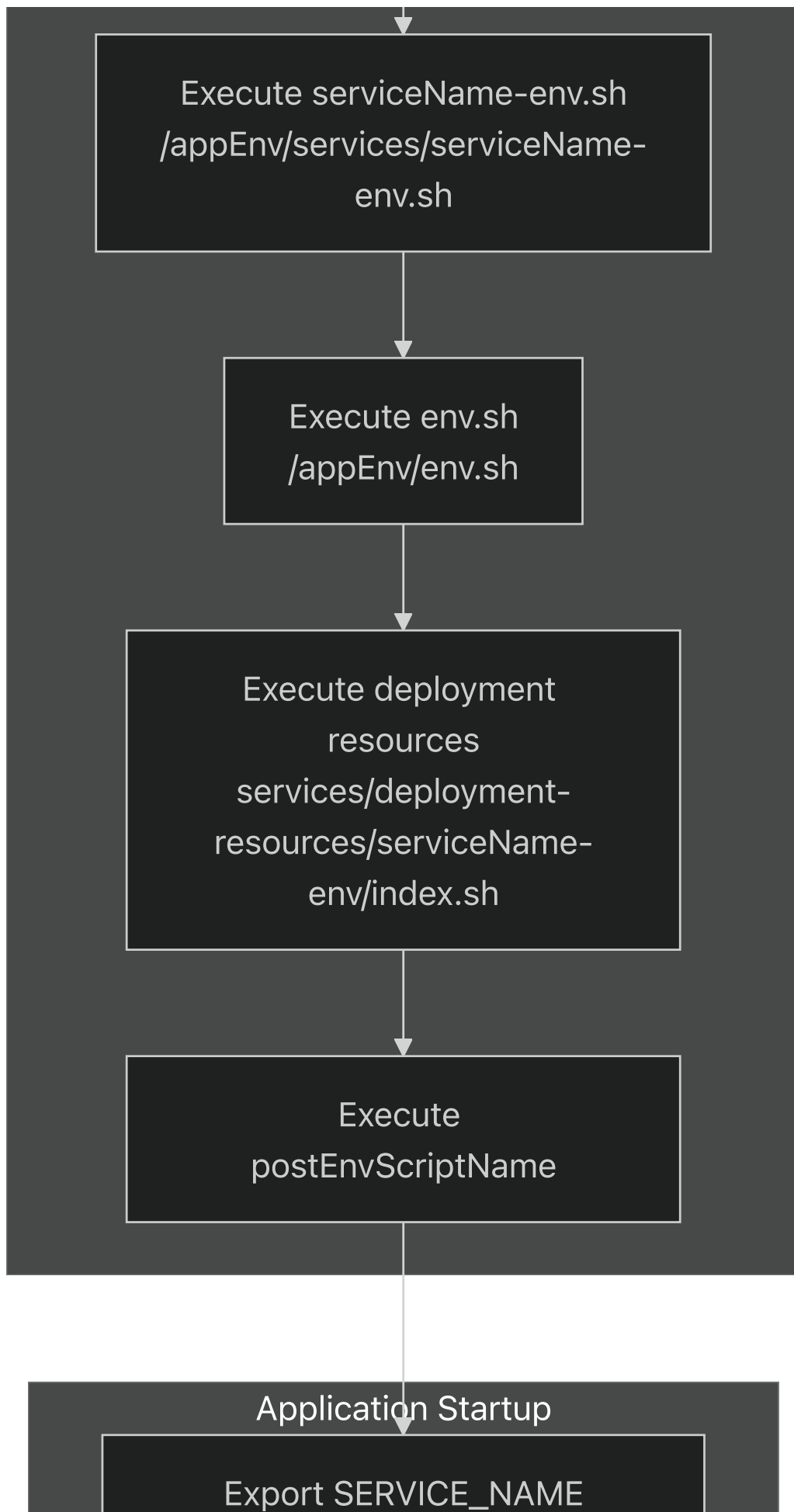
Sources: (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L42-L42>)

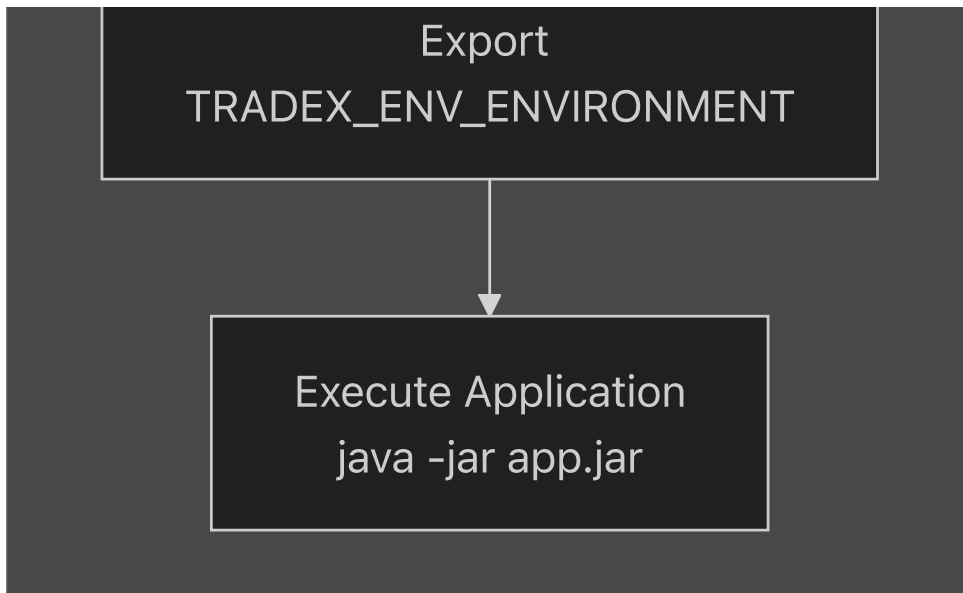
[dockerfile42](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L42-L42) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L42-L42>).

Service Orchestration System

The `entrypoint.sh` script implements a sophisticated service orchestration system that handles environment configuration loading, service-specific setup, and application startup coordination.







Environment Configuration Hierarchy

The entrypoint script loads environment configuration in a specific hierarchical order:

Order	Script Location	Purpose
1	/appEnv/app_env.sh	Global application environment
2	/appEnv/services/{serviceName}.sh	Service-specific configuration
3	/appEnv/services/{serviceName}-{env}.sh	Service and environment specific
4	/appEnv/env.sh	Environment-specific overrides
5	/appEnv/services/deployment-resources/{serviceName}-{env}/index.sh	Deployment resources
6	{postEnvScriptName}	Post-environment setup script

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L37-L82>.

[entrypoint.sh37-82](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L37-L82) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L37-L82>).

Command Line Interface

The entrypoint script accepts the following command line arguments:

- `-s|--serviceName` : Specifies the service name for configuration loading
- `-e|--env` : Specifies the environment (e.g., dev, staging, prod)
- `--` : Ends argument parsing, remaining arguments passed to application

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L7-L30>

[entrypoint.sh7-30](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L7-L30) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L7-L30>).

Environment Variable Exports

The script exports key environment variables for application consumption:

- `TRADEX_WORKING_DIR` : Set to current working directory
- `SERVICE_NAME` : Set to the serviceName parameter
- `TRADEX_ENV_ENVIRONMENT` : Set to the env parameter (if not already set)

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L35-L87>.

[entrypoint.sh35-87](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L35-L87) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L35-L87>).

Configuration Management Strategy

The deployment system supports multi-environment configuration through a layered approach that allows for global defaults, service-specific settings, and environment-specific overrides.



This layered approach enables:

- **Environment Promotion:** Same service configuration across dev/staging/prod with environment-specific overrides
- **Service Isolation:** Each service can have independent configuration without affecting others
- **Deployment Flexibility:** Deployment-specific resources can be injected per service and environment
- **Configuration Inheritance:** Later layers override earlier ones, providing a clear precedence model

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L37-L82>.

[entrypoint.sh37-82](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L37-L82) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L37-L82>).

Development Environment Configuration

The repository includes standard development environment configuration to ensure consistent builds across different development setups.

Ignored Artifacts

The `.gitignore` configuration excludes common development and build artifacts:

Category	Patterns	Purpose
IDE Files	<code>.vscode/**</code> , <code>.idea/**</code> , <code>.settings/**</code>	IDE-specific configurations
Build Artifacts	<code>target/**</code> , <code>*.iml</code>	Maven build outputs and IntelliJ modules
Logs	<code>*.log*</code>	Application and build log files
Temporary Files	<code>*.tbl</code> , <code>master*</code>	Temporary and generated files
Configuration	<code>index-mapping-code*.json</code>	Generated configuration mappings

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore#L1-L12>

[.gitignore1-12](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore#L1-L12) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore#L1-L12>).

Build System Integration

The Docker build integrates with the Maven build system, respecting Maven's standard directory structure and build lifecycle. The multi-stage build ensures that development dependencies and build tools are not included in the production image, reducing the final image size and attack surface.

The build process skips tests during Docker builds (`-DskipTests`) to optimize build time, expecting that testing occurs in the CI/CD pipeline before image creation.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L17-L17>

[dockerfile17](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L17-L17) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L17-L17>

Docker Deployment

Relevant source files

- [

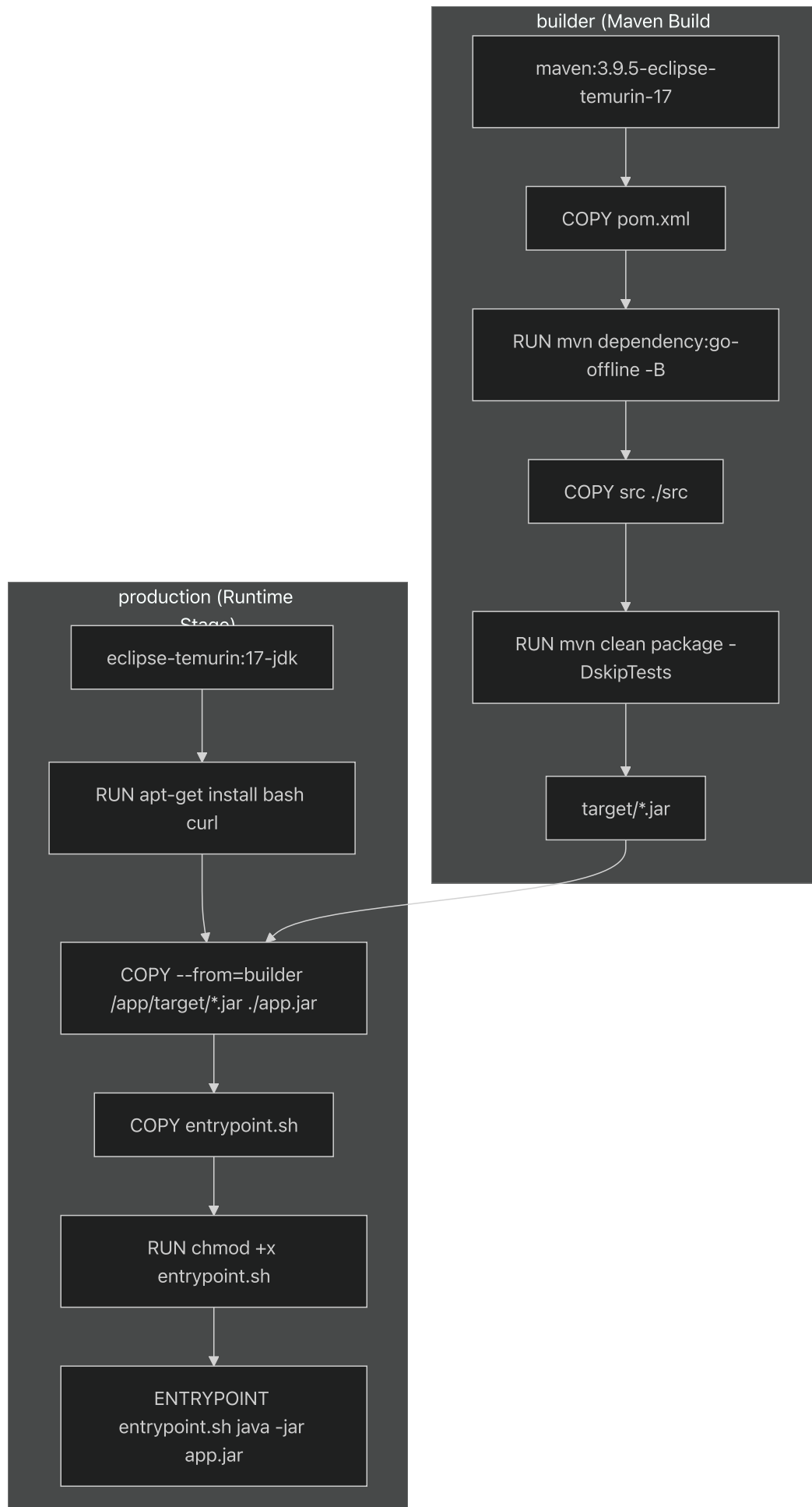
dockerfile](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile> <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile>))

This document covers the Docker containerization strategy for the tradex-realtime application, including the multi-stage build process, production image configuration, and container execution model. For information about the service orchestration and startup process handled by the entrypoint script, see [Service Orchestration](#).

Container Build Architecture

The application uses a multi-stage Docker build process that separates the build environment from the production runtime environment. This approach optimizes image size and security by including only production dependencies in the final image.

Multi-Stage Build Process



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L1-L43>.

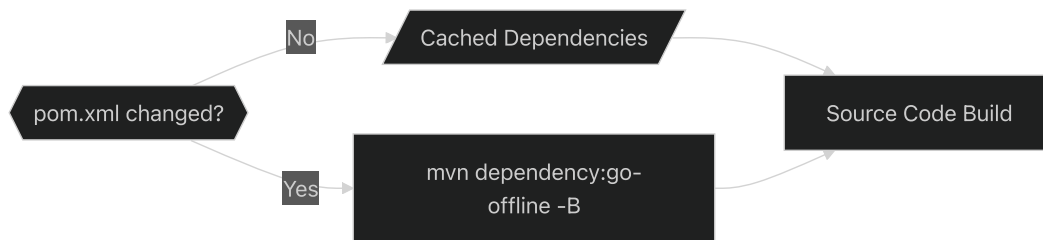
[dockerfile1-43](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L1-L43) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L1-L43>).

Build Stage Configuration

The build stage uses Maven 3.9.5 with Eclipse Temurin 17 as the base image to compile the Spring Boot application and package it as an executable JAR.

Dependency Management Strategy

The build process implements Docker layer caching optimization by copying `pom.xml` separately before copying source code. This ensures that dependency downloads are cached and only re-executed when dependencies change:



Build Step	Command	Purpose
Dependency Resolution	<code>mvn dependency:go-offline -B</code>	Downloads all dependencies for offline build
Package Build	<code>mvn clean package -DskipTests</code>	Compiles and packages without running tests
Artifact Location	<code>target/*.jar</code>	Executable JAR file location

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L7-L17>.

[dockerfile7-17](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L7-L17) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L7-L17>).

Production Stage Configuration

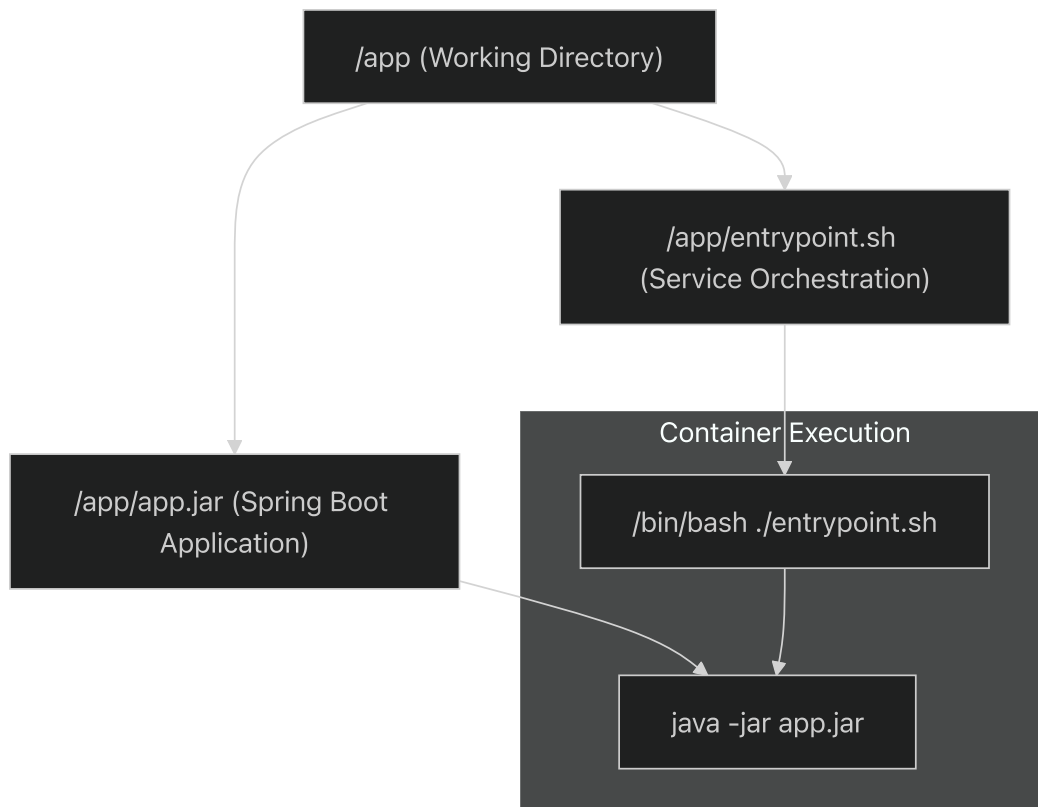
The production stage creates a lightweight runtime environment based on Eclipse Temurin 17 JDK, containing only the necessary components for running the application.

Runtime Dependencies

The production image installs minimal system packages required for application operation and debugging:

Package	Purpose
<code>bash</code>	Shell interpreter for entrypoint script execution
<code>curl</code>	HTTP client for health checks and debugging

Application Deployment Structure



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L19-L42>

[dockerfile19-42](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L19-L42) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L19-L42>)

Container Execution Model

The container uses a bash-based entrypoint script to orchestrate service startup rather than directly executing the Java application. This approach enables:

- Environment variable processing
- Service dependency verification
- Configuration validation
- Graceful error handling

Entrypoint Configuration

The container entrypoint is configured to execute the `entrypoint.sh` script with the Java application command as an argument:

```
ENTRYPOINT ["/bin/bash", "./entrypoint.sh", "java -jar app.jar"]
```

This design allows the entrypoint script to perform initialization tasks before starting the Spring Boot application, ensuring proper service orchestration in containerized environments.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L34-L42>

[dockerfile34-42](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L34-L42) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/dockerfile#L34-L42>)

Service Orchestration

Relevant source files

- [
entrypoint.sh](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh>
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh>))

This document covers the service orchestration mechanism implemented through the `entrypoint.sh` script, which manages the startup sequence, environment configuration, and execution of the tradex-realtime application. The orchestration system provides a structured approach to loading configuration scripts, setting environment variables, and launching services with environment-specific parameters.

For information about the Docker deployment configuration, see [Docker Deployment](#). For development environment setup details, see [Development Environment](#).

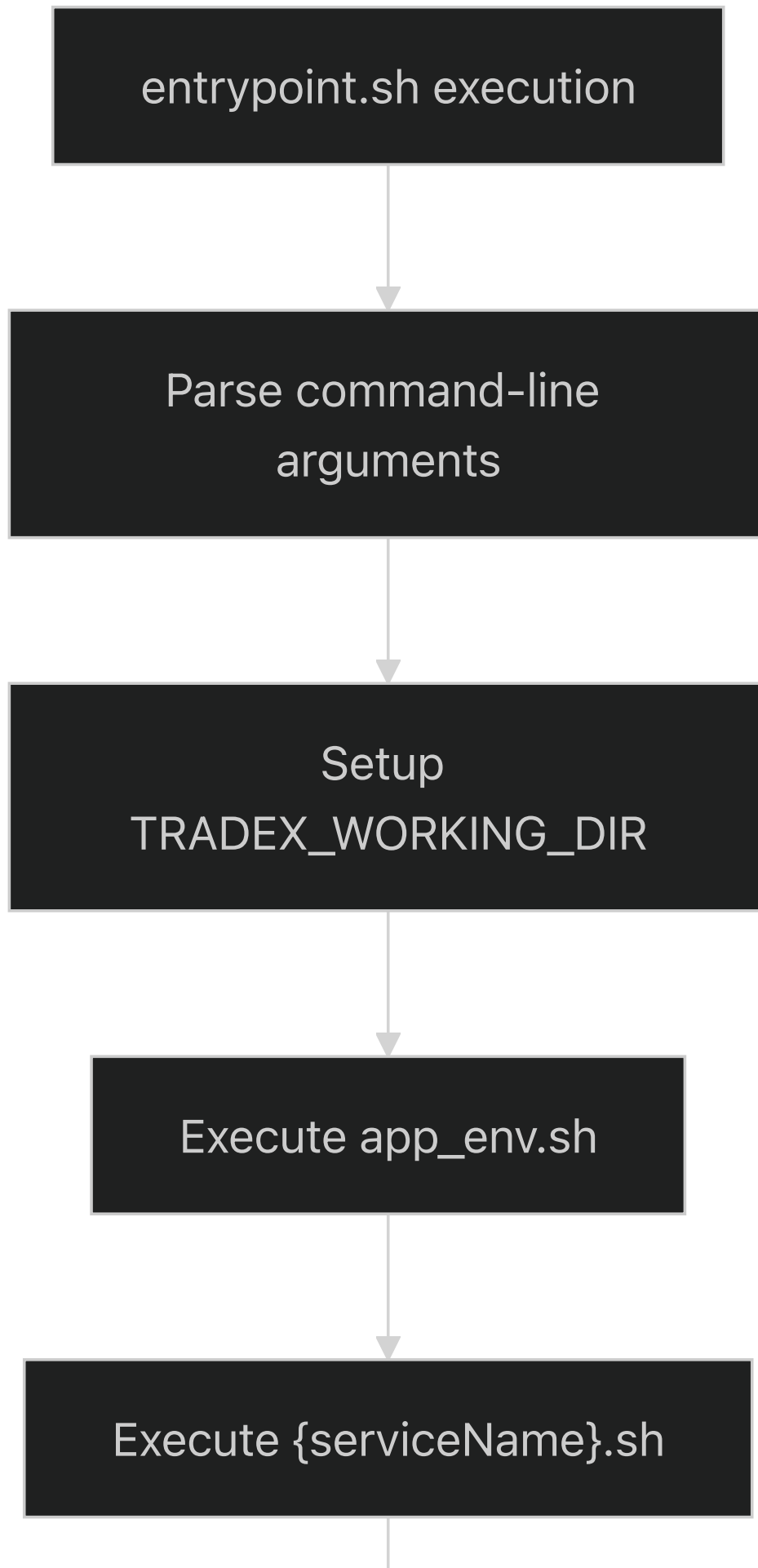
Purpose and Scope

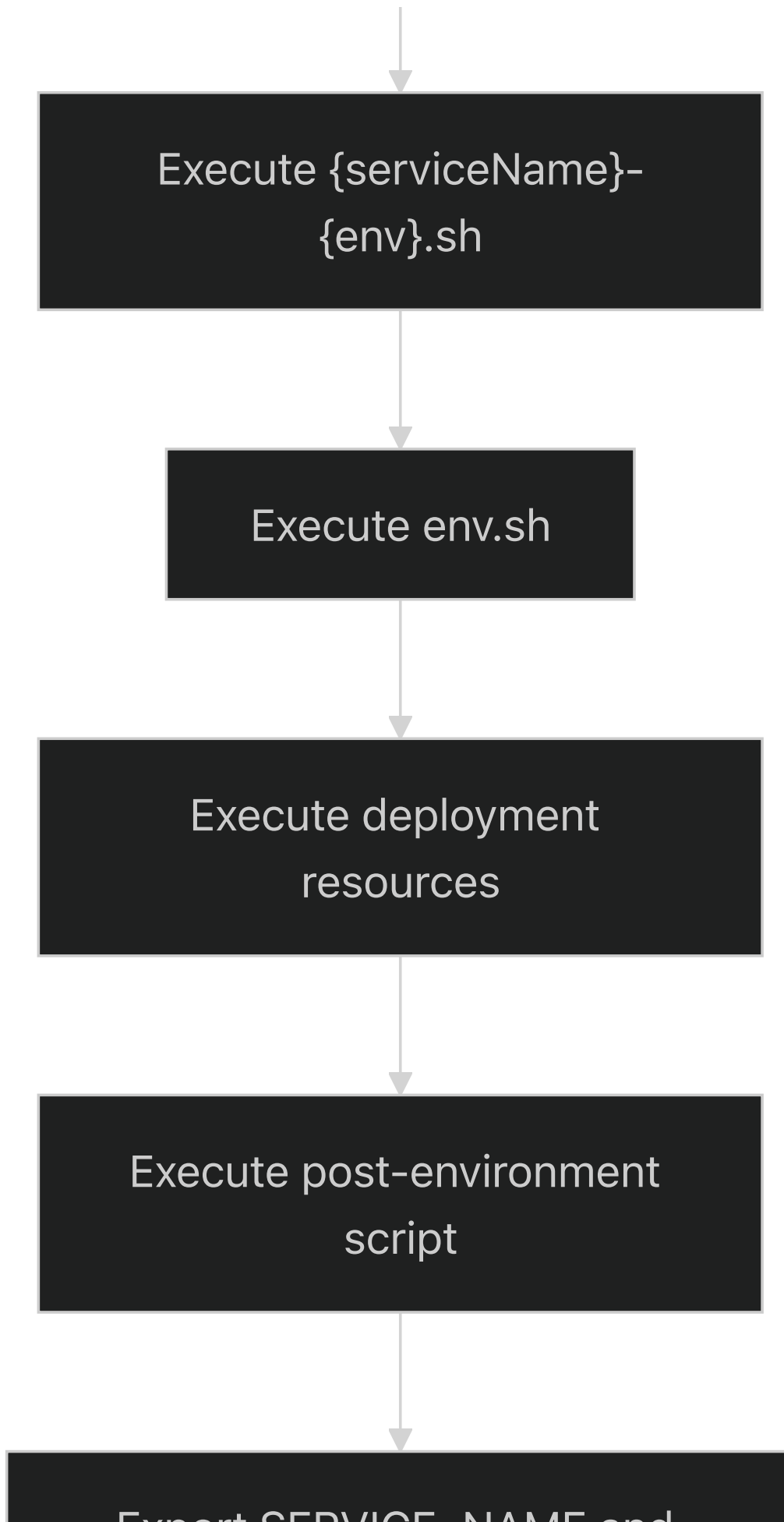
The service orchestration system handles:

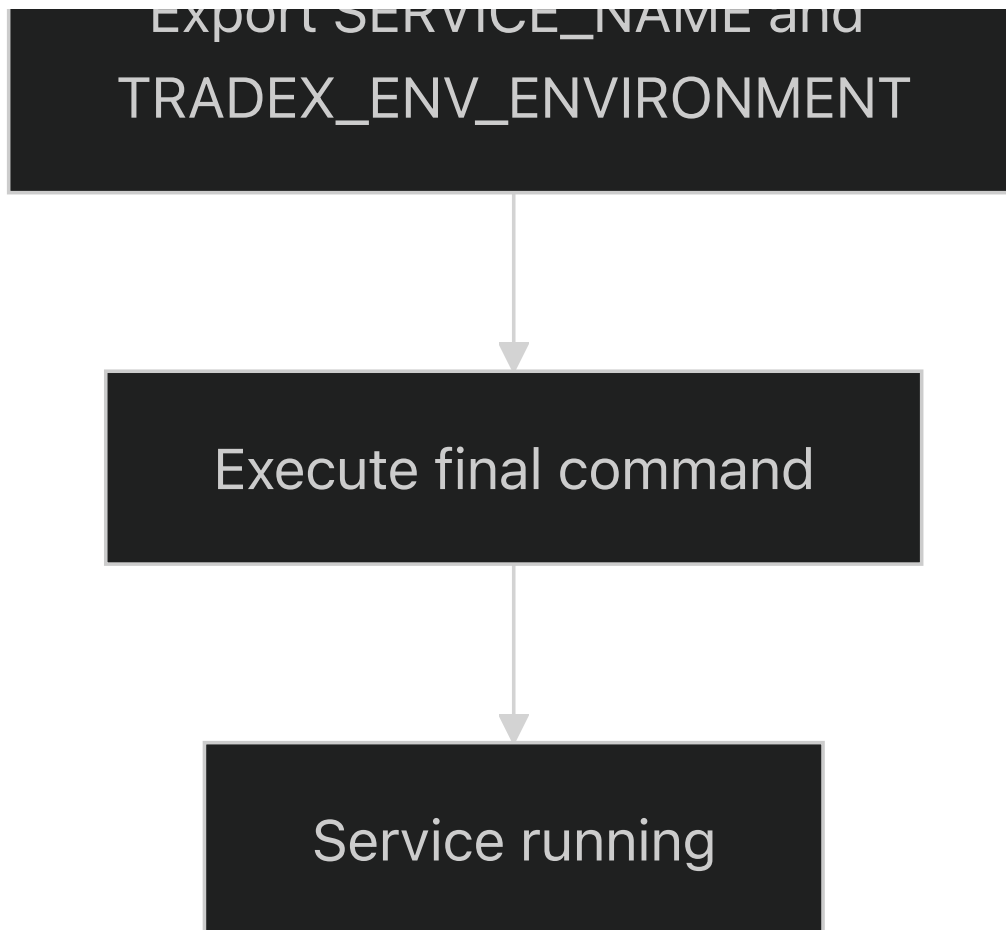
- Command-line argument parsing for service name and environment
- Sequential execution of configuration scripts in a defined hierarchy
- Environment variable management and export
- Working directory setup and navigation
- Service-specific and environment-specific configuration loading

Service Orchestration Flow

The orchestration process follows a structured sequence of operations, from initial argument parsing through final command execution.





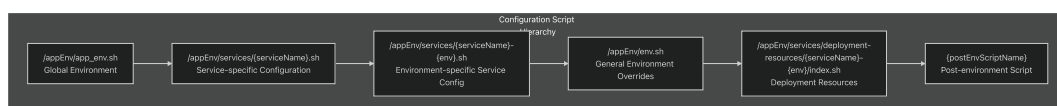


Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L1-L90>

[entrypoint.sh#L1-90](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L1-L90) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L1-L90>

Script Execution Hierarchy

The orchestration system executes configuration scripts in a specific hierarchy, allowing for layered configuration from general to specific.



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L37-L82>

[entrypoint.sh#37-82](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L37-L82) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L37-L82>

Argument Processing and Variable Setup

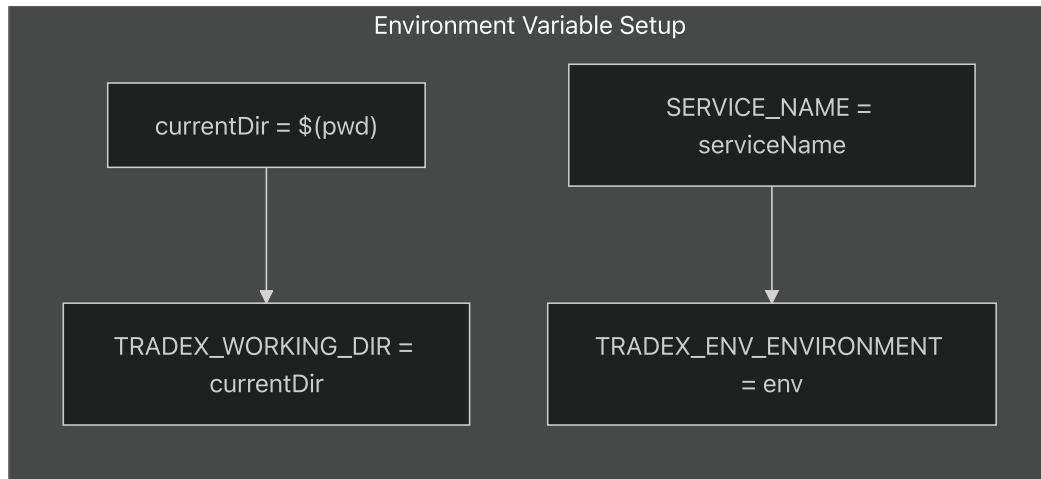
The entrypoint script processes command-line arguments and establishes the runtime environment through a systematic approach.

Command-line Arguments

The script accepts the following arguments:

Argument	Short Form	Purpose	Usage
<code>--serviceName</code>	<code>-s</code>	Specifies the service name	Used to locate service-specific scripts
<code>--env</code>	<code>-e</code>	Specifies the environment	Used for environment-specific configuration
Positional arguments	N/A	Command to execute	Stored in <code>PARAMS</code> variable

Environment Variable Management



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L7-L30>

[entrypoint.sh7-30](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L7-L30) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L7-L30>

[entrypoint.sh34-35](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L34-L35) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L34-L35> [%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L34-L35)

[entrypoint.sh84-87](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L84-L87) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L84-L87> <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L84-L87>

Configuration Script Types

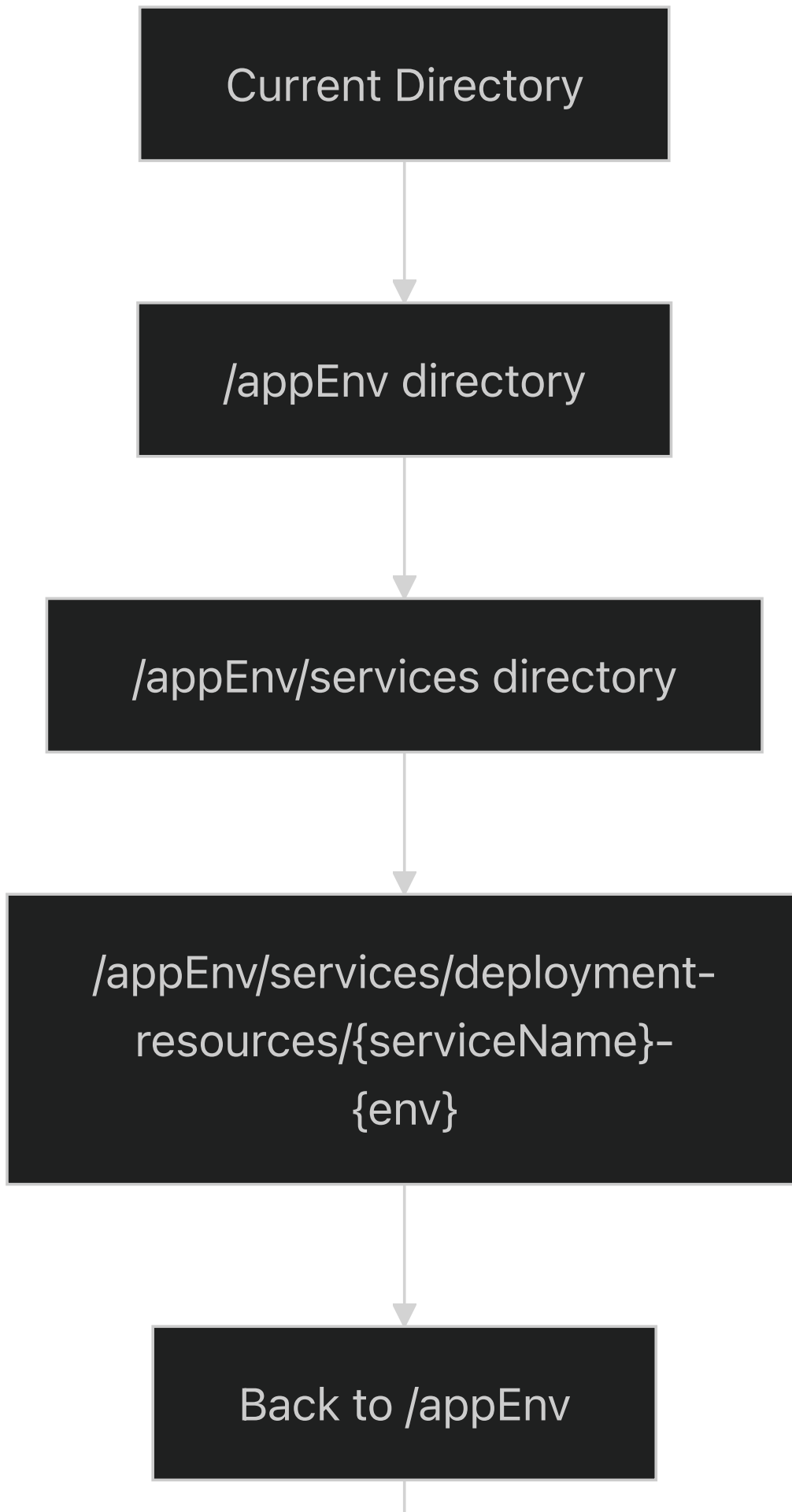
The orchestration system supports multiple types of configuration scripts, each serving a specific purpose in the setup process.

Script Type Hierarchy

Script Type	File Pattern	Purpose	Execution Order
Global Environment	<code>app_env.sh</code>	System-wide environment setup	1
Service Script	<code>{serviceName}.sh</code>	Service-specific configuration	2
Environment Service	<code>{serviceName}-{env}.sh</code>	Environment-specific service config	3
General Environment	<code>env.sh</code>	General environment overrides	4
Deployment Resources	<code>deployment-resources/{serviceName}-{env}/index.sh</code>	Deployment-specific resources	5
Post-Environment	<code>{postEnvScriptName}</code>	Final configuration adjustments	6

Directory Navigation Pattern

The script follows a consistent pattern of directory navigation to execute scripts:





Back to Current Directory

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L37-L76>

[entrypoint.sh#37-76](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L37-L76) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L37-L76>).

Error Handling and Safety

The orchestration script implements several safety mechanisms:

- **Set -e:** Enables immediate exit on command failure[

entrypoint.sh32](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L32-L32>
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L32-L32>))

- **Conditional execution:** Scripts are only executed if they exist[

entrypoint.sh44](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L44-L44>)[
([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L44-L44)).

entrypoint.sh50](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L50-L50>)[
([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L50-L50)).

entrypoint.sh58](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L58-L58>)[
([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L58-L58)).

entrypoint.sh67](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L67-L67>)[
([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L67-L67)).

entrypoint.sh78](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L78-L78>
([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L78-L78)))

- **Permission setting:** Scripts are made executable before execution[

entrypoint.sh38](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L38-L38>)[
([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L38-L38)).

entrypoint.sh45](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L45-L45>)[
([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L45-L45)).

entrypoint.sh51](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L51-L51>)[
([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L51-L51)).

entrypoint.sh59](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L59-L59>)[
([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L59-L59)).

entrypoint.sh69](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L69-L69>)[
([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L69-L69)).

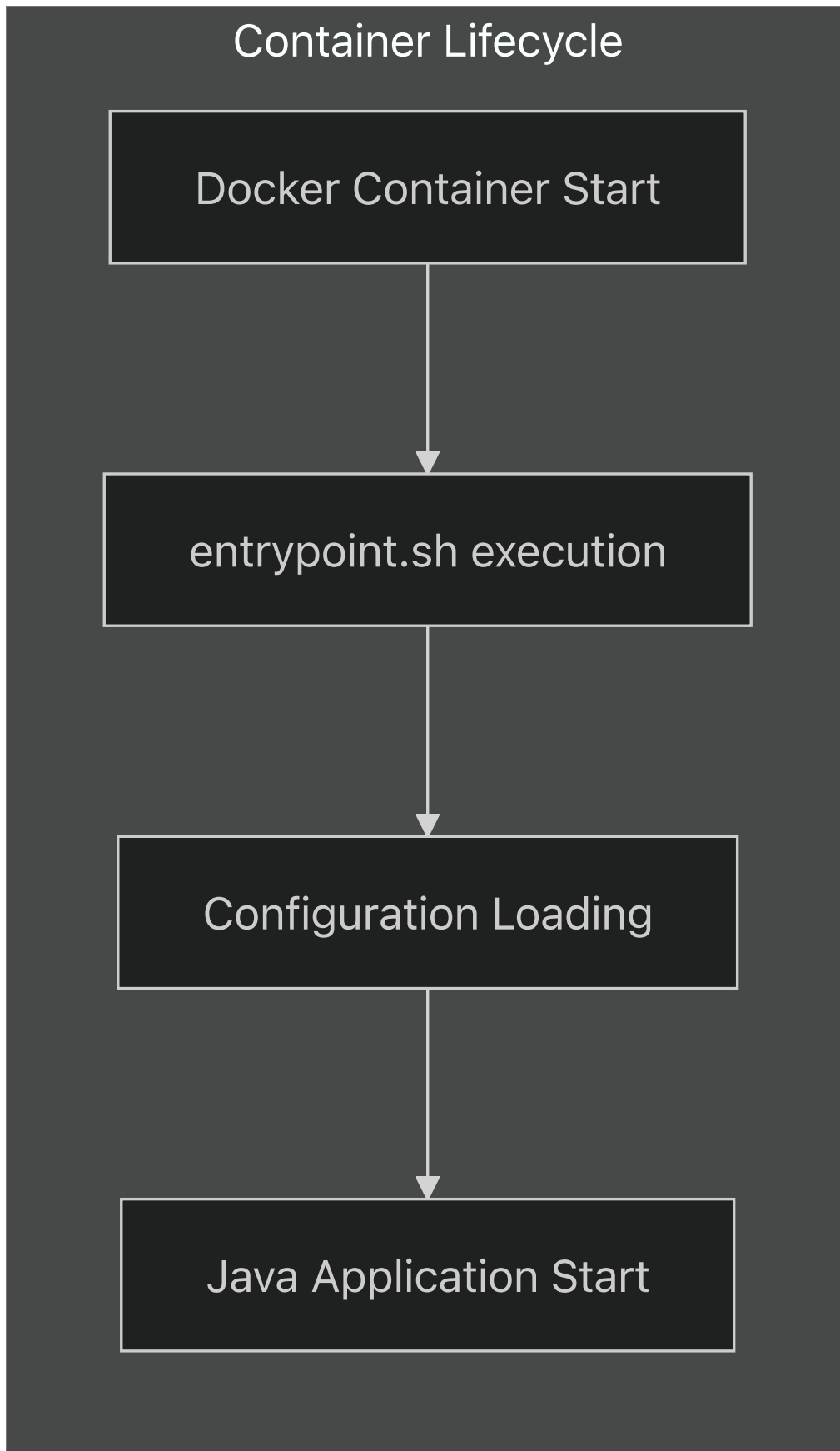
entrypoint.sh79](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L79-L79>
([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L79-L79)))

- **Argument validation:** Unsupported flags cause script termination[

entrypoint.sh21-24](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L21-L24>
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L21-L24>))

Integration with Application Lifecycle

The orchestration script serves as the bridge between container startup and application execution, providing a standardized way to configure and launch services with environment-specific parameters.



The final command execution occurs after all configuration scripts have been processed and environment variables have been exported, ensuring the application starts with a fully configured environment.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L89-L90>

[entrypoint.sh#89-90](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L89-L90) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/entrypoint.sh#L89-L90>).

Development Environment

Relevant source files

- [
 [.gitignore](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore)](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore>))
- [
 [pom.xml](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml)](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml>) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml>))

This page documents the development environment setup for the tradex-realtime application, including prerequisites, build configuration, IDE setup, and development tools required for local development and debugging.

For information about deployment and containerization, see [Docker Deployment](#). For operational aspects like service orchestration, see [Service Orchestration](#).

Prerequisites and System Requirements

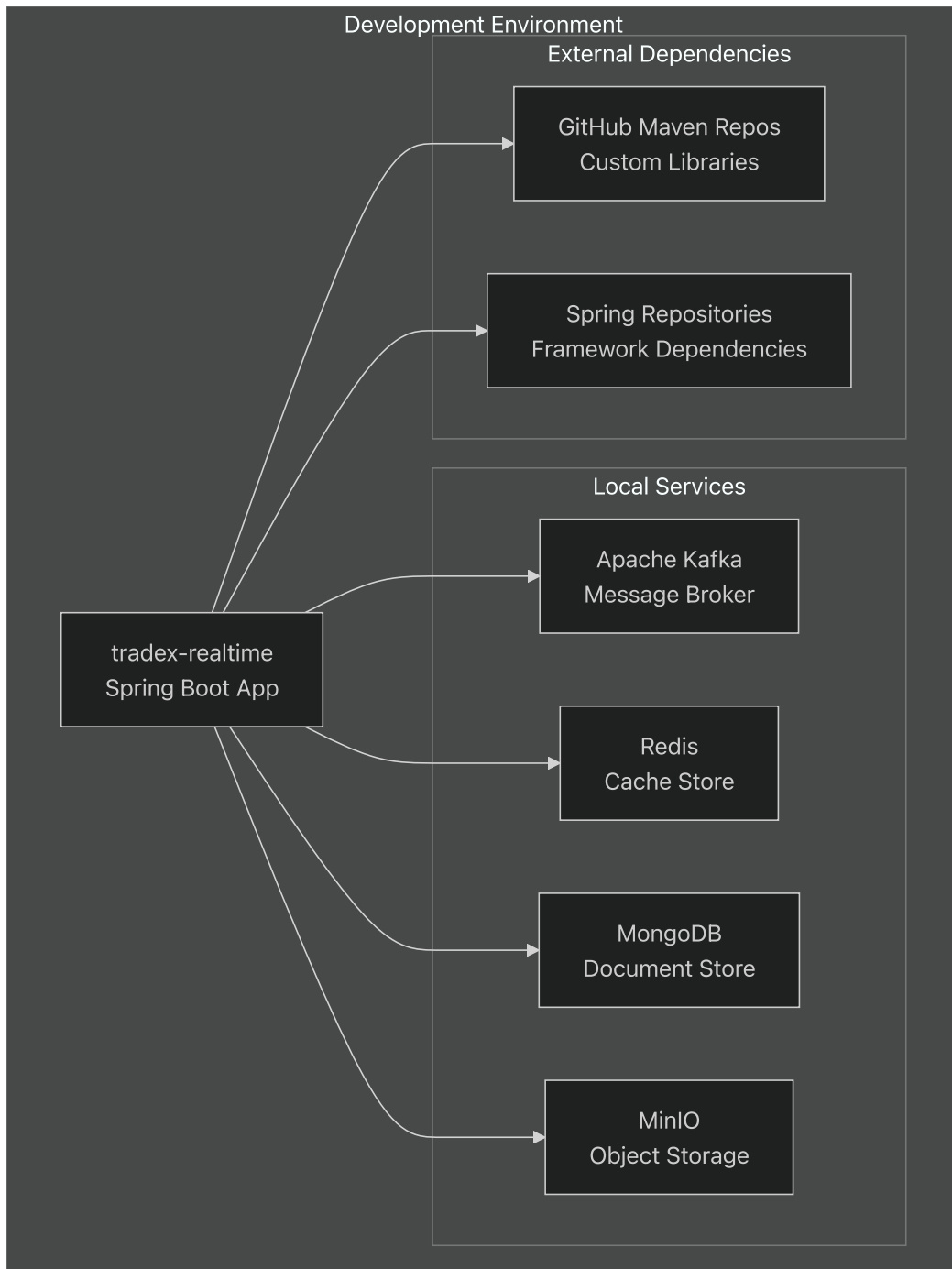
The tradex-realtime application requires specific versions of Java, build tools, and supporting services for development.

Required Software

Component	Version	Purpose
Java	17	Runtime and compilation target
Maven	3.6+	Build tool and dependency management
Git	2.20+	Version control
Docker	20.10+	Local service dependencies
IDE	IntelliJ IDEA/Eclipse/VS Code	Development environment

Runtime Dependencies

The application connects to several external services that must be available during development:



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L163-L176>.

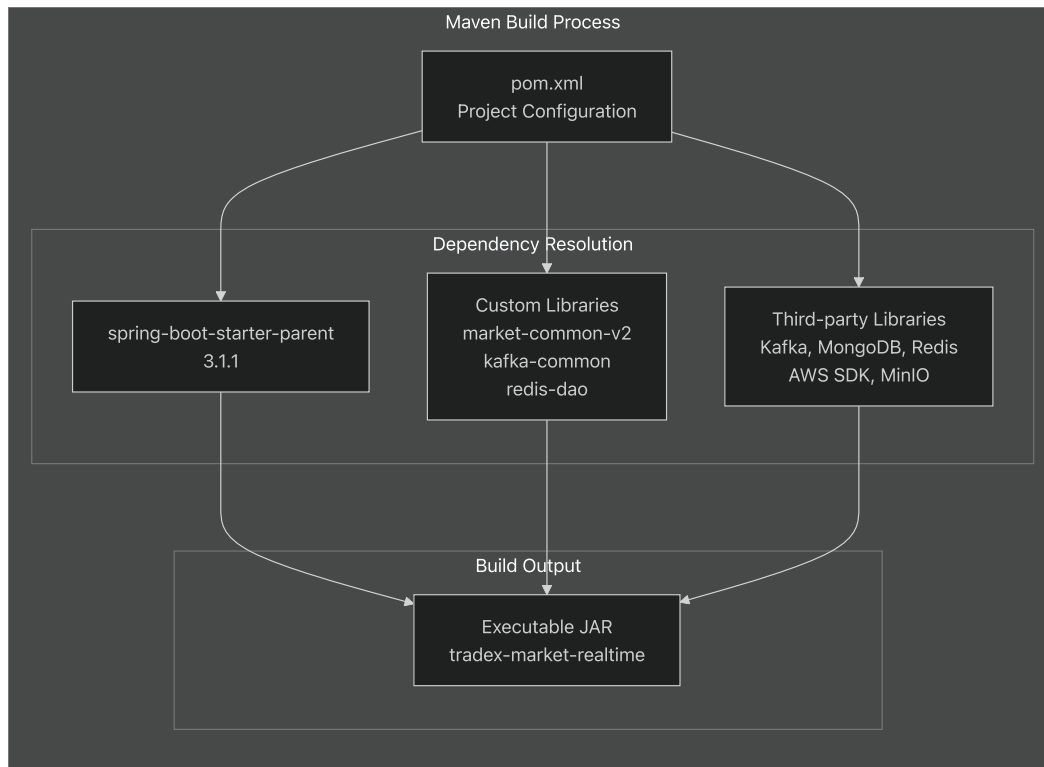
[pom.xml#L163-L176](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L163-L176) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L163-L176>).

Build System Configuration

The application uses Maven as its primary build tool with a comprehensive dependency management setup.

Maven Configuration

The project is configured as a Spring Boot application with Java 17 targeting:



Key Dependencies

The following dependencies are critical for development:

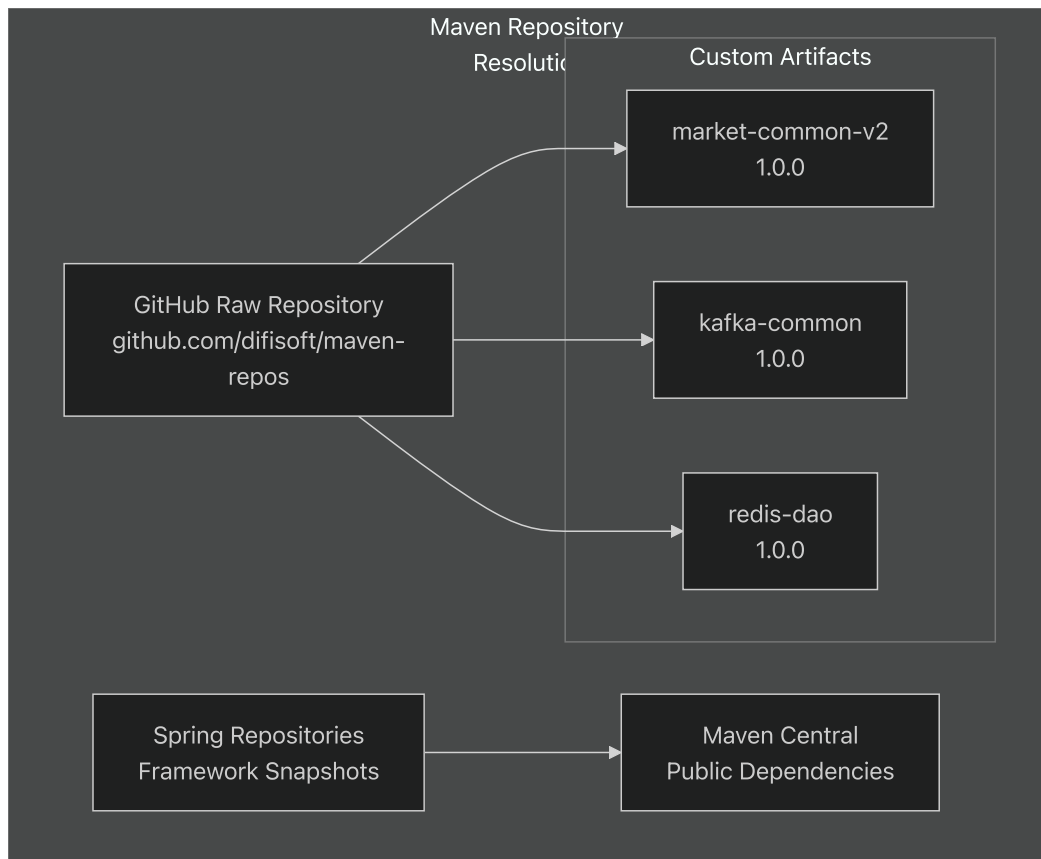
Category	Artifact	Version	Purpose
Framework	spring-boot-starter-parent	3.1.1	Base Spring Boot framework
Data Access	spring-boot-starter-data-mongodb	3.1.1	MongoDB integration
Data Access	spring-boot-starter-data-redis	3.1.1	Redis integration
Messaging	kafka_2.12	2.3.1	Kafka message processing
Custom	market-common-v2	1.0.0	Market data models
Custom	kafka-common	1.0.0	Kafka utilities
Custom	redis-dao	1.0.0	Redis data access
Storage	minio	8.3.7	Object storage client
Utilities	lombok	provided	Code generation

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L27-L149>

[pom.xml#L27-L149](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L27-L149) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L27-L149>)

Repository Configuration

The project depends on custom Maven repositories for proprietary libraries:



Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L163-L186>.

[pom.xml#L163-L186](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L163-L186) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L163-L186>.

IDE and Development Tools

IDE Configuration

The project includes configuration files that should be ignored in version control to maintain developer flexibility:

Excluded IDE Files

The following IDE-specific files are excluded from version control:

Pattern	IDE/Tool	Purpose
<code>.vscode/**</code>	Visual Studio Code	VS Code configuration
<code>*.iml</code>	IntelliJ IDEA	IDEA module files
<code>.idea/**</code>	IntelliJ IDEA	IDEA project configuration
<code>.settings/**</code>	Eclipse	Eclipse project settings
<code>.project</code> , <code>.classpath</code> , <code>.factorypath</code>	Eclipse	Eclipse project files

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore#L1-L12>.

[.gitignore#L1-L12](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore#L1-L12) <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore#L1-L12>.

Development Artifacts

Several file patterns are excluded from version control to keep the repository clean:

Pattern	Purpose
<code>target/**</code>	Maven build output
<code>*.log*</code>	Application log files
<code>*.tbl</code>	Table files
<code>master*</code>	Master data files
<code>index-mapping-code*.json</code>	Index mapping configuration

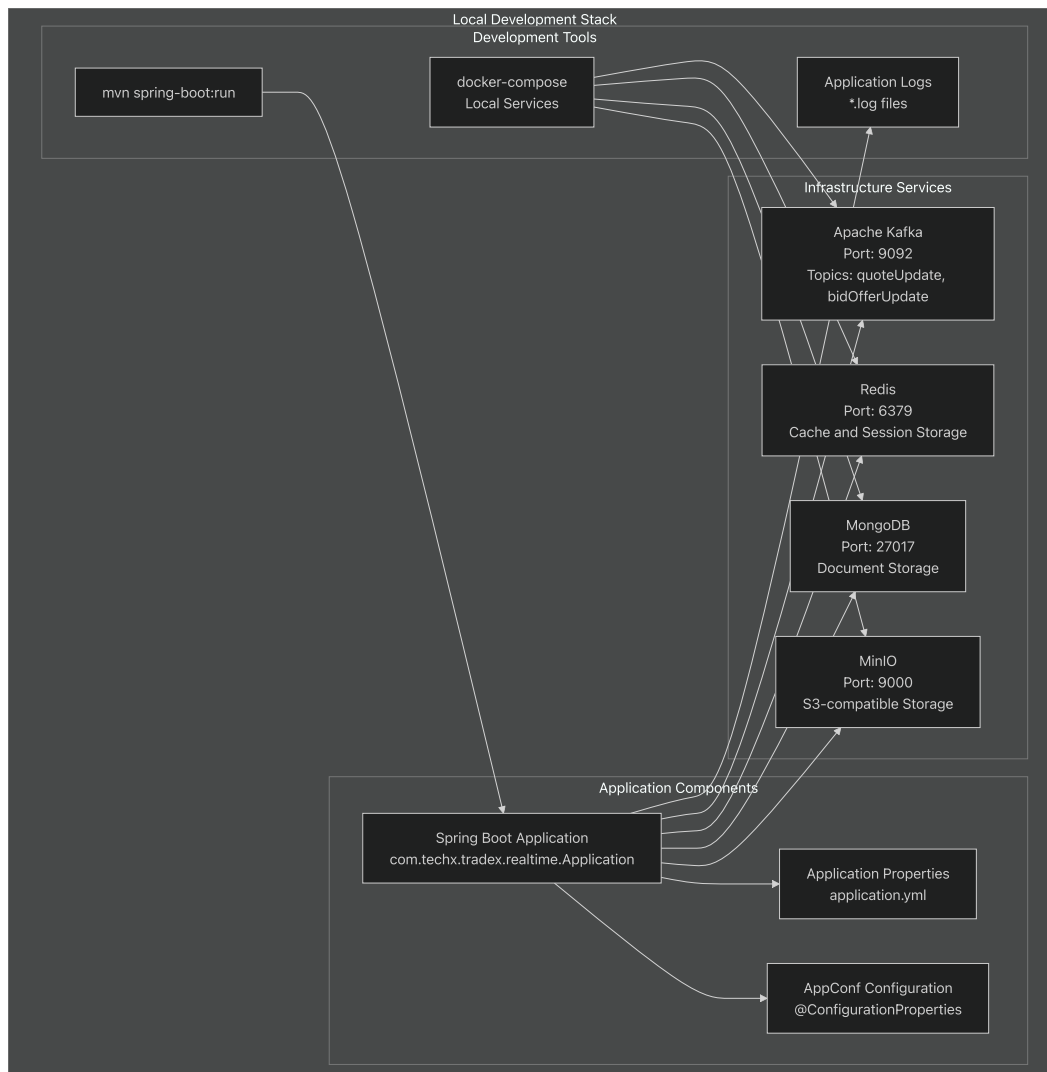
Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore#L4-L11>

[.gitignore4-11](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore#L4-L11) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/.gitignore#L4-L11>)

Local Development Setup

Environment Configuration

To run the application locally, developers need to set up the following components:



Build Commands

Common Maven commands for development:

Command	Purpose
<code>mvn clean compile</code>	Clean and compile source code
<code>mvn spring-boot:run</code>	Run application in development mode
<code>mvn test</code>	Run unit tests (currently skipped)
<code>mvn package</code>	Create executable JAR
<code>mvn dependency:tree</code>	Analyze dependency tree

Testing Configuration

The project has test execution disabled by default through Maven configuration:

```
<maven.test.skip>true</maven.test.skip>
```

This suggests the project relies on integration testing or external test suites rather than traditional unit tests.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L24-L24>.

[pom.xml#L24-L24](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L24-L24) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L24-L24>)

[pom.xml#L151-L161](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L151-L161) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L151-L161>)
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L151-L161>)

Development Workflow

Code Organization

The application follows Spring Boot conventions with component scanning configured for:

- `com.difisoft.market.*` - Market data processing components
- `com.techx.tradex.realtime.*` - Real-time processing components

This dual package structure indicates integration between different organizational codebases.

Dependency Management

The project uses Spring Boot's dependency management features with specific version overrides for:

- Java 17 compilation target
- Executable JAR generation
- Custom library integration

The build configuration supports both snapshot and release versions of custom dependencies, enabling development against the latest changes.

Sources: <https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L20-L25>.

[pom.xml#L20-L25](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L20-L25) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L20-L25>)

[pom.xml#L154-L159](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L154-L159) (<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L154-L159>)
([%5B](https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L154-L159))

pom.xml167-175](<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L167-L175>
(<https://github.com/nhsv-tradex/realtime-v2/blob/d6659ada/pom.xml#L167-L175>))
